



Ain Shams University

Faculty of Computer & Information Sciences

Artificial Intelligence Department.



# Shifaa :An AI-Powered Clinical Decision Support & Patient Monitoring System

*This documentation is submitted in partial fulfillment of the requirements for the  
Bachelor's degree in Computer and Information Sciences*

By

|                             |            |                           |
|-----------------------------|------------|---------------------------|
| <b>Omar Elsayed Ibrahim</b> | 2022170827 | [Artificial Intelligence] |
| <b>Omar Mohamed Adel</b>    | 2022170829 | [Artificial Intelligence] |
| <b>Ali Tarek Fekry</b>      | 2022170825 | [Artificial Intelligence] |
| <b>Hazem Mohamed</b>        | 2022170810 | [Artificial Intelligence] |
| <b>Abdelrhman Mahmoud</b>   | 2022170846 | [Artificial Intelligence] |

**Under Supervision of**

**Dr. Salsabil Amin**

Lecturer, Basic Science Department,  
Faculty of Computer and Information Sciences, Ain Shams University.

**T.A. Manar Sultan**

Teaching Assistant, Scientific Computing Department,  
Faculty of Computer and Information Sciences, Ain Shams University.

**June 2026**



# Acknowledgements

All praise and thanks are due to Allah, who granted us the strength, patience, and ability to complete this work. We hope this effort is accepted and made beneficial.

We are deeply grateful to our parents and families, whose endless help, encouragement, and support carried us throughout all the years of our study. We hope to one day repay even a fraction of what they have given us.

We offer our sincerest gratitude to our supervisors, Dr. Salsabil Amin and T.A. Manar Sultan, who guided us throughout this project with their patience, knowledge, and experience. Their continuous feedback and insightful advice shaped both the technical direction and the academic quality of this work.

We would also like to thank the faculty members and teaching staff of the Artificial Intelligence Department at the Faculty of Computer & Information Sciences, Ain Shams University, for the solid foundation they provided across machine learning, deep learning, software engineering, and database systems, knowledge that this project draws upon directly.

Finally, we thank our friends and colleagues who supported and encouraged us, and everyone who contributed, directly or indirectly, to bringing Shifaa to completion.

# Abstract

Traditional and current medical monitoring systems are often slow and impractical. Patient information is scattered across many disconnected modules of an Electronic Health Record, so retrieving a single fact (an allergy, a recent lab trend, the list of active drugs) is tedious and error-prone at the bedside. At the same time, the events that most threaten patients are easy to miss with limited staff. Falls are the second leading cause of unintentional-injury death worldwide, with roughly 684,000 fatal falls each year [1], and between 700,000 and one million patients fall in hospitals each year, with a large share causing injury [2]. Epilepsy affects about 50 million people globally [3], and seizures can be life-threatening when no one responds in time. Cardiovascular disease remains the leading cause of death worldwide, with an estimated 17.9 million deaths per year [4], and arrhythmias raise the risk of stroke and sudden cardiac death. Manual observation simply cannot watch every patient continuously.

Shifaa is built to solve these problems together in one platform. To make data retrieval fast, it unifies the patient record in a single database and places an AI assistant in front of it: a clinician asks a question in plain language and receives an accurate, source-cited answer in real time, powered by the MedGemma 1.5 medical language model. To catch dangerous events the moment they happen, Shifaa adds three real-time AI detectors (a video-based fall-detection model, a video-based seizure-detection model, and a two-stage ECG arrhythmia-detection model), each of which automatically raises an alert to the responsible staff.

The objectives were to deliver a secure, role-aware, room-centered clinical platform that (1) answers natural-language questions grounded strictly in real patient data, (2) detects falls, seizures, and arrhythmias in real time and routes every alert to the responsible staff, and (3) presents everything through one intuitive interface while keeping patient data private and local. These objectives were met. The conversational assistant reached 94.1% intent-classification accuracy and 94.0% entity-extraction F1 with a hallucination rate within 1%; the ECG cascade reached 96.10% and 94.92% accuracy on its two stages; the fall detector reached 91.59% accuracy with a 94.87% fall F1-score and 98.83% recall; and the seizure detector, built as a native C++/ONNX runtime paired with an asynchronous transformer worker, runs in real time at roughly 37 frames per second with a 96.69% segment AUROC. All of this is delivered as a microservices system: a FastAPI gateway, three independent inference services on dedicated ports, a relational database, and a React web application that organizes monitoring around hospital rooms.

In conclusion, Shifaa demonstrates that combining a grounded medical language model with real-time deep-learning detectors can make clinical information both instantly accessible and continuously

monitored, turning a slow, fragmented workflow into a fast, unified, and safer one, without sending any patient data outside the hospital.

# Table of Contents

|                                                                |    |
|----------------------------------------------------------------|----|
| Acknowledgements.....                                          | 1  |
| Abstract.....                                                  | 2  |
| Table of Contents.....                                         | 4  |
| List of Figures.....                                           | 8  |
| List of Tables.....                                            | 9  |
| List of Abbreviations.....                                     | 10 |
| Chapter 1: Introduction.....                                   | 13 |
| 1.1 Problem Definition.....                                    | 13 |
| 1.1.1 History.....                                             | 14 |
| 1.1.2 Applications.....                                        | 14 |
| 1.1.3 Project Scope.....                                       | 15 |
| 1.2 Motivation.....                                            | 15 |
| 1.3 Objectives.....                                            | 16 |
| 1.4 Time Plan.....                                             | 16 |
| 1.5 Documentation Outline.....                                 | 17 |
| Chapter 2: Background.....                                     | 20 |
| 2.1 Clinical Information Systems and Medical Data.....         | 20 |
| 2.2 Natural Language Processing and Large Language Models..... | 20 |
| 2.2.1 The Transformer and Self-Attention.....                  | 20 |
| 2.2.2 LLMs as Structured Classifiers and Extractors.....       | 21 |
| 2.2.3 Retrieval-Grounded Generation and Hallucination.....     | 21 |
| 2.2.4 MedGemma 1.5.....                                        | 22 |
| 2.3 Deep Learning for ECG Arrhythmia Detection.....            | 23 |
| 2.3.1 The Electrocardiogram.....                               | 23 |
| 2.3.2 Cascade Classification with CNN–BiLSTM–Attention.....    | 23 |
| 2.3.3 Datasets.....                                            | 24 |
| 2.4 Computer Vision for Activity and Fall Detection.....       | 24 |
| 2.4.1 Object Detection and Tracking (YOLO).....                | 24 |
| 2.4.2 Role Classification (MobileNetV3).....                   | 25 |
| 2.4.3 Pose Estimation (MediaPipe and OpenPose).....            | 25 |
| 2.4.4 Skeleton-Based Action Recognition and CTR-GCN.....       | 25 |

|                                                         |    |
|---------------------------------------------------------|----|
| 2.5 Video-Based Seizure Detection.....                  | 26 |
| 2.5.1 Clinical Background.....                          | 26 |
| 2.5.2 The VSViG Temporal Branch.....                    | 26 |
| 2.5.3 ViViT and Cross-Joint Attention.....              | 26 |
| 2.5.4 Series-Gate Fusion.....                           | 27 |
| 2.5.5 Why a Series Gate, Not a Single Classifier.....   | 27 |
| 2.6 Synthetic Clinical Data (Synthea).....              | 28 |
| 2.7 Web and Systems Technologies.....                   | 28 |
| 2.8 Evaluation Metrics.....                             | 28 |
| 2.9 Related Work and Positioning.....                   | 29 |
| 2.10 Ethics, Privacy, and Safety Considerations.....    | 30 |
| Chapter 3: Proposed System.....                         | 32 |
| 3.1 System Overview and Design Goals.....               | 32 |
| 3.2 High-Level Architecture.....                        | 33 |
| 3.3 Microservices Decomposition.....                    | 34 |
| 3.4 Data Model.....                                     | 35 |
| 3.4.1 Detailed Table Schemas.....                       | 37 |
| 3.5 Core Workflows.....                                 | 40 |
| 3.5.1 Conversational Query Resolution.....              | 40 |
| 3.5.2 Monitoring and Alert Publication.....             | 41 |
| 3.6 Security and Access Control.....                    | 42 |
| 3.7 Design Decisions and Trade-offs.....                | 42 |
| Chapter 4: System Implementation.....                   | 44 |
| 4.1 Backend Gateway and Alert Manager.....              | 45 |
| 4.1.1 Authentication and Authorization.....             | 46 |
| 4.1.2 Patient, User, and Room Routers.....              | 46 |
| 4.1.3 Room Status Aggregation.....                      | 47 |
| 4.2 Conversational AI Pipeline.....                     | 48 |
| 4.2.1 Stage 1, Intent Classification.....               | 48 |
| 4.2.2 Stage 2, Medical Named-Entity Recognition.....    | 49 |
| 4.2.3 Stage 3, Database Retrieval.....                  | 49 |
| 4.2.4 Stage 4, Grounded Answer Generation.....          | 50 |
| 4.2.5 Streaming and the Reasoning (<think>) Parser..... | 51 |

|                                                            |    |
|------------------------------------------------------------|----|
| 4.2.6 Multimodal Image Analysis.....                       | 51 |
| 4.2.7 Results and Discussion, Conversational Pipeline..... | 52 |
| 4.3 ECG Arrhythmia Detection Cascade.....                  | 54 |
| 4.3.1 Results and Discussion, Arrhythmia Cascade.....      | 55 |
| 4.4 Patient Fall Detection Pipeline.....                   | 56 |
| 4.4.1 Results and Discussion, Fall Pipeline.....           | 57 |
| 4.5 Real-Time Seizure Detection Pipeline.....              | 58 |
| 4.5.1 Results and Discussion, Seizure Pipeline.....        | 59 |
| 4.6 Frontend Single-Page Application.....                  | 61 |
| 4.7 Data Import Pipeline.....                              | 63 |
| 4.8 Integration and Inter-Service Communication.....       | 64 |
| 4.8.1 Arrhythmia Client.....                               | 65 |
| 4.8.2 Fall Camera Bridge.....                              | 65 |
| 4.8.3 Seizure Event Relay and Internal IPC.....            | 65 |
| Chapter 5: System Testing.....                             | 67 |
| 5.1 Test Environment and Prerequisites.....                | 68 |
| 5.2 Installation and Running the System.....               | 68 |
| 5.2.1 Backend, Database, and Services.....                 | 68 |
| 5.2.2 Frontend.....                                        | 69 |
| 5.3 User-Interface Walkthrough.....                        | 70 |
| 5.3.1 Login and Role-Based Entry.....                      | 70 |
| 5.3.2 Doctor Dashboard.....                                | 71 |
| 5.3.3 Patient Hub.....                                     | 72 |
| 5.3.4 Registering a New Patient.....                       | 73 |
| 5.3.5 Patient Detail: Overview.....                        | 74 |
| 5.3.6 Patient Detail: Specialized Chat.....                | 75 |
| 5.3.7 Patient Detail: Monitoring History.....              | 77 |
| 5.3.8 Global Chat.....                                     | 77 |
| 5.3.9 Live Monitoring: Ward Central Station.....           | 79 |
| 5.3.10 Room Monitoring View.....                           | 79 |
| 5.3.11 Model Testing Sandbox (Demonstration).....          | 80 |
| 5.3.12 Nurse Portal.....                                   | 83 |
| 5.3.13 Admin Portal: System, Users, and Rooms.....         | 84 |



|                                                      |     |
|------------------------------------------------------|-----|
| 5.4 Automated Test Suites.....                       | 87  |
| 5.5 API Reference Summary.....                       | 89  |
| 5.6 Consolidated Benchmark Results.....              | 90  |
| 5.7 Troubleshooting.....                             | 91  |
| 5.8 Production Deployment.....                       | 92  |
| Chapter 6: Conclusion and Future Work.....           | 93  |
| 6.1 Conclusion.....                                  | 94  |
| 6.2 Future Work.....                                 | 95  |
| Tools.....                                           | 97  |
| Hardware Tools.....                                  | 97  |
| Software Tools.....                                  | 97  |
| Appendix A: Environment Configuration Reference..... | 99  |
| Appendix B: Detailed API Examples.....               | 101 |
| B.1 Login.....                                       | 101 |
| B.2 Patient Record.....                              | 101 |
| B.3 Arrhythmia Analysis.....                         | 101 |
| B.4 Create and Configure a Room.....                 | 101 |
| B.5 Acknowledge an Alert.....                        | 102 |
| B.6 Dashboard Summary.....                           | 102 |
| Appendix C: Glossary of Clinical Terms.....          | 103 |
| References.....                                      | 104 |

# List of Figures

|                                                                                                       |    |
|-------------------------------------------------------------------------------------------------------|----|
| Figure 1: Shifaa high-level architecture (gateway, inference services, and data tier)                 | 34 |
| Figure 2: Conversational query-resolution workflow                                                    | 41 |
| Figure 3: Monitoring and alert lifecycle workflow                                                     | 42 |
| Figure 4: Two-stage ECG arrhythmia cascade                                                            | 54 |
| Figure 5: Four-stage fall-detection pipeline                                                          | 56 |
| Figure 6: Real-time hybrid C++/ONNX + Python seizure-detection pipeline                               | 58 |
| Figure 7: Login screen with role-based demo credentials                                               | 70 |
| Figure 8: Doctor Clinical Dashboard: alert counters, active-alert feed, and service health            | 71 |
| Figure 9: Patient hub: searchable patient list with demographics and alert status                     | 72 |
| Figure 10: Register New Patient modal                                                                 | 73 |
| Figure 11: Patient detail: Overview tab (active diagnoses and current medications)                    | 74 |
| Figure 12: Patient-grounded specialized chat with a cited medical-history answer                      | 75 |
| Figure 13: Specialized chat: live reasoning (thinking) block                                          | 76 |
| Figure 14: Patient detail: Monitoring history (per-patient alert audit trail)                         | 77 |
| Figure 15: Global Chat: multi-modal MRI upload and AI analysis                                        | 78 |
| Figure 16: Global Chat: grounded refusal and reasoning block                                          | 78 |
| Figure 17: Ward Central Monitoring Station (doctor view of all rooms)                                 | 79 |
| Figure 18: Single-room monitoring view (camera relay, live ECG, and diagnostics log)                  | 80 |
| Figure 19: Sandbox, ECG arrhythmia cascade: live trace with Stage 1/Stage 2 results and probabilities | 81 |
| Figure 20: Sandbox, fall detection: CTR-GCN fall confirmed with active alert latch                    | 82 |
| Figure 21: Sandbox, seizure detection: series-gate score crossing threshold with latch                | 82 |
| Figure 22: Nurse Dashboard: alerts requiring attention and assigned patients                          | 83 |
| Figure 23: Nurse Live Monitor: ward station scoped to assigned rooms                                  | 84 |
| Figure 24: Admin System Dashboard: backend and inference-service health                               | 85 |
| Figure 25: Admin User Management                                                                      | 85 |
| Figure 26: Admin Room Configuration Management                                                        | 86 |
| Figure 27: Admin: Edit Room dialog (services and nurse assignment)                                    | 87 |
| Figure 28: Admin: Create Room dialog                                                                  | 87 |

# List of Tables

|                                                                                 |    |
|---------------------------------------------------------------------------------|----|
| Table 1: Project time plan (Gantt chart)                                        | 17 |
| Table 2: Selected MedGemma 1.5 4B benchmark results (higher is better)          | 22 |
| Table 3: Public ECG datasets used for the arrhythmia cascade                    | 24 |
| Table 4: Positioning of the chatbot approach against baselines                  | 30 |
| Table 5: Shifaa services, ports, and responsibilities                           | 34 |
| Table 6: Principal database tables                                              | 36 |
| Table 7: Roles and their capabilities                                           | 42 |
| Table 8: Key design decisions and their rationale                               | 43 |
| Table 9: Backend routers and their endpoints                                    | 47 |
| Table 10: The eight supported query intents                                     | 48 |
| Table 11: Retrieval strategy by intent                                          | 49 |
| Table 12: Conversational pipeline evaluation summary                            | 52 |
| Table 13: Per-class intent classification results (Ollama backend)              | 52 |
| Table 14: Per-entity NER results                                                | 52 |
| Table 15: Dual-branch ECG model architecture (per stage)                        | 54 |
| Table 16: Binary screening confusion matrix (Stage 1)                           | 55 |
| Table 17: Sub-type classifier per-class F1 (Stage 2)                            | 55 |
| Table 18: Fall classifier results (72-frame sliding window, threshold 0.50)     | 57 |
| Table 19: Fall-detection model weights                                          | 57 |
| Table 20: Series-gate constants and clinical sensitivity modes                  | 59 |
| Table 21: Seizure detector, segment-level classification metrics (952 segments) | 60 |
| Table 22: Seizure detector, clinical runtime evaluation (10 videos)             | 60 |
| Table 23: Principal frontend pages by role                                      | 62 |
| Table 24: Synthea-to-schema mapping (imported tables)                           | 63 |
| Table 25: Synthea export files and import status                                | 63 |
| Table 26: Data-quality handling during import                                   | 64 |
| Table 27: Inference-service interfaces and failure handling                     | 66 |
| Table 28: Model Testing Sandbox: cases and models                               | 80 |
| Table 29: Representative retriever test cases                                   | 88 |
| Table 30: Principal API endpoints                                               | 89 |
| Table 31: Consolidated system benchmark results                                 | 90 |
| Table 32: Chat end-to-end latency by backend                                    | 90 |
| Table 33: Advantages and limitations of Shifaa                                  | 94 |

# List of Abbreviations

| Abbreviation | Meaning                                                      |
|--------------|--------------------------------------------------------------|
| AF           | Atrial Fibrillation                                          |
| AI           | Artificial Intelligence                                      |
| AMP          | Automatic Mixed Precision                                    |
| API          | Application Programming Interface                            |
| AP (sum)     | Accumulated Probability (rolling sum)                        |
| AUROC        | Area Under the Receiver Operating Characteristic Curve       |
| BiLSTM       | Bidirectional Long Short-Term Memory                         |
| CJ           | Cross-Joint (Attention)                                      |
| CNN          | Convolutional Neural Network                                 |
| CORS         | Cross-Origin Resource Sharing                                |
| CTR-GCN      | Channel-wise Topology Refinement Graph Convolutional Network |
| CT           | Computed Tomography                                          |
| ECG / EKG    | Electrocardiogram                                            |
| EHR          | Electronic Health Record                                     |
| EMU          | Epilepsy Monitoring Unit                                     |
| F1           | F1-score (harmonic mean of precision and recall)             |
| FAR          | False Alert Rate                                             |
| FHIR         | Fast Healthcare Interoperability Resources                   |
| FPS          | Frames Per Second                                            |
| FTL          | First-Token Latency                                          |
| GCN          | Graph Convolutional Network                                  |
| GQA          | Grouped-Query Attention                                      |
| HMR          | Hot Module Replacement                                       |
| IAVB         | First-Degree Atrioventricular Block                          |
| ICD-10       | International Classification of Diseases, 10th Revision      |
| IMU          | Inertial Measurement Unit                                    |
| IPC          | Inter-Process Communication                                  |
| JWT          | JSON Web Token                                               |
| LLM          | Large Language Model                                         |

| Abbreviation | Meaning                                                                                     |
|--------------|---------------------------------------------------------------------------------------------|
| LCO / LEO    | Latency to Clinical Onset / Latency to EEG Onset                                            |
| MRI          | Magnetic Resonance Imaging                                                                  |
| NER          | Named-Entity Recognition                                                                    |
| NTU          | Nanyang Technological University (skeleton joint format)                                    |
| ONNX         | Open Neural Network Exchange (model format / runtime)                                       |
| ORM          | Object-Relational Mapping                                                                   |
| OCR          | Optical Character Recognition                                                               |
| PAF          | Part Affinity Field                                                                         |
| RDBMS        | Relational Database Management System                                                       |
| RBAC         | Role-Based Access Control                                                                   |
| ROI          | Region of Interest                                                                          |
| SB           | Sinus Bradycardia                                                                           |
| SPA          | Single-Page Application                                                                     |
| SQL          | Structured Query Language                                                                   |
| SSE          | Server-Sent Events                                                                          |
| STach        | Sinus Tachycardia                                                                           |
| TRT          | Total Response Time                                                                         |
| VSViG        | Video-Based Seizure detection via Skeleton-Based Spatiotemporal Vision Graph Neural Network |
| ViViT        | Video Vision Transformer                                                                    |
| VSViG        | Video-based Skeleton Vision Graph (network)                                                 |
| WSI          | Whole-Slide Imaging                                                                         |
| YOLO         | You Only Look Once (object detector)                                                        |

# Chapter One

## Introduction

---

- 1.1 Problem Definition
- 1.2 Motivation
- 1.3 Objectives
- 1.4 Time Plan
- 1.5 Documentation Outline

# Chapter 1: Introduction

Modern hospitals run on data. Every patient generates a continuous stream of structured records (diagnoses, prescriptions, laboratory results, allergies, and clinical encounters) alongside live physiological signals such as the electrocardiogram and bedside video. Two persistent challenges arise from this abundance of data. First, the information a clinician needs is rarely in one place: answering a simple bedside question can require navigating several screens of an Electronic Health Record (EHR). Second, the events that matter most for patient safety (a dangerous arrhythmia, an unwitnessed fall, or the onset of a seizure) happen in real time and are easily missed when staff are stretched thin. Shifaa is a graduation project that tackles both challenges within a single, coherent clinical platform.

This chapter introduces the problem the project addresses, the motivation behind it, the concrete objectives the team set out to achieve, the time plan that guided development, and a road-map to the remainder of this document.

## 1.1 Problem Definition

The project can be framed around two complementary problems that together define the clinical workflow of an inpatient ward:

1. **Information retrieval.** A clinician needs fast, trustworthy answers to natural-language questions about a specific patient (“Is she allergic to penicillin?”, “What was the last HbA1c?”, “Which medications are currently active?”). Traditional EHR interfaces force the user to remember which module holds which fact and to perform multiple clicks, which is slow and error-prone in time-critical situations.
2. **Continuous patient safety monitoring.** Several high-risk events (cardiac arrhythmias, falls, and seizures) must be detected the moment they occur. Manual observation does not scale, and conventional threshold alarms produce either too many false alerts or miss subtle events entirely.

Shifaa therefore unifies a conversational history-retrieval assistant and three automated monitoring subsystems under one role-aware platform, so that the same clinician can both query a patient’s record in plain language and receive automated, AI-generated safety alerts for that patient.

### 1.1.1 History

Clinical decision-support systems (CDSS) have evolved over five decades. Early rule-based expert systems such as MYCIN (1970s) demonstrated that computers could encode medical reasoning, but their hand-crafted rules were brittle and hard to maintain. The 1990s and 2000s brought structured EHRs and the standardization of clinical vocabularies (ICD-10, SNOMED CT, LOINC), which made patient data machine-readable but did little to make it conversational. The 2010s saw the rise of deep learning: convolutional neural networks reached expert-level performance on medical imaging, and recurrent networks were applied to physiological time-series such as the ECG.

The most recent shift, and the one this project builds upon, is the arrival of Large Language Models (LLMs). General-purpose models such as the GPT and Gemma families, and medical-specialized derivatives such as Google's MedGemma, can read and write clinical text fluently, interpret medical images, and follow detailed instructions. This makes it feasible (for the first time) to build a natural-language interface to a patient record that is both accurate and grounded in real data. In parallel, advances in real-time computer vision (single-stage detectors, pose estimation, and graph convolutional networks for skeleton-based action recognition) have made continuous, camera-based monitoring of physical events such as falls and seizures practical on commodity hardware.

### 1.1.2 Applications

A platform that combines grounded clinical question-answering with real-time event detection has direct application across several care settings:

- Inpatient wards, rapid bedside history lookup during rounds, plus continuous fall monitoring for elderly or post-operative patients.
- Cardiac care and telemetry units, automated screening of 12-lead ECG signals for arrhythmias, flagging abnormal rhythms for clinician review.
- Epilepsy Monitoring Units (EMUs), non-invasive, video-based seizure detection that complements electroencephalography and reduces the burden of manual video review.
- Emergency departments, instant allergy and medication checks before administering drugs, reducing adverse-event risk.
- Medical education and research, a privacy-safe sandbox, populated with synthetic Synthea data, for experimenting with clinical AI without exposing real patient information.



### 1.1.3 Project Scope

To keep the project achievable within an academic year while still delivering a complete, demonstrable system, its scope was defined explicitly. The table below states what is in and out of scope.

| In scope                                          | Out of scope                                         |
|---------------------------------------------------|------------------------------------------------------|
| Doctor-facing natural-language history retrieval. | Patient-facing or public-facing interfaces.          |
| Grounded answering over a structured EHR.         | Autonomous diagnosis or treatment decisions.         |
| Arrhythmia, fall, and seizure detection.          | Non-motor seizures and unlisted arrhythmia types.    |
| Role-based access for doctor / nurse / admin.     | Hospital-wide identity federation (SSO/LDAP).        |
| Local-first deployment with synthetic data.       | Clinical certification and regulatory approval.      |
| Real-time alerting and dashboards.                | Long-term analytics and population health reporting. |

Within this scope, the team aimed for a vertically complete system: from data import, through model inference, to a polished user interface, rather than a single component studied in isolation. This breadth is itself a deliberate contribution, demonstrating how disparate AI techniques can be integrated into one coherent clinical product.

## 1.2 Motivation

Three observations motivated the team to build Shifaa:

- **The cost of fragmented information.** Studies of clinical workflow repeatedly show that clinicians spend a large fraction of their time searching for information rather than treating patients. A single natural-language entry point that returns a cited, trustworthy answer can reclaim much of that time.
- **Patient safety and the limits of human vigilance.** Falls and seizures frequently occur without a witness, and dangerous arrhythmias can be transient. Automated, always-on detection provides a safety net that human staff cannot maintain continuously.
- **Privacy and the local-first opportunity.** Patient data is among the most sensitive of all personal data. The recent availability of capable open medical models that run locally (through Ollama) means a clinical assistant can be built that never sends patient information to a third-party cloud, a decisive advantage for real-world deployment.

Equally, the project was an opportunity for the team to apply, end-to-end, the full breadth of an Artificial Intelligence curriculum: natural-language processing, deep learning for signals and vision, graph neural networks, software architecture, databases, and human–computer interaction.

## 1.3 Objectives

The project pursues the following concrete objectives:

1. Build a secure, role-aware backend that authenticates doctors, nurses, and administrators and isolates each clinician’s patients.
2. Implement a multi-stage LLM pipeline that classifies a doctor’s query into one of eight clinical intents and extracts the medical entities it contains.
3. Retrieve only the relevant patient records from a relational database based on the detected intent and entities, and generate a strictly grounded, source-cited answer.
4. Stream answers to the clinician in real time over WebSocket, exposing the model’s clinical reasoning in an expandable view.
5. Support multi-modal analysis of medical images (chest X-ray, CT/MRI, lab reports, dermatology, handwritten notes) through the same assistant.
6. Detect cardiac arrhythmias from 12-lead ECG signals using a two-stage deep-learning cascade.
7. Detect patient falls from a video feed using a four-stage computer-vision pipeline.
8. Detect epileptic seizures from bedside video in real time using a fused skeleton-based detector with configurable clinical sensitivity.
9. Persist every AI-generated alert, broadcast it live to the relevant staff, and present it on a clinical dashboard.
10. Deliver an intuitive, responsive React single-page application for all of the above.

## 1.4 Time Plan

Development followed an incremental plan across the academic year, organized into phases that overlapped where dependencies allowed. The Gantt chart below summarizes the schedule; a filled cell (■) indicates active work on a task during that month.

Table 1: Project time plan (Gantt chart)

| Task / Month                              | Oct | Nov | Dec | Jan | Feb | Mar | Apr | May | Jun |
|-------------------------------------------|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| Research & literature review              | ■   | ■   |     |     |     |     |     |     |     |
| Dataset preparation (Synthea, ECG, video) |     | ■   | ■   |     |     |     |     |     |     |
| Backend & database design                 |     |     | ■   | ■   |     |     |     |     |     |
| LLM chatbot pipeline                      |     |     |     | ■   | ■   | ■   |     |     |     |
| Arrhythmia detection cascade              |     |     | ■   | ■   | ■   |     |     |     |     |
| Fall detection pipeline                   |     |     |     |     | ■   | ■   | ■   |     |     |
| Seizure detection pipeline                |     |     |     |     | ■   | ■   | ■   | ■   |     |
| React frontend SPA                        |     |     |     |     |     | ■   | ■   | ■   |     |
| Integration & microservices               |     |     |     |     |     |     | ■   | ■   |     |
| Testing, evaluation & benchmarking        |     |     |     |     |     |     |     | ■   | ■   |
| Documentation                             |     |     |     |     |     |     |     | ■   | ■   |

## 1.5 Documentation Outline

The remainder of this documentation is organized into the following chapters:

- **Chapter 2, Background:** the literature, theory, algorithms, and technologies the project builds upon, from clinical NLP and LLMs to deep learning for signals, computer vision, and graph neural networks.
- **Chapter 3, Proposed System:** the overall architecture and design of Shifaa, its microservices, data model, and the workflows that connect its components.
- **Chapter 4, System Implementation:** a component-by-component account of how each subsystem was built, expressed through flowcharts and pseudocode, together with each component's results and discussion.
- **Chapter 5, System Testing:** how to install, run, and test the platform, with walkthroughs and screenshots of the user interface, plus the automated test suites and benchmark results.
- **Chapter 6, Conclusion and Future Work:** a summary of what was achieved, the advantages and limitations of the system, and directions for future development.

- **Tools and References:** the hardware and software tools used, followed by the cited literature and resources.

## Chapter Two

# Background

---

- 2.1 Clinical Information Systems
- 2.2 Natural Language Processing & Large Language Models
- 2.3 Deep Learning for ECG Arrhythmia Detection
- 2.4 Computer Vision for Activity & Fall Detection
- 2.5 Video-Based Seizure Detection
- 2.6 Synthetic Clinical Data
- 2.7 Web & Systems Technologies

# Chapter 2: Background

This chapter surveys the concepts, algorithms, and technologies that Shifaa is built upon. It begins with clinical information systems and the medical data standards the project consumes, then reviews the natural-language-processing and Large Language Model techniques behind the conversational assistant. It continues with the deep-learning and computer-vision foundations of the three monitoring subsystems (arrhythmia, fall, and seizure detection) before closing with the synthetic-data tooling and the web and systems technologies that tie the platform together.

## 2.1 Clinical Information Systems and Medical Data

An Electronic Health Record (EHR) is the digital record of a patient’s health information, maintained over time by one or more care providers. A typical record is highly relational: a single patient links to many diagnoses, medications, laboratory observations, allergies, and clinical encounters (visits). To be interoperable, these records rely on standardized vocabularies. The International Classification of Diseases, 10th revision (ICD-10) codes diagnoses; SNOMED CT provides a comprehensive clinical terminology; LOINC standardizes laboratory tests; and FHIR (Fast Healthcare Interoperability Resources) defines a modern, web-friendly exchange format. Shifaa models a normalized subset of this structure in a relational database and uses ICD-10-style codes for diagnoses.

Clinical Decision Support Systems (CDSS) sit on top of the EHR to assist clinical judgment. They range from simple drug–allergy alerts to sophisticated diagnostic reasoners. Shifaa is, in essence, a modern CDSS: it does not make decisions for the clinician, but it retrieves and presents the right information at the right moment and flags safety-critical events for human review.

## 2.2 Natural Language Processing and Large Language Models

### 2.2.1 The Transformer and Self-Attention

Virtually all modern LLMs are based on the Transformer architecture [5]. The Transformer replaced the sequential recurrence of earlier models with the self-attention mechanism, which lets every token in a sequence attend directly to every other token. For each token, the model computes a query, a key, and a value vector; the attention weight between two tokens is the scaled dot-product of one token’s query with another’s key, normalized by a softmax. The output for a token is the attention-weighted sum of all value

vectors. Stacking many such attention layers, interleaved with feed-forward networks and residual connections, yields a model that captures long-range dependencies efficiently and trains well in parallel.

Decoder-only Transformers (the family that includes GPT, Gemma, and MedGemma) are trained to predict the next token given all previous tokens. Despite this simple objective, scaling these models to billions of parameters and trillions of tokens produces emergent abilities: instruction following, in-context learning, structured (JSON) output, and multi-step reasoning. Two refinements relevant to this project are Grouped-Query Attention (GQA), which reduces the memory cost of attention at inference, and long-context support, which lets a model condition on tens of thousands of tokens of patient data at once.

### 2.2.2 LLMs as Structured Classifiers and Extractors

A key design idea in Shifaa's chatbot is to use a general-purpose LLM not as a free-form chat partner but as a deterministic, structured component. By constraining the model with a strict system prompt and requesting JSON-formatted output at temperature zero, the same model can act as: (a) an intent classifier that maps a free-text query to one of a fixed set of labels, and (b) a named-entity extractor that pulls out structured fields (drug names, lab tests, conditions, date ranges, and numeric limits). This avoids the need to train and maintain bespoke classification or NER models, while retaining the flexibility and paraphrase-robustness of a large model. The trade-off (latency and the need for careful prompt engineering) is acceptable for an interactive clinical tool.

### 2.2.3 Retrieval-Grounded Generation and Hallucination

LLMs are prone to hallucination: generating fluent text that is not supported by any source. In a clinical setting this is unacceptable. The standard mitigation is retrieval-grounded generation, in which the model is given the relevant source documents and instructed to answer only from them. Shifaa applies a strict form of this pattern: rather than retrieving free text, it queries a structured database for exactly the records implied by the user's intent and entities, formats those records into a compact context block, and instructs the model to answer only from that block, to cite the record each fact comes from, and to reply with a fixed "the records do not contain this information" sentence when the data is absent. This converts the open-ended generation problem into a tightly bounded, auditable one.

### 2.2.4 MedGemma 1.5

MedGemma 1.5 is the medical LLM at the heart of Shifaa's assistant [6]. Developed by Google, it is a 4-billion-parameter, multimodal, instruction-tuned variant of the Gemma 3 family [7], further trained on

medical text and images. It uses the same decoder-only Transformer architecture as Gemma 3, with grouped-query attention and support for at least 128K tokens of context. Inputs may be text or images (normalized to 896×896 and encoded to 256 tokens each); output is text up to roughly 8,192 tokens. A SigLIP image encoder pre-trained on de-identified medical images gives the model its visual abilities across chest X-rays, dermatology, ophthalmology, and histopathology.

MedGemma 1.5 4B extends its predecessor with several capabilities directly relevant to this project: interpretation of 3-D CT and MRI volumes, whole-slide histopathology, longitudinal chest-X-ray comparison, anatomical localization via bounding boxes, structured extraction from unstructured lab reports, and text-based EHR understanding. The table below summarizes representative benchmark results reported in the MedGemma 1.5 technical report, illustrating its strength on the EHR-understanding and document-extraction tasks that matter most here.

*Table 2: Selected MedGemma 1.5 4B benchmark results (higher is better)*

| Task / Dataset                             | Metric   | Gemma 3 4B | MedGemma 1.5 4B |
|--------------------------------------------|----------|------------|-----------------|
| MedQA (4-option)                           | Accuracy | 50.7       | 69.1            |
| MedMCQA                                    | Accuracy | 45.4       | 59.8            |
| EHRQA (EHR understanding)                  | Accuracy | 70.9       | 89.6            |
| EHRNoteQA                                  | Accuracy | 78.0       | 80.4            |
| PDF-to-JSON lab extraction (EHR Dataset 2) | Macro F1 | 84.0       | 91.0            |
| MIMIC-CXR (top-5 conditions)               | Macro F1 | 81.2       | 89.5            |
| EyePACS (fundus)                           | Accuracy | 14.4       | 76.8            |

Two further architectural details of MedGemma 1.5 are used directly by the platform. First, the model emits its private chain-of-thought reasoning between two special tokens (an opening reasoning token and a closing reasoning token) before its final answer; Shifaa parses these to drive an expandable “thinking” view in the interface. Second, because MedGemma may be over-confident and is explicitly not intended to make autonomous clinical decisions, the platform always keeps a human in the loop and treats model output as preliminary, in line with the model’s stated limitations.



## 2.3 Deep Learning for ECG Arrhythmia Detection

### 2.3.1 The Electrocardiogram

The electrocardiogram (ECG) records the heart's electrical activity over time. A standard clinical ECG uses 12 leads, each offering a different spatial view of the heart's depolarization and repolarization. A normal beat shows a characteristic P wave, QRS complex, and T wave; arrhythmias distort the timing, morphology, or regularity of these components. Shifaa's arrhythmia module operates on 10-second, 12-lead recordings sampled at 500 Hz, i.e. arrays of shape  $5000 \times 12$ .

### 2.3.2 Cascade Classification with CNN–BiLSTM–Attention

Detecting and sub-typing arrhythmias is naturally hierarchical: most recordings are normal, and only the minority that are abnormal need fine-grained classification. Shifaa exploits this with a two-stage cascade. Stage 1 is a high-sensitivity binary screen (Normal vs. Abnormal); only signals flagged abnormal proceed to Stage 2, which assigns one of four clinical sub-types. This cascade design concentrates the harder, multi-class decision on the small fraction of abnormal signals, improving both accuracy and efficiency.

Each stage uses a dual-branch neural network. A waveform branch processes the raw 12-lead signal through three 1-D convolutional blocks (each combining convolution, batch normalization, a non-linearity, max-pooling, and dropout) to extract local morphological features, followed by a bidirectional Long Short-Term Memory (BiLSTM) network that models temporal dependencies in both directions, and a self-attention pooling layer that summarizes the sequence into a fixed context vector. A parallel auxiliary branch ingests seven hand-crafted QRS/heart-rate features (mean and standard deviation of the R-R interval, QRS amplitude statistics, mean heart rate, R-R coefficient of variation, and beats per second) through a small multilayer perceptron. The two contexts are merged with a gated fusion module and passed to a classifier head. Each stage has roughly 1.12 million trainable parameters, making it lightweight enough for near-real-time inference.

Convolutional neural networks (CNNs) learn translation-invariant local patterns and are well suited to detecting waveform morphology; LSTMs capture sequential rhythm context; and attention lets the network focus on the diagnostically salient portions of the beat. Combining all three, plus explicit rhythm statistics, gives the cascade a strong inductive bias for the ECG domain.

### 2.3.3 Datasets

The arrhythmia models were trained on a merged corpus drawn from three widely used public ECG datasets: PTB-XL [8], the China Physiological Signal Challenge / CINC2020 collection [9], and CODE-15 [10]. To prevent data leakage between training, validation, and test sets, records that appear in more than one source (for example, PTB-XL records duplicated inside CINC2020) were grouped by a common identifier, and CODE-15 records were grouped by patient, so that no patient or recording crosses split boundaries. Fixed train/validation/test splits were used rather than cross-validation, with the split manifests retained for reproducibility.

*Table 3: Public ECG datasets used for the arrhythmia cascade*

| Dataset                        | Origin                                          | Role                                                        |
|--------------------------------|-------------------------------------------------|-------------------------------------------------------------|
| PTB-XL                         | Physikalisch-Technische Bundesanstalt (Germany) | Large 12-lead diagnostic corpus; primary source.            |
| CINC2020 / PhysioNet Challenge | Multi-institution challenge collection          | Diverse rhythm labels; cross-checked for PTB-XL duplicates. |
| CODE-15                        | Telehealth Network of Minas Gerais (Brazil)     | Population-scale 12-lead records grouped by patient.        |

## 2.4 Computer Vision for Activity and Fall Detection

The fall-detection subsystem is a pipeline of four computer-vision models, each addressing a distinct sub-problem. This section reviews the techniques behind each stage.

### 2.4.1 Object Detection and Tracking (YOLO)

Object detection localizes and classifies objects in an image. The YOLO (“You Only Look Once”) family performs detection in a single forward pass, predicting bounding boxes and class probabilities directly on a grid, which makes it fast enough for real-time video. Shifaa uses YOLOv8 [11] to detect people in each frame. To keep a consistent identity for each person across frames, the detector is paired with a multi-object tracker ByteTrack [12], which associates detections over time using motion and appearance cues. Tracking is essential so that the system can follow one patient’s motion through a sliding window of frames.

### 2.4.2 Role Classification (MobileNetV3)

Not everyone in a hospital-room camera view is the patient: nurses, doctors, and visitors also appear. To avoid wasting computation (and raising false alarms) on non-patients, each detected person crop is passed to a lightweight image classifier that labels it as patient, medical staff, or other. Shifaa uses MobileNetV3-Large [13], an efficient CNN designed for mobile and edge devices through depthwise-separable convolutions, squeeze-and-excitation attention blocks, and a hardware-aware architecture search. Only crops classified as the patient continue down the pipeline.

### 2.4.3 Pose Estimation (MediaPipe and OpenPose)

Human pose estimation extracts the 2-D or 3-D coordinates of body joints. The fall pipeline uses Google's MediaPipe Pose [14], which returns 33 body landmarks per person in real time. These are remapped to the 25-joint, NTU-style skeleton expected by the downstream action-recognition network. (The seizure subsystem, discussed next, instead uses a lightweight OpenPose-18 model [15] tuned for the clinical bedside view.) Representing a person by a compact skeleton, rather than raw pixels, makes the subsequent motion model robust to appearance, lighting, and background, and greatly reduces its input dimensionality.

### 2.4.4 Skeleton-Based Action Recognition and CTR-GCN

A skeleton sequence is naturally a spatio-temporal graph: joints are nodes connected by bones (spatial edges) and by their own trajectory over time (temporal edges). Graph Convolutional Networks (GCNs) generalize convolution to such graphs and have become the dominant approach to skeleton-based action recognition. Shifaa's fall classifier is a Channel-wise Topology Refinement Graph Convolutional Network (CTR-GCN) [16], which improves on earlier GCNs by learning a distinct, refined adjacency (graph topology) for each feature channel, allowing different joint relationships to be emphasized for different features. The classifier consumes a sliding window of 72 skeleton frames (about four seconds of motion at 18 frames per second) centered on the moment of impact during training, where impact is localized from synchronized inertial-measurement-unit (IMU) acceleration peaks. It outputs a binary fall / non-fall decision.

## 2.5 Video-Based Seizure Detection

### 2.5.1 Clinical Background

Epileptic seizures with motor manifestations produce characteristic, often violent, movements. In an Epilepsy Monitoring Unit (EMU), patients are observed by video and electroencephalography (EEG) for days to capture and characterize seizures. Manual review of this footage is labor-intensive. A video-based detector that runs in real time can alert staff the moment a motor seizure begins, improving safety and reducing review burden. Shifaa's seizure subsystem is designed for exactly this setting and detects visible motor manifestations from a single bedside camera.

To meet the real-time requirement, the subsystem is deployed as a hybrid native-and-managed runtime: the high-frequency work (video decoding, pose extraction, the live temporal branch, and the fusion gate) runs natively in C++ on ONNX Runtime [21], while the heavy asynchronous transformer runs in a parallel Python process and communicates with the C++ engine over an operating-system named pipe. This separation is what allows the detector to sustain real-time throughput; the underlying deep-learning techniques are reviewed in the following subsections, and the deployment is detailed in Chapters 3 and 4.

### 2.5.2 The VSViG Temporal Branch

The live branch of the detector is VSViG (Video-based Skeleton Vision Graph network) [17]. It extracts a small set of clinically relevant joints with OpenPose-18, builds Gaussian image patches around each joint, and processes the resulting skeleton-plus-patch representation with a Vision Graph Neural Network (ViG) that treats image patches as graph nodes. VSViG runs continuously at a low frame rate (around five frames per second) and produces an instantaneous seizure-risk score via a sigmoid, used directly by the series gate. Being lightweight and causal, it can sustain a real-time stream on modest hardware. The deployed model further fine-tunes a ProtoGCN (Prototype Graph Convolutional Network) variant of this network using hard-negative examples drawn from the NTU RGB+D 120 dataset, relabeled as the non-seizure class, to sharpen discrimination between seizure motion and structured non-seizure activities such as turning or repositioning. Before this enhancement, the unmodified VSViG checkpoint was reproduced and validated locally: it detected all 33 of 33 seizure events in the local cohort with a mean Latency to EEG Onset of +4.92 s and a mean Latency to Clinical Onset of -9.93 s, consistent with the original paper's reported +5.1 s / -13.1 s and confirming the checkpoint was functioning correctly before the ProtoGCN hard-negative fine-tuning was applied on top of it.

### 2.5.3 ViViT and Cross-Joint Attention

Running alongside the live branch is a heavier, more accurate asynchronous classifier that fires roughly every five seconds. It assembles joint-centric video tubelets (short spatio-temporal volumes cropped around each tracked joint) and feeds them to a Video Vision Transformer (ViViT) [18] equipped with a Cross-Joint (CJ) attention head [19]. ViViT extends the Transformer to video by tokenizing space-time tubelets and applying factorized spatial and temporal attention; the Cross-Joint head additionally attends across joints, capturing the coordinated, whole-body patterns that distinguish a seizure from ordinary movement. The deployment uses an ensemble of two distilled CJ heads trained with different random seeds. These heads are trained by knowledge distillation from the officially released Zamzam et al. checkpoint (a 0.5 soft-label / 0.5 hard-label mix across five random seeds), which lets the deployed system run without depending on the released weights while remaining competitive with them.

### 2.5.4 Series-Gate Fusion

The two branches are combined by a series gate that balances the responsiveness of the live branch against the accuracy of the asynchronous one. When the CJ result is missing or stale, the gate trusts the accumulated live-branch evidence. When the CJ probability is high it is trusted directly; when it is very low the non-seizure verdict is trusted directly; and in the intermediate, uncertain band the two signals are blended (with the live evidence weighted more heavily). The fused score is compared against a mode-dependent threshold, and once a seizure fires, a 30-second latch holds the alert active to prevent flickering. Two clinical modes (monitor and safety) expose a precision/recall trade-off through different thresholds.

### 2.5.5 Why a Series Gate, Not a Single Classifier

Before adopting the series gate, several alternative fusion and classification strategies were evaluated and rejected, providing direct evidence for the chosen design:

- Reproducing the Cross-Joint model locally from scratch (5 random seeds) reached AUROC  $0.923 \pm 0.021$ , below the officially released checkpoint's 0.958, and a calibration check showed the locally reproduced heads were poorly calibrated (temperature  $\approx 10$ ) versus the official checkpoint (temperature  $\approx 1.3$ ). This gap, not architecture choice, is what motivated training the deployed heads by knowledge distillation from the official checkpoint (Section 2.5.3) rather than from scratch.
- A validation-learned weighted average of the two branches (0.76 Cross-Joint + 0.24 VSViG) reached only F1 0.823 on the held-out segments, well below the series gate's F1 0.9018 — a fixed linear blend cannot adapt to how confident the Cross-Joint signal is on a given segment. A further

"FusionMLP" variant that also used EEG-relative timing as an input feature scored higher still, but only because that timing feature leaks the label; it is not a valid comparison point and is noted here only to flag it as inadmissible.

- The distilled Cross-Joint head alone, with no VSViG rescue at all, reached F1 0.818 — confirming that the temporal branch's contribution is not redundant with the transformer branch.
- A family of classification-only alternatives ("SeizureNet": Transformer, GNN, and Multi-Latent-Attention heads, plus MobileViT-XS and VideoMAE-Small encoders) was tested as a way to obtain pre-clinical alerting without any gating logic. Every one of these alternatives produced a positive mean Latency to Clinical Onset (alerts arriving after visible clinical onset, e.g. +5.3 s to +7.2 s), the opposite of the clinically required behavior — demonstrating that segment-level classification alone, at five-second resolution, cannot achieve pre-clinical alerting regardless of head architecture.

Together these results show that the series gate's advantage is not incidental: VSViG supplies the frame-level responsiveness needed for negative LCO, while the Cross-Joint head supplies the discriminative power needed for high precision, and neither alone nor a static blend of the two reproduces what the gate achieves.

## 2.6 Synthetic Clinical Data (Synthea)

Training and demonstrating a clinical system requires realistic patient data, but using real records raises serious privacy and regulatory concerns. Synthea, an open-source synthetic patient generator developed by The MITRE Corporation [20], resolves this tension. It simulates the full lifetime medical history of entirely fictional patients (complete with realistic ICD-10 and SNOMED codes, medications, observations, allergies, and encounters) through a model of disease progression and care. Because the data is synthetic, it carries zero privacy risk while remaining clinically plausible, and it is widely used in academic medical-AI research. Shifaa imports Synthea CSV exports into its relational schema; a localized variant of the patient file substitutes region-appropriate names to make demonstrations more relevant to the target deployment context.

## 2.7 Web and Systems Technologies

Shifaa is delivered as a set of cooperating web services. The principal technologies are summarized below.

| Technology       | Role in Shifaa                                                                                                           |
|------------------|--------------------------------------------------------------------------------------------------------------------------|
| FastAPI (Python) | High-performance asynchronous web framework hosting the REST and WebSocket gateway; auto-generates interactive API docs. |

| Technology                            | Role in Shifaa                                                                                                                                                                                                  |
|---------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SQLAlchemy + SQLite / PostgreSQL      | Object-Relational Mapping over a relational database (SQLite by default for portable deployment; PostgreSQL for production) storing patients, rooms, clinical records, alerts, history, vitals, and audit logs. |
| Pydantic                              | Declarative request/response validation and serialization for every endpoint.                                                                                                                                   |
| JWT (python-jose) + bcrypt (passlib)  | Stateless, signed authentication tokens and securely hashed passwords underpinning role-based access control.                                                                                                   |
| WebSocket & Server-Sent Events        | Bidirectional, low-latency channels for streaming chat tokens, live alerts, and camera frames.                                                                                                                  |
| OpenAI-compatible client / Ollama     | A single client SDK that targets either a local Ollama server (MedGemma, fully private) or a cloud model, selectable by configuration.                                                                          |
| React 18 + Vite + Tailwind + Zustand  | Component-based single-page application with fast tooling, utility-first styling, and lightweight global state.                                                                                                 |
| PyTorch, Ultralytics, MediaPipe, timm | Deep-learning runtimes for the ECG, fall, and seizure inference services.                                                                                                                                       |
| ONNX Runtime + native C++             | High-performance native inference (OpenPose, VSViG, Cross-Joint head) for the real-time seizure runtime, with CUDA/TensorRT execution providers.                                                                |
| Win32 named-pipe IPC                  | Binary inter-process channel carrying ViViT/Cross-Joint token packets between the seizure runtime's C++ engine and its Python worker.                                                                           |

Two architectural patterns deserve emphasis. First, the platform is organized as microservices: the FastAPI gateway is independent of the three heavy inference services (arrhythmia, fall, and seizure), each of which can be scaled, deployed, or replaced on its own. Second, the system is local-first by design: every component, including the LLM, can run on-premises with no external dependency, which is the decisive property for handling sensitive patient data.

## 2.8 Evaluation Metrics

Throughout this document the subsystems are assessed with a common vocabulary of classification metrics, summarized here for completeness. A prediction is a true positive (TP) or true negative (TN) when it matches the ground truth, and a false positive (FP) or false negative (FN) when it does not. From these counts:

- **Accuracy** =  $(TP + TN) / \text{total}$ , the overall fraction of correct predictions; it can be misleading on imbalanced data.
- **Precision** =  $TP / (TP + FP)$ , of the items flagged positive, how many truly are.
- **Recall (sensitivity)** =  $TP / (TP + FN)$ , of the truly positive items, how many were caught. This is the critical metric for safety alerts.
- **F1-score** = the harmonic mean of precision and recall, a single balanced measure. The macro-F1 averages F1 equally across classes.
- **AUROC** = the area under the receiver-operating-characteristic curve, the probability the model ranks a random positive above a random negative, independent of any single threshold.

A confusion matrix tabulates predictions against ground truth and exposes exactly where errors occur. For Shifaa’s safety-critical detectors, recall is weighted heavily (a missed fall or arrhythmia is far costlier than a false alarm that a clinician quickly dismisses), which is why the fall classifier, for instance, is tuned for very high recall.

## 2.9 Related Work and Positioning

Shifaa sits at the intersection of several active research areas. For clinical question answering, two dominant paradigms exist: fine-tuned domain models (such as task-specific BioBERT-based extractors) and retrieval-augmented generation over free text. The former requires labeled training data and retraining as the schema evolves; the latter retrieves unstructured passages and can still hallucinate. Shifaa takes a third path (structured, schema-aware retrieval feeding a strictly grounded general LLM), which needs no task-specific training while keeping answers fully traceable to database records. The table below positions this approach against common baselines.

*Table 4: Positioning of the chatbot approach against baselines*

| Approach               | Intent accuracy | Training needed         | Hallucination control     |
|------------------------|-----------------|-------------------------|---------------------------|
| Keyword matching       | ~65%            | None                    | N/A (no generation)       |
| Fine-tuned BioBERT NER | ~88%            | Labeled data + training | Limited                   |
| RAG over free text     | ~90%            | Index building          | Partial (source passages) |



| Approach                           | Intent accuracy | Training needed | Hallucination control     |
|------------------------------------|-----------------|-----------------|---------------------------|
| Shifaa (structured + grounded LLM) | ~94%            | None            | Strong (cited DB records) |

For the monitoring subsystems, Shifaa builds directly on the state of the art: CTR-GCN for skeleton-based action recognition, and the VSViG and Cross-Joint-attention approaches for video-based seizure detection. Its contribution is less a new model than the integration of these techniques into one coherent, role-aware, privacy-preserving clinical platform with a unified alerting and dashboard layer, a system-level synthesis rather than a single-algorithm advance.

## 2.10 Ethics, Privacy, and Safety Considerations

Clinical AI carries responsibilities beyond accuracy. Three considerations shaped the project from the outset. First, privacy: patient data is among the most sensitive personal data, so Shifaa uses synthetic Synthea data for development and is designed to run entirely on-premises, keeping real records local. Second, the human-in-the-loop principle: in line with MedGemma’s own stated limitations, the assistant never makes autonomous clinical decisions; it presents records and flags events for a clinician to act upon, and its answers are explicitly grounded and cited. Third, transparency: the assistant surfaces its reasoning, every alert stores its raw inference output for audit, and every chat exchange is logged. These properties make the system’s behavior inspectable, which is a prerequisite for trust in any clinical tool.

It is equally important to be candid about the boundaries of the system. The monitoring detectors recognize only the phenomena they were trained on (visible motor seizures, fall-like motion, and a fixed set of arrhythmias) and should be treated as assistive screening rather than definitive diagnosis. The models can err, and their outputs require independent clinical verification. These safety boundaries are revisited as limitations and future work in Chapter 6. The next chapter shows how all of these pieces are assembled into the proposed system.

## Chapter Three

# Proposed System

---

3.1 System Overview & Design Goals

3.2 High-Level Architecture

3.3 Microservices Decomposition

3.4 Data Model

3.5 Core Workflows

3.6 Security & Access Control

## Chapter 3: Proposed System

This chapter presents the architecture and design of Shifaa. It states the design goals, describes the high-level three-tier and microservices architecture, decomposes the system into its services and components, defines the relational data model, walks through the core workflows that connect everything, and explains the security and access-control design.

### 3.1 System Overview and Design Goals

Shifaa is a clinical decision-support and patient-monitoring platform with four functional modules (the conversational history assistant and the arrhythmia, fall, and seizure detectors) unified behind a single role-aware backend and a single web application. The design was driven by the following goals:

- **Accuracy and trust.** Every answer the assistant gives must be grounded in real patient records and cite its sources; every alert must carry the raw inference output for auditing.
- **Privacy by default.** The entire stack, including the LLM, can run locally with no third-party cloud dependency.
- **Separation of concerns.** The lightweight web gateway is decoupled from the heavy AI inference workloads, which run as independent services.
- **Real-time responsiveness.** Chat answers stream token-by-token, and safety alerts are pushed to staff the moment they are generated.
- **Role-appropriate access.** Doctors, nurses, and administrators each see only what their role and patient assignments permit.

### 3.2 High-Level Architecture

The platform follows a three-tier architecture (presentation, application, and data) augmented by an AI inference tier and external device sources. The React single-page application is the presentation tier. The FastAPI gateway (port 8000) is the application tier; it authenticates requests, serves patient and room data, runs the chatbot pipeline, and orchestrates the inference services. A relational database (SQLite by default, PostgreSQL for production) is the data tier. The three inference microservices, each an independent process on its own port (arrhythmia 8001, fall 8002, seizure 8003), plus the local or cloud LLM, form the AI tier. The seizure service is itself a hybrid of a native C++/ONNX runtime and a Python

transformer worker linked by a named pipe. The diagram below shows the tiers and the principal data flows between them.

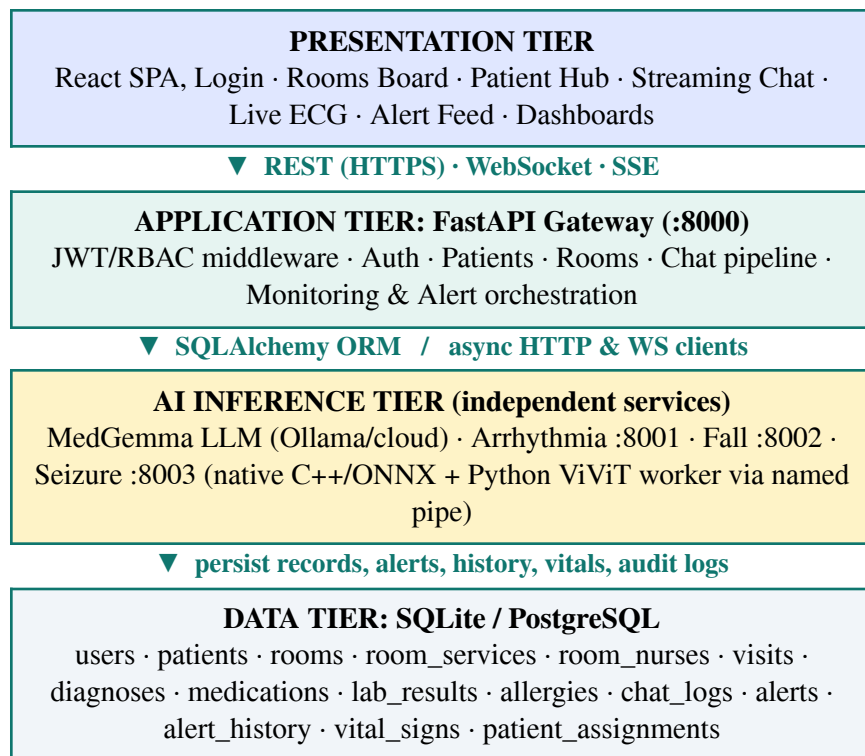


Figure 1: Shifaa high-level architecture (gateway, inference services, and data tier)

### 3.3 Microservices Decomposition

Rather than embedding the deep-learning models inside the web process, Shifaa isolates each heavy workload behind its own service, running on its own port, with a narrow interface. The gateway communicates with each service over HTTP and WebSocket through a dedicated client module, and degrades gracefully when a service is unavailable. This decomposition keeps the gateway responsive, lets each model be deployed on appropriate hardware (for example, a GPU host for the vision services), and allows any service to be developed and tested in isolation. A single launcher script (start\_all\_services) starts all four processes together.

Table 5: Shifaa services, ports, and responsibilities

| Service         | Port | Interface        | Responsibility                                                                  |
|-----------------|------|------------------|---------------------------------------------------------------------------------|
| FastAPI Gateway | 8000 | REST + WebSocket | Auth, RBAC, patients, rooms, chatbot pipeline, alert orchestration, dashboards. |

| Service                      | Port  | Interface                                          | Responsibility                                                                    |
|------------------------------|-------|----------------------------------------------------|-----------------------------------------------------------------------------------|
| LLM Backend (Ollama / cloud) | 11434 | OpenAI-compatible HTTP                             | Intent, NER, grounded generation, multimodal analysis with MedGemma 1.5.          |
| Arrhythmia Service           | 8001  | HTTP (request/response)                            | Two-stage ECG cascade; returns Normal/Abnormal, sub-type, probabilities.          |
| Fall Detection Service       | 8002  | HTTP + WebSocket (frames)                          | Four-stage vision pipeline; per-frame fall events from a camera feed.             |
| Seizure Detection Service    | 8003  | HTTP + WebSocket; native C++/ONNX + named-pipe IPC | VSViG + Cross-Joint series-gate detector; per-frame status and seizure events.    |
| Database                     | -     | SQL (via ORM)                                      | Durable storage of all clinical, room, alert, and audit data (SQLite/PostgreSQL). |

Within the gateway, functionality is organized into nine routers (auth, patients, chat, uploads, users, arrhythmia, monitoring, dashboard, and rooms) and a set of services (intent, ner, retriever, llm, the three inference clients, and a central alert manager). The alert manager is the hub of the monitoring side: any router can publish an alert through it, which persists the alert to the database, automatically broadcasts the updated room state, and pushes the alert to every connected dashboard over the alert WebSocket.

### 3.4 Data Model

All persistent state lives in a normalized relational schema, hosted on SQLite by default for portable deployment and on PostgreSQL for production. The patient table is the hub: every clinical sub-record references it. A unified users table holds doctors, nurses, and administrators, distinguished by a role column, an evolution of the original doctors-only design that preserves backward compatibility (the historical doctor\_id foreign keys still resolve to user records). A rooms subsystem (rooms, room\_services, and a room\_nurses association table) models the physical hospital ward, binding each room to a patient, a set of nurses, and a set of enabled monitoring services. The monitoring side follows an explicit alert lifecycle: active alerts live in the alerts table with a status column, and once acknowledged they are moved to a permanent alert\_history table; both, like vital\_signs, carry a free-form JSON column so the complete raw inference payload is never lost. The entity-relationship structure is summarized below.

*Table 6: Principal database tables*

| Table               | Key Columns                                                                                         | Purpose                                                    |
|---------------------|-----------------------------------------------------------------------------------------------------|------------------------------------------------------------|
| users               | id, name, email (unique), password (bcrypt), role, is_active                                        | Doctors, nurses, admins; role gates API access.            |
| patients            | id, name, dob, gender, blood_type, phone                                                            | Core patient demographics; hub of the schema.              |
| rooms               | id, room_number (unique), room_name, patient_id→, floor, monitoring_status                          | Physical ward rooms; bind a patient, nurses, and services. |
| room_services       | id, room_id→, service_name                                                                          | Enabled services per room (ecg / seizure / fall).          |
| room_nurses         | room_id→, user_id→ (composite PK)                                                                   | Many-to-many room ↔ nurse assignment.                      |
| visits              | id, patient_id→, doctor_id→, visit_date, chief_complaint, notes                                     | Clinical encounters.                                       |
| diagnoses           | id, patient_id→, icd10_code, description, is_active, severity                                       | Coded diagnoses, current or resolved.                      |
| medications         | id, patient_id→, drug_name, dose, start/end_date, is_active                                         | Prescriptions with dosing and status.                      |
| lab_results         | id, patient_id→, test_name, value, unit, reference, test_date, is_abnormal                          | Laboratory observations.                                   |
| allergies           | id, patient_id→, allergen, reaction, severity                                                       | Known drug/substance allergies.                            |
| chat_logs           | id, doctor_id→, patient_id→, query, response, intent_detected                                       | Audit log of every chatbot interaction.                    |
| alerts              | id, patient_id→, alert_type, severity, details (JSON), status                                       | Active (unacknowledged) AI alerts.                         |
| alert_history       | id, alert_id, patient_id→, alert_type, severity, details, acknowledged_by→, acknowledged_at, status | Permanent log of acknowledged alerts.                      |
| vital_signs         | id, patient_id→, source, metric, value, unit, raw_payload (JSON), recorded_at                       | Time-series vitals from devices/inference.                 |
| patient_assignments | id, user_id→, patient_id→, assigned_at                                                              | Legacy clinician–patient responsibility links.             |

Enumerations constrain the monitoring data: a user’s role is one of doctor, nurse, or admin; an alert’s type is one of arrhythmia, fall, or seizure; and its severity is one of low, medium, high, or critical. Composite indexes on (patient\_id, created\_at) and (alert\_type, severity) for alerts, on (patient\_id, created\_at) for alert history, and on (patient\_id, metric, recorded\_at) for vital signs accelerate the most common monitoring

queries. The complete relationships are: a patient has many visits, diagnoses, medications, lab results, allergies, alerts, history records, and vital signs, and is optionally bound to one room; a room has many services and many nurses (and one patient); a user conducts many visits, generates many chat logs, may be assigned to many rooms, and may acknowledge many alerts. The alerts and alert\_history tables together implement a two-stage lifecycle rather than a single mutable record.

### 3.4.1 Detailed Table Schemas

The column-level schema of each principal table is given below, including data types, constraints, and notes. Together these tables fully specify the persistent state of the platform.

#### users

| Column     | Type         | Constraints / Notes                       |
|------------|--------------|-------------------------------------------|
| id         | INTEGER      | Primary key, auto-increment.              |
| name       | VARCHAR(100) | NOT NULL; display name.                   |
| email      | VARCHAR(100) | UNIQUE, NOT NULL; login credential.       |
| password   | VARCHAR(200) | NOT NULL; bcrypt hash.                    |
| specialty  | VARCHAR(100) | Nullable; relevant for doctors.           |
| role       | ENUM         | doctor   nurse   admin; gates API access. |
| is_active  | BOOLEAN      | NOT NULL, default true.                   |
| created_at | DATETIME     | Server default NOW().                     |

#### patients

| Column     | Type         | Constraints / Notes          |
|------------|--------------|------------------------------|
| id         | INTEGER      | Primary key.                 |
| name       | VARCHAR(100) | NOT NULL; full name.         |
| dob        | DATE         | Nullable; date of birth.     |
| gender     | VARCHAR(10)  | Nullable; “Male” / “Female”. |
| blood_type | VARCHAR(5)   | Nullable; e.g. “O+”, “AB-”.  |
| phone      | VARCHAR(20)  | Nullable; contact number.    |
| created_at | DATETIME     | Server default NOW().        |

#### diagnoses

| Column       | Type        | Constraints / Notes               |
|--------------|-------------|-----------------------------------|
| id           | INTEGER     | Primary key.                      |
| patient_id   | INTEGER     | FK → patients.id, NOT NULL.       |
| icd10_code   | VARCHAR(50) | Nullable; e.g. “E11”.             |
| description  | TEXT        | NOT NULL; condition name.         |
| diagnosed_on | DATE        | Nullable.                         |
| is_active    | BOOLEAN     | Default true; active vs resolved. |
| severity     | VARCHAR(20) | mild / moderate / severe.         |

### medications

| Column     | Type        | Constraints / Notes                 |
|------------|-------------|-------------------------------------|
| id         | INTEGER     | Primary key.                        |
| patient_id | INTEGER     | FK → patients.id, NOT NULL.         |
| drug_name  | TEXT        | NOT NULL; brand or generic.         |
| dose       | VARCHAR(50) | Nullable; e.g. “500mg twice daily”. |
| start_date | DATE        | Nullable; prescription start.       |
| end_date   | DATE        | Nullable; NULL ⇒ still active.      |
| is_active  | BOOLEAN     | Default true.                       |

### lab\_results

| Column      | Type        | Constraints / Notes            |
|-------------|-------------|--------------------------------|
| id          | INTEGER     | Primary key.                   |
| patient_id  | INTEGER     | FK → patients.id, NOT NULL.    |
| test_name   | TEXT        | NOT NULL; e.g. “HbA1c”.        |
| value       | FLOAT       | Nullable; numeric result.      |
| unit        | VARCHAR(30) | Nullable; e.g. “mg/dL”, “%”.   |
| reference   | VARCHAR(50) | Nullable; normal-range string. |
| test_date   | DATE        | NOT NULL.                      |
| is_abnormal | BOOLEAN     | Default false; flagged by lab. |

### alerts (active alerts)



| Column     | Type        | Constraints / Notes                                     |
|------------|-------------|---------------------------------------------------------|
| id         | INTEGER     | Primary key.                                            |
| patient_id | INTEGER     | FK → patients.id, NOT NULL.                             |
| alert_type | ENUM        | arrhythmia   fall   seizure.                            |
| severity   | ENUM        | low   medium   high   critical.                         |
| details    | JSON        | Raw inference output (probabilities, scores).           |
| status     | VARCHAR(20) | Default “ACTIVE”; deleted on acknowledge/cancel/expire. |
| created_at | DATETIME    | Server default NOW().                                   |

#### alert\_history (acknowledged alerts)

| Column                | Type        | Constraints / Notes                            |
|-----------------------|-------------|------------------------------------------------|
| id                    | INTEGER     | Primary key.                                   |
| alert_id              | INTEGER     | Original alert id, NOT NULL.                   |
| patient_id            | INTEGER     | FK → patients.id, NOT NULL.                    |
| alert_type / severity | VARCHAR     | Copied from the original alert.                |
| details               | JSON        | Raw inference output preserved from the alert. |
| acknowledged_by       | INTEGER     | FK → users.id; who acknowledged it.            |
| acknowledged_at       | DATETIME    | When it was acknowledged.                      |
| created_at            | DATETIME    | Original alert creation timestamp.             |
| status                | VARCHAR(20) | Default “ACKNOWLEDGED”.                        |

#### rooms

| Column            | Type         | Constraints / Notes                               |
|-------------------|--------------|---------------------------------------------------|
| id                | INTEGER      | Primary key.                                      |
| room_number       | VARCHAR(50)  | UNIQUE, NOT NULL; indexed.                        |
| room_name         | VARCHAR(100) | Nullable; human-readable label.                   |
| patient_id        | INTEGER      | FK → patients.id, nullable; current occupant.     |
| floor             | VARCHAR(50)  | Nullable; ward floor.                             |
| monitoring_status | VARCHAR(50)  | Default “Idle”; e.g. Idle / Monitoring / Offline. |

| Column                  | Type     | Constraints / Notes                         |
|-------------------------|----------|---------------------------------------------|
| created_at / updated_at | DATETIME | Server defaults; updated_at auto-refreshes. |

Two supporting tables complete the room model: room\_services (id, room\_id → rooms.id with ON DELETE CASCADE, service\_name) lists the services enabled for each room, and room\_nurses is an association table with a composite primary key of (room\_id, user\_id), both cascading on delete, that links rooms to their assigned nurses.

### vital\_signs

| Column      | Type        | Constraints / Notes                  |
|-------------|-------------|--------------------------------------|
| id          | INTEGER     | Primary key.                         |
| patient_id  | INTEGER     | FK → patients.id, NOT NULL.          |
| source      | VARCHAR(50) | Device class: ecg   camera   sensor. |
| metric      | VARCHAR(50) | e.g. heart_rate, spo2.               |
| value       | FLOAT       | Nullable; measured value.            |
| unit        | VARCHAR(20) | Nullable.                            |
| raw_payload | JSON        | Full inference output for audit.     |
| recorded_at | DATETIME    | Server default NOW().                |

The remaining tables follow the same pattern: visits (patient\_id, doctor\_id, visit\_date, chief\_complaint, notes), allergies (patient\_id, allergen, reaction, severity), chat\_logs (doctor\_id, patient\_id, query, response, intent\_detected, created\_at), and patient\_assignments (user\_id, patient\_id, assigned\_at). All foreign keys reference the patient or user hub tables, and every clinical sub-table cascades conceptually from a single patient.

## 3.5 Core Workflows

### 3.5.1 Conversational Query Resolution

When a clinician asks a question about a patient, the gateway resolves it through a four-stage pipeline before streaming the answer back. The flow below shows the sequence; the answer is delivered token-by-token over the chat WebSocket, and the full exchange is written to the audit log.

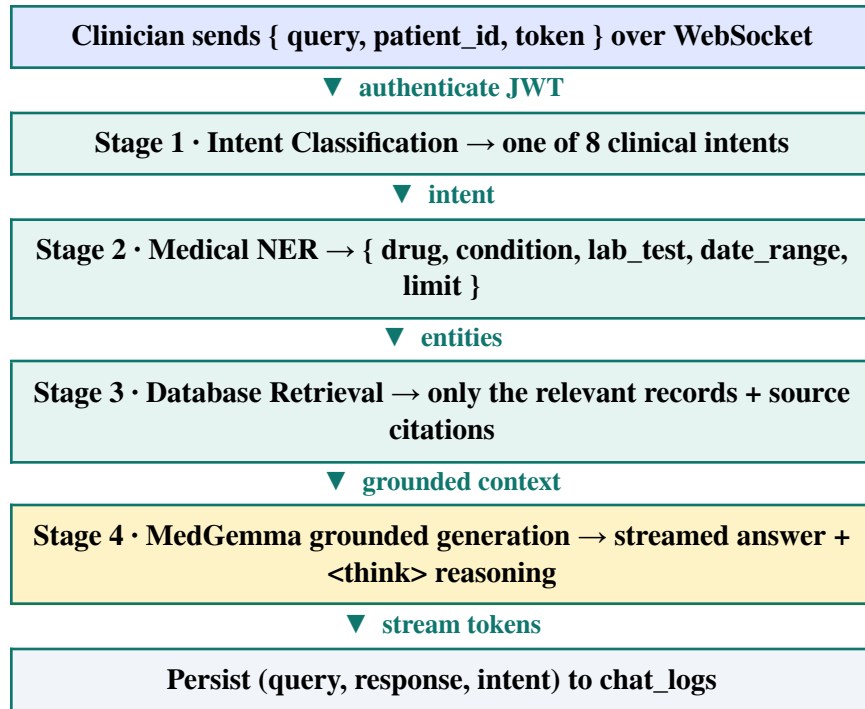
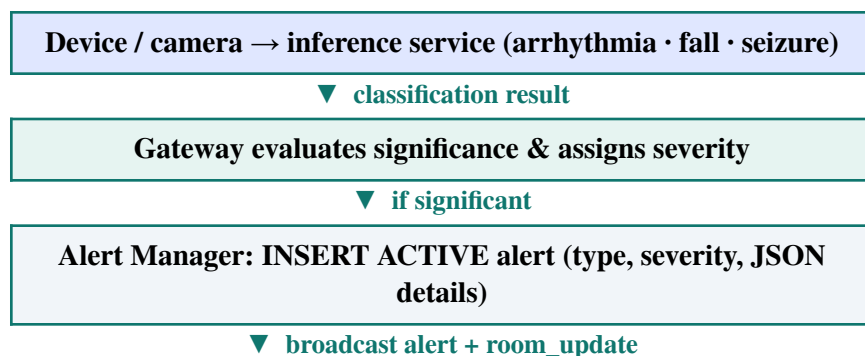


Figure 2: Conversational query-resolution workflow

### 3.5.2 Monitoring and Alert Publication

The three monitoring subsystems share a common alert path with an explicit lifecycle. A device or camera feeds a signal to the relevant inference service; the service returns a classification; and when the result is clinically significant, the gateway's alert manager persists an ACTIVE alert, automatically broadcasts the updated room card, and pushes the alert to every connected dashboard. When a clinician acknowledges the alert it is moved to alert\_history; a cancel deletes it; and any active alert left unacknowledged for 24 hours is expired automatically by a background loop. Each of these transitions is broadcast as its own event. To avoid alert storms, the seizure relay fires only on the transition into a seizure state, not on every frame.



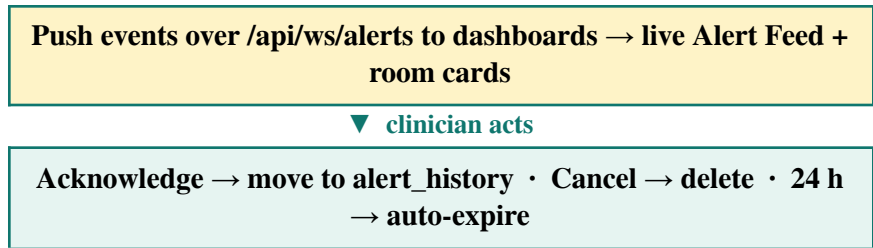


Figure 3: Monitoring and alert lifecycle workflow

## 3.6 Security and Access Control

Security rests on three pillars: authentication, role-based authorization, and patient-level data isolation. Passwords are stored only as bcrypt hashes. On login, the gateway issues a signed JSON Web Token (JWT, HS256) carrying the user's identity and role; every protected endpoint requires this token. Authorization is enforced per-endpoint by a role requirement, for example, only doctors and nurses may start a monitoring session, while user management is restricted to administrators.

Table 7: Roles and their capabilities

| Role   | Capabilities                                                                                                                                                |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Doctor | Broad clinical access: view all rooms and patients, run the chatbot and ECG analysis, start/stop monitoring, acknowledge or cancel alerts, view dashboards. |
| Nurse  | Scoped to assigned rooms: room cards and live alert feed, acknowledge alerts, start/stop monitoring for assigned rooms.                                     |
| Admin  | Manage users (create/activate) and rooms (create, assign patients, nurses, and services), system-wide service health and dashboards; exempt from scoping.   |

Data isolation is enforced primarily through the room model. A nurse sees only the rooms they are assigned to via the `room_nurses` association; the alert broadcaster checks, for every event, that the receiving nurse is assigned to the room of the patient in question, and otherwise skips them (falling back to the legacy `patient_assignments` link when a patient has no room). Doctors have broad read access across all rooms, reflecting the cross-ward nature of medical cover, while administrators are exempt from scoping so they can configure the whole ward. This room-centered model mirrors how responsibility is actually allocated on a hospital floor and replaces the earlier purely patient-by-patient assignment scheme.

## 3.7 Design Decisions and Trade-offs

Several deliberate engineering choices shaped the architecture. Each balanced competing concerns, and the rationale is recorded below for transparency.

Table 8: Key design decisions and their rationale

| Decision                                        | Alternative                     | Rationale                                                                               |
|-------------------------------------------------|---------------------------------|-----------------------------------------------------------------------------------------|
| LLM as structured classifier (intent/NER)       | Train bespoke classifiers       | No labeled data or retraining; robust to paraphrase.                                    |
| Structured DB retrieval + strict grounding      | RAG over free text              | Fully traceable, auditable answers; minimal hallucination.                              |
| Microservices for AI inference                  | Embed models in the web process | Keeps the gateway responsive; independent scaling/deployment.                           |
| Local-first LLM via Ollama                      | Cloud-only model                | Patient data never leaves the premises.                                                 |
| Single users table with role column             | Separate tables per role        | Simpler auth; backward-compatible with legacy doctor_id keys.                           |
| JSON details column on alerts/vitals            | Rigid typed columns only        | Preserves full raw inference output as the schema evolves.                              |
| Seizure alert on state transition only          | Alert every frame               | Avoids alert storms during sustained episodes.                                          |
| Dual-mode (mock/live) frontend                  | Live-only                       | Enables offline demonstration and UI development.                                       |
| SQLite by default (PostgreSQL optional)         | PostgreSQL-only                 | Zero-config, portable deployment; swap via one env variable for production.             |
| Native C++/ONNX seizure runtime + Python worker | Pure Python pipeline            | Sustains real-time (~37 FPS) by running hot paths natively, transformer asynchronously. |
| Room-centered monitoring & nurse scoping        | Patient-only assignment         | Matches ward responsibility; one place to bind patient, nurses, and services.           |
| Active alerts + permanent history split         | Single mutable alert row        | Clean lifecycle; full audit trail of acknowledged events is never overwritten.          |

The recurring theme across these decisions is to favor accuracy, auditability, and privacy over raw model sophistication, appropriate priorities for a clinical system where a wrong or untraceable answer is worse than no answer. The next chapter describes how each of these components was implemented.

## Chapter Four

# System Implementation

---

- 4.1 Backend Gateway & Alert Manager
- 4.2 Conversational AI Pipeline
- 4.3 ECG Arrhythmia Detection Cascade
- 4.4 Patient Fall Detection Pipeline
- 4.5 Real-Time Seizure Detection Pipeline
- 4.6 Frontend Single-Page Application
- 4.7 Data Import Pipeline

# Chapter 4: System Implementation

This chapter describes how each component of Shifaa was implemented. Following the structure of Chapter 3, it treats the backend gateway, the conversational AI pipeline, the three monitoring subsystems, the frontend application, and the data-import pipeline in turn. In keeping with good documentation practice, the algorithms are presented as flowcharts and pseudocode rather than literal source code, and each component is accompanied by its results and a short discussion.

## 4.1 Backend Gateway and Alert Manager

The gateway is a FastAPI application that boots by creating any missing database tables, mounting an /uploads directory for static files, applying permissive CORS for the development frontend, and registering nine routers (auth, patients, chat, uploads, users, arrhythmia, monitoring, dashboard, and rooms). On startup it also launches a background task that expires stale alerts. It exposes a root status endpoint and a health endpoint for uptime monitors. Authentication and authorization are implemented as reusable dependencies: one decodes and validates the JWT to resolve the current user, and a role-requirement factory rejects users whose role is not permitted for an endpoint.

The alert manager is the shared hub for the monitoring side. It maintains a map of connected dashboard WebSockets to their authenticated users and offers four operations: publish, which writes an ACTIVE alert and broadcasts it; broadcast\_event, which emits custom lifecycle events (acknowledge, cancel, expire, room\_update); a background expiration loop; and connect/disconnect. When it broadcasts, it first emits an updated room card for the affected patient, then delivers the payload only to subscribers permitted to see it, a nurse who is not assigned to the relevant room is skipped. Its publish logic is summarized below.

### Algorithm 4.1, Alert publication and room-aware broadcast

```
function publish(db, patient_id, alert_type, severity, details):
    alert = new Alert(patient_id, alert_type, severity,
                      details=details, status='ACTIVE')
    db.insert(alert); db.commit()
    broadcast(db, patient_id, { type:'alert', id:alert.id, ... })
    return alert

function broadcast(db, patient_id, payload):
    if payload.type not in (room_update, room_delete) and patient_id:
        room = rooms.where(patient_id == patient_id).first()
        if room: broadcast(db, patient_id, room_update(serialize(room)))
```

```

for (ws, user) in subscribers:
    if user.role == nurse and not nurse_assigned_to_room(user, room):
        continue          # room-scoped confidentiality
    try: ws.send_json(payload)
    except: drop ws

```

Because every alert type flows through this one function, the database record and the live broadcast can never diverge, the JSON details column preserves the full raw inference output for later auditing, and the room card on every dashboard is kept in sync automatically.

Two further pieces complete the lifecycle. A background loop runs once a minute and deletes any ACTIVE alert older than 24 hours, broadcasting an expire event for each. Acknowledgement (in the dashboard router) copies an active alert into alert\_history with the acknowledging user and timestamp, deletes the active row, and broadcasts an acknowledge event; cancellation simply deletes the active alert and broadcasts a cancel event.

#### 4.1.1 Authentication and Authorization

Authentication is implemented in a dedicated module. On login, the gateway looks up the user by email, verifies the submitted password against the stored bcrypt hash, and (on success) issues a signed JWT containing the user's identifier, name, and role, with a configurable expiry. Protected endpoints depend on a function that decodes the token from the Authorization header, validates its signature and expiry, and loads the corresponding user; a separate role-requirement factory wraps this to additionally assert that the user's role is among those permitted for the endpoint. WebSocket endpoints, which cannot easily carry custom headers, accept the token as a query parameter and validate it before accepting the connection.

##### **Algorithm 4.1b, Login and token issue**

```

function login(email, password):
    user = users.where(email == email).first()
    if user is None or not bcrypt.verify(password, user.password):
        raise 401 'Incorrect email or password'
    claims = { sub: user.id, name: user.name, role: user.role,
              exp: now() + ACCESS_TOKEN_EXPIRE_MINUTES }
    return { access_token: jwt.encode(claims, SECRET_KEY, HS256),
            token_type: 'bearer' }

```

#### 4.1.2 Patient, User, and Room Routers

The patients router exposes listing, registration, full-record retrieval, and filtered sub-record endpoints (medications, labs, diagnoses, and active alerts). The users router, restricted to administrators, supports



creating, listing, and activating or deactivating hospital accounts. The rooms router is the centerpiece of the new ward model: administrators create, update, and delete rooms and assign a patient, a set of nurses, and a set of enabled services to each; any clinician can list rooms (nurses see only their assigned rooms) and read a room's live status. Each room's response is enriched on the fly with its current active alerts and an aggregated risk score, computed from the inference details stored on those alerts.

*Table 9: Backend routers and their endpoints*

| Router     | Representative endpoints                                                       | Allowed roles             |
|------------|--------------------------------------------------------------------------------|---------------------------|
| auth       | POST /api/auth/login                                                           | Public                    |
| patients   | GET/POST /api/patients, GET /api/patients/{id}/{labslmedslalerts}              | Doctor, Nurse, Admin      |
| rooms      | GET/POST/PUT/DELETE /api/rooms, POST .../assign-{patientlnurseslservices}      | Admin (write), all (read) |
| chat       | POST /api/chat, WS /api/ws/chat                                                | Doctor                    |
| uploads    | POST /api/chat/analyze-image                                                   | Doctor                    |
| users      | GET/POST/PATCH /api/admin/users                                                | Admin                     |
| arrhythmia | POST .../analyze, GET .../history/{id}, GET .../sandbox/signals                | Doctor, Nurse             |
| monitoring | POST/DELETE .../fallseizure (start/stop/reset-latch), WS /api/ws/alertslcamera | Doctor, Nurse             |
| dashboard  | GET .../summary, PATCH .../{id}/acknowledge, DELETE .../{id}/cancel            | Doctor, Nurse, Admin      |

### 4.1.3 Room Status Aggregation

The room model turns the raw alert stream into an at-a-glance ward view. Whenever a room is serialized for the API or a broadcast, the gateway computes its live status on the fly: it gathers the active alerts for the room's patient, derives a numeric risk score from the inference details on those alerts (for example, a fall probability or a seizure gate score), and sets the room's monitoring status accordingly (Idle when no patient is assigned, Monitoring otherwise). This computed view (status, risk score, active alerts, assigned nurses, and enabled services) is what the room cards and the automatic room\_update broadcasts carry, so every dashboard reflects the true ward state in real time without polling.

#### Algorithm 4.1c, Room status and risk aggregation

```
function serialize_room(db, room):
```

```

if room.patient_id is None: return (status='Idle', risk=0, alerts=[])
active = alerts.where(patient_id == room.patient_id,
                      status == 'ACTIVE').all()
risk = max(extract_probability(a.details) for a in active, default=0)
status = 'Monitoring'
return room + { services, nurses, active_alerts: active,
               risk_score: risk, monitoring_status: status }

```

## 4.2 Conversational AI Pipeline

The assistant resolves a clinician’s question through four sequential stages, with two alternative entry paths: a general-medical mode that bypasses retrieval, and an image-analysis path that accepts an uploaded scan. The stages and their files are intent classification, medical NER, database retrieval, and grounded generation. Each LLM-backed stage uses the same client SDK, which targets a local Ollama server running MedGemma 1.5 by default, or a cloud model if configured.

### 4.2.1 Stage 1, Intent Classification

The first stage maps the free-text query to exactly one of eight clinical intents. The model is prompted with a strict system message, asked for JSON output, and run at temperature zero for determinism. An invalid or unparseable response falls back to a safe default intent.

*Table 10: The eight supported query intents*

| Intent           | Meaning                    | Example query                    |
|------------------|----------------------------|----------------------------------|
| history_lookup   | General medical background | “What is the patient’s history?” |
| medication_check | Current prescriptions      | “What drugs is this patient on?” |
| lab_results      | Laboratory test values     | “Show the last HbA1c result.”    |
| allergy_check    | Known allergies            | “Is she allergic to penicillin?” |
| visit_summary    | Past encounters            | “Summarize the last 3 visits.”   |
| diagnosis_check  | Active/past diagnoses      | “Does he have diabetes?”         |
| risk_flag        | Abnormal/risky findings    | “Anything concerning here?”      |
| general_question | General clinical knowledge | “Normal range for HbA1c?”        |

#### Algorithm 4.2, Intent classification

```

function classify_intent(query):
    messages = [ system(INTENT_PROMPT), user(query) ]
    resp = llm.chat(messages, temperature=0, format='json')

```

```

obj = parse_json(resp)
if obj.intent not in VALID_INTENTS:
    return { intent: 'history_lookup', confidence: 0.0 } # safe fallback
return { intent: obj.intent, confidence: obj.confidence }

```

### 4.2.2 Stage 2, Medical Named-Entity Recognition

The second stage extracts the structured fields needed to filter the database. It returns a JSON object with five nullable fields: `date_range`, `condition`, `drug`, `lab_test`, and `limit`. Using an LLM as the extractor makes the stage robust to paraphrase and medical abbreviation without any task-specific training.

#### Algorithm 4.3, Entity extraction

```

function extract_entities(query):
    messages = [ system(NER_PROMPT), user(query) ]
    resp = llm.chat(messages, temperature=0, format='json')
    e = parse_json(resp)
    return { date_range: e.date_range, condition: e.condition,
            drug: e.drug, lab_test: e.lab_test, limit: e.limit }

# e.g. 'HbA1c from the last 3 months' ->
# { lab_test:'HbA1c', date_range:'last 3 months', others:null }

```

### 4.2.3 Stage 3, Database Retrieval

The retriever turns the intent and entities into one or more targeted SQL queries through the ORM, applies entity-based filters, and returns both the records and a list of human-readable source citations. A small helper parses natural-language date ranges (for example, “last 3 months”) into a cutoff date. The mapping from intent to table and filters is given below.

Table 11: Retrieval strategy by intent

| Intent                     | Table(s)                                          | Filters applied                              |
|----------------------------|---------------------------------------------------|----------------------------------------------|
| medication_check           | medications                                       | drug name (ILIKE), is_active = true          |
| lab_results                | lab_results                                       | test_name (ILIKE), test_date ≥ cutoff, LIMIT |
| allergy_check              | allergies                                         | patient_id only                              |
| visit_summary              | visits                                            | visit_date ≥ cutoff, LIMIT                   |
| diagnosis_check            | diagnoses                                         | description (ILIKE condition)                |
| history_lookup / risk_flag | diagnoses + medications + lab_results + allergies | combined (one of each type)                  |

| Intent           | Table(s)        | Filters applied          |
|------------------|-----------------|--------------------------|
| general_question | , (no DB query) | returns empty record set |

#### Algorithm 4.4, Intent-aware retrieval

```

function retrieve(patient_id, intent, entities, db):
    records = []; sources = []
    cutoff = parse_date_range(entities.date_range) # may be null
    switch intent:
        case 'medication_check':
            q = medications.where(patient_id, is_active=True)
            if entities.drug: q = q.where(drug_name ILIKE %drug%)
        case 'lab_results':
            q = labs.where(patient_id)
            if entities.lab_test: q = q.where(test_name ILIKE %lab_test%)
            if cutoff: q = q.where(test_date >= cutoff)
            q = q.order_by(test_date desc).limit(entities.limit or 10)
        case 'allergy_check': q = allergies.where(patient_id)
        case 'history_lookup' | 'risk_flag':
            collect diagnoses + medications + labs + allergies
        case 'general_question': return ([], [])
    records = q.all()
    sources = [ format_citation(r) for r in records ]
    return (records, sources)

```

#### 4.2.4 Stage 4, Grounded Answer Generation

The final stage formats the retrieved records into a compact, labeled context block (patient header, diagnoses, active medications, recent labs with abnormal flags, and allergies) and submits it with a strict grounding system prompt. The prompt instructs the model to answer only from the provided data, to cite the record each fact comes from, to use the fixed sentence “The records do not contain this information.” when data is missing, to be concise and clinically precise, never to make recommendations, and to flag abnormal values or dangerous drug combinations. Generation runs at low temperature with a bounded token budget.

#### Algorithm 4.5, Grounded generation

```

function generate_answer(query, intent, records, patient):
    context = format_patient_block(patient, records)
    messages = [ system(GROUNDING_PROMPT),
                 user(context + '\n\nQuestion: ' + query) ]
    stream = llm.chat(messages, temperature=0.2, max_tokens=1024,
                      stream=True)

```

```
for token in stream: yield token # forwarded to the think-parser
```

### 4.2.5 Streaming and the Reasoning (<think>) Parser

MedGemma 1.5 prefixes its private reasoning with a special start token and ends it with a special stop token before producing the final answer. The gateway streams the model output token-by-token over the WebSocket, but first passes it through a stateful parser that separates reasoning from the answer and emits structured UI events. The parser recognizes three styles (MedGemma’s special tokens, standard XML-style think tags, and a plain text “thought” prefix) and emits a `think_start` event to open the collapsible reasoning panel, `think` chunks while reasoning, a `think_done` event (carrying the reasoning duration) when reasoning ends, and ordinary chunks for the final answer.

The central difficulty is that a control tag may be split across two network tokens (for example, “<un” then “used95>”). To prevent such fragments from escaping detection, the parser buffers any partial that could still grow into a known tag and only flushes it once a match or mismatch is certain.

#### Algorithm 4.6, Stateful think-stream parser

```
state = ANSWER ; buffer = ""
for token in model_stream:
    buffer += token
    while buffer not empty:
        if buffer matches a START marker (<unused94> | <think> | 'thought'):
            emit think_start; state = THINKING; consume marker; continue
        if state == THINKING and buffer matches END (<unused95> | </think>):
            emit think_done(duration); state = ANSWER; consume marker; continue
        if buffer is a strict PREFIX of any marker:
            break # keep buffering; tag may still complete
        ch = pop_one_char(buffer)
        emit (state == THINKING ? think_chunk(ch) : answer_chunk(ch))
    emit done(intent, sources)
```

### 4.2.6 Multimodal Image Analysis

A separate endpoint accepts an uploaded medical image together with a scan-type category (chest X-ray, CT/MRI, lab report, handwritten note, dermatology, or general). The image is base64-encoded and sent to MedGemma’s multimodal interface, and the response is streamed back to the browser using Server-Sent Events with the same think-parsing logic as the WebSocket path. To keep the interface responsive under heavy local-GPU load, the endpoint is wrapped in a time-bounded fail-safe: if the local model does not respond within a few seconds, the system streams a structured clinical-scan template instead of hanging, preserving continuous operation.

### 4.2.7 Results and Discussion, Conversational Pipeline

Each stage of the pipeline was evaluated on a curated test set. Intent classification was tested on 80 doctor queries (ten per intent, including paraphrases and abbreviations); NER on 50 annotated queries; and retrieval against a controlled, seeded database. The headline results are summarized below.

Table 12: Conversational pipeline evaluation summary

| Stage                 | Metric                           | Result       |
|-----------------------|----------------------------------|--------------|
| Intent classification | Accuracy / Macro-F1 (80 queries) | 94.1% / 0.94 |
| Medical NER           | Macro-F1 (50 queries)            | 94.0%        |
| Retrieval             | Exact record match               | 97.0%        |
| Retrieval             | Correct table selected           | 100%         |
| Grounding             | Answers grounded in records      | ≥ 99%        |
| Grounding             | Hallucination rate               | ≤ 1%         |

Table 13: Per-class intent classification results (Ollama backend)

| Intent class     | Precision | Recall | F1   |
|------------------|-----------|--------|------|
| history_lookup   | 0.95      | 0.90   | 0.92 |
| medication_check | 0.98      | 1.00   | 0.99 |
| lab_results      | 0.97      | 0.95   | 0.96 |
| allergy_check    | 1.00      | 1.00   | 1.00 |
| visit_summary    | 0.95      | 0.95   | 0.95 |
| diagnosis_check  | 0.92      | 0.95   | 0.94 |
| risk_flag        | 0.90      | 0.85   | 0.87 |
| general_question | 0.95      | 0.90   | 0.92 |
| Macro average    | 0.95      | 0.94   | 0.94 |

Table 14: Per-entity NER results

| Entity type | Precision | Recall | F1   |
|-------------|-----------|--------|------|
| date_range  | 0.92      | 0.88   | 0.90 |
| condition   | 0.95      | 0.90   | 0.92 |
| drug        | 0.98      | 0.95   | 0.96 |
| lab_test    | 0.97      | 0.94   | 0.95 |
| limit       | 1.00      | 0.95   | 0.97 |

| Entity type | Precision | Recall | F1   |
|-------------|-----------|--------|------|
| Average     | 0.96      | 0.92   | 0.94 |

The per-class intent results were strongest for `allergy_check` (F1 = 1.00) and `medication_check` (F1 = 0.99) and weakest for `risk_flag` (F1 = 0.87), which overlaps semantically with `history_lookup`; the combined retrieval used for both intents mitigates this overlap in practice. End-to-end latency depends almost entirely on the LLM backend: with a local GPU, first-token latency is around 0.8 seconds, rising to roughly 2 seconds on CPU-only hardware. The principal limitation is the absence of conversational memory (each query is currently resolved statelessly), which is noted as future work. Overall, the results confirm that general-purpose LLMs, constrained by strict prompts and grounded retrieval, can deliver accurate, auditable clinical question-answering without any task-specific model training.

### 4.3 ECG Arrhythmia Detection Cascade

The arrhythmia subsystem classifies a 10-second, 12-lead ECG (a  $5000 \times 12$  array) through a two-stage cascade. Stage 1 screens the signal as Normal or Abnormal; only abnormal signals proceed to Stage 2, which assigns one of four sub-types: Atrial Fibrillation (AF), First-Degree Atrioventricular Block (IAVB), Sinus Bradycardia (SB), or Sinus Tachycardia (STach). The gateway submits ECGs to this service through the arrhythmia router, which validates the patient, runs the cascade, and (if the result is abnormal) raises an alert whose severity reflects the sub-type and confidence.

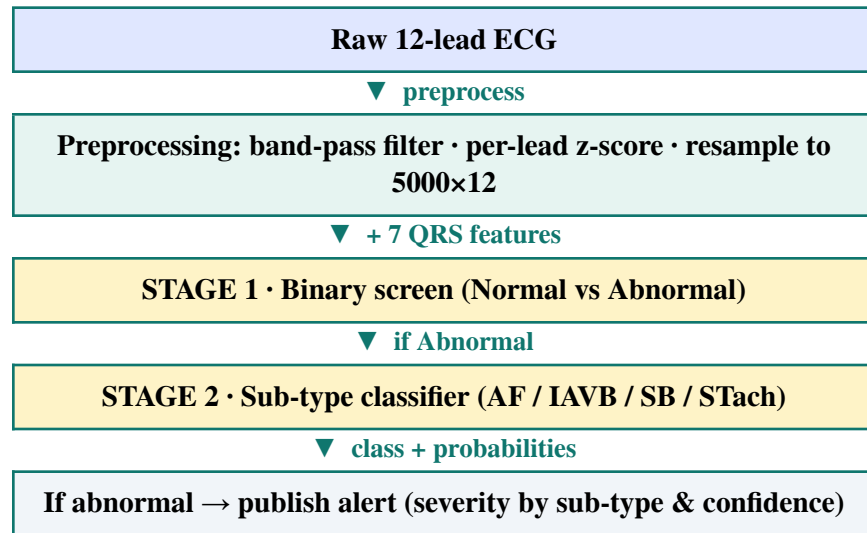


Figure 4: Two-stage ECG arrhythmia cascade

Both stages share the dual-branch CNN–BiLSTM–attention architecture described in Section 2.3.2: a waveform branch (three 1-D convolutional blocks, a bidirectional LSTM, and self-attention pooling) fused with an auxiliary branch that ingests seven QRS/heart-rate statistics. Preprocessing applies a Butterworth band-pass filter (0.5–45 Hz) to remove baseline wander and powerline noise, normalizes each lead with a z-score, and resamples or truncates to exactly 5000 samples.

Table 15: Dual-branch ECG model architecture (per stage)

| Branch / Block         | Configuration                                        |
|------------------------|------------------------------------------------------|
| Waveform, Conv block 1 | Conv1D 12→32, kernel 7, BN, ReLU, MaxPool, Dropout.  |
| Waveform, Conv block 2 | Conv1D 32→64, kernel 5, BN, ReLU, MaxPool, Dropout.  |
| Waveform, Conv block 3 | Conv1D 64→128, kernel 3, BN, ReLU, MaxPool, Dropout. |
| Waveform, Recurrent    | BiLSTM, hidden 128, 2 layers, bidirectional.         |
| Waveform, Pooling      | Self-attention pooling → context vector.             |
| QRS branch             | MLP 7→32→64→64 with BatchNorm, GELU, Dropout.        |



| Branch / Block | Configuration                                                        |
|----------------|----------------------------------------------------------------------|
| Fusion + head  | Gated fusion → Linear→256→128→n_classes (GELU, Dropout).             |
| Parameters     | Binary $\approx 1,120,930$ ; sub-type $\approx 1,121,188$ trainable. |

The seven auxiliary QRS/heart-rate features fed to the second branch are: mean R-R interval, standard deviation of R-R, mean QRS amplitude, standard deviation of QRS amplitude, mean heart rate (bpm), R-R coefficient of variation, and beats per second. These explicit rhythm statistics complement the learned waveform features and improve discrimination of rate-related arrhythmias such as sinus bradycardia and tachycardia.

#### Algorithm 4.7, Cascade inference

```

function predict(signal_5000x12, qrs7):
    x = preprocess(signal_5000x12)      # filter, z-score, resample
    p1 = binary_model(x, qrs7)          # P(Abnormal)
    if p1 < 0.5:
        return { stage1:'Normal', stage1_conf:1-p1 }
    probs = subtype_model(x, qrs7)      # over {AF, IAVB, SB, STach}
    cls = argmax(probs)
    return { stage1:'Abnormal', stage1_conf:p1,
            stage2_class:cls, stage2_conf:probs[cls],
            all_probabilities:probs }

```

#### 4.3.1 Results and Discussion, Arrhythmia Cascade

On held-out test data, Stage 1 reached 96.10% accuracy with a 93.42% macro-F1 and 98.26% macro-AUROC; Stage 2 reached 94.92% accuracy with a 94.71% macro-F1 and 99.43% macro-AUROC. The confusion matrices and per-class results are given below.

Table 16: Binary screening confusion matrix (Stage 1)

|               | Predicted Normal | Predicted Abnormal |
|---------------|------------------|--------------------|
| True Normal   | 15,830           | 501                |
| True Abnormal | 272              | 3,202              |

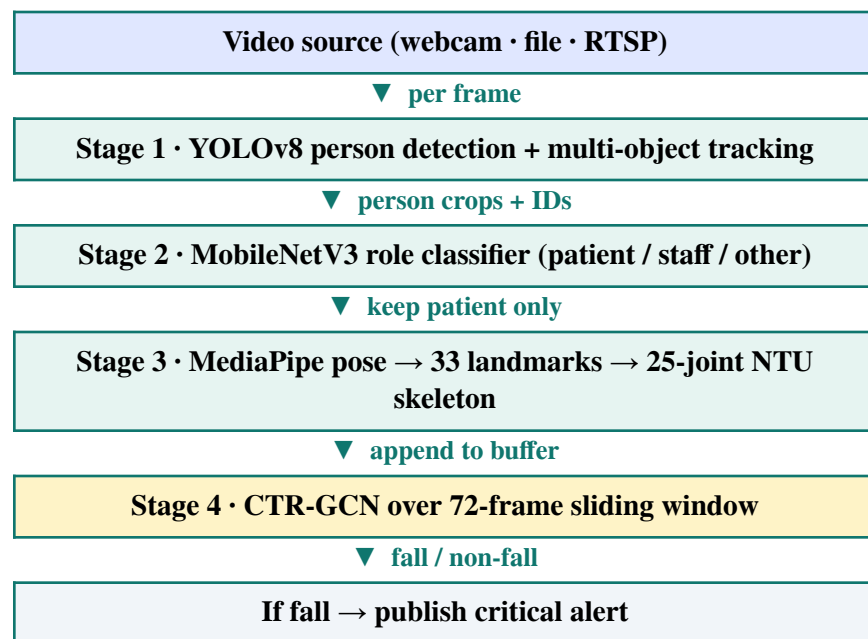
Table 17: Sub-type classifier per-class F1 (Stage 2)

| AF     | IAVB   | SB     | STach  |
|--------|--------|--------|--------|
| 95.13% | 92.27% | 94.96% | 96.46% |

The cascade design pays off: by screening first, the rarer multi-class decision is made only on the small abnormal minority, keeping both accuracy and inference cost favorable. Strict patient-level grouping across the merged PTB-XL, CINC2020, and CODE-15 corpora ensured these figures are not inflated by leakage. The main residual error is the 501 false-positive abnormal calls in Stage 1, an intentional bias toward sensitivity that is clinically preferable to missing true abnormalities, since every flagged signal is reviewed by a clinician.

## 4.4 Patient Fall Detection Pipeline

The fall subsystem is a four-stage vision pipeline that turns a camera feed into per-frame fall decisions. The gateway bridges a camera client to the fall service over a WebSocket: it forwards JPEG frames to the service and relays the service’s events back to the client, raising a critical alert whenever a fall is detected for a numeric room/patient identifier.



*Figure 5: Four-stage fall-detection pipeline*

To balance responsiveness against compute, the stages run at different cadences: YOLO tracking runs every frame, the role classifier is refreshed every 10 frames, and the CTR-GCN classifier is evaluated every 6 frames once the 72-frame buffer is full. To suppress spurious alarms, a fall is confirmed only when the fall probability exceeds 0.90 on two consecutive evaluations, after which a 30-second latch holds the alert. The second person stream in the CTR-GCN input tensor is zero-filled, so the deployed service performs single-patient inference while preserving the two-person tensor contract.

#### Algorithm 4.8, Real-time fall detection (per frame)

```
buffer = ring_buffer(size=72); streak = 0
for i, frame in enumerate(stream):
    dets = yolov8.track(frame)          # boxes + persistent IDs
    if i % 10 == 0: roles = mobilenet_roles(dets) # refresh roles
    patient = first(d for d in dets if roles[d] == 'patient')
    if patient is None: continue
    joints25 = normalize(ntu_map(mediapipe_pose(crop(frame, patient))))
    buffer.push(joints25)
    if buffer.full() and i % 6 == 0:
        p_fall = ctrgcn(buffer.as_window()) # (1,3,72,25,2)
        streak = streak + 1 if p_fall >= 0.90 else 0
        if streak >= 2: emit fall_event(p_fall); latch(30s)
```

#### 4.4.1 Results and Discussion, Fall Pipeline

Evaluated with a 72-frame sliding window on the validation set (654 valid of 655 samples), the CTR-GCN classifier achieved 91.59% accuracy, 91.22% fall precision, 98.83% fall recall, a 94.87% fall F1-score, and a 95.98% ROC-AUC at the 0.50 decision threshold. The MobileNetV3 role classifier reached 80.93% validation accuracy. A threshold sweep showed that raising the decision threshold trades recall for precision on long non-fall videos, for example, a 0.95 threshold reached 93.27% accuracy with far fewer false positives.

Table 18: Fall classifier results (72-frame sliding window, threshold 0.50)

| Accuracy | Fall Precision | Fall Recall | Fall F1 | ROC-AUC |
|----------|----------------|-------------|---------|---------|
| 91.59%   | 91.22%         | 98.83%      | 94.87%  | 95.98%  |

The very high recall (98.83%) is the most important property for a safety system (a missed fall is far costlier than a false alarm), and the role-classification stage is what keeps false alarms manageable by focusing the expensive motion model on the patient alone. Impact-centered training, using synchronized IMU acceleration peaks to align each 72-frame window on the moment of impact, gave the network a consistent temporal reference that materially improved its discrimination.

Table 19: Fall-detection model weights

| Weight file                             | Model             | Purpose                                 |
|-----------------------------------------|-------------------|-----------------------------------------|
| patient_detection_yolov8n.pt            | YOLOv8-n          | Person detection and tracking.          |
| role_classification_mobilenetv3_best.pt | MobileNetV3-Large | Patient / staff / other classification. |

| Weight file                       | Model   | Purpose                                   |
|-----------------------------------|---------|-------------------------------------------|
| fall_motion_ctrgrcn_72f_impact.pt | CTR-GCN | 72-frame impact-centered fall classifier. |

The final CTR-GCN training used 72 frames per sample at an 18-FPS target, spanning two seconds before and after the IMU-localized impact, with binary fall / non-fall labels; the deployed checkpoint was selected at epoch 20. The role classifier's deployed checkpoint was selected at epoch 19 (chosen by validation macro-F1, 0.779); its best validation accuracy of 80.93% was reached at epoch 25.

## 4.5 Real-Time Seizure Detection Pipeline

The seizure subsystem detects motor seizures from a bedside camera and emits two clinical states, NORMAL and SEIZURE, with a 30-second alert latch. To reach real-time throughput it is engineered as a hybrid native-and-managed runtime. A native C++ engine, running on ONNX Runtime with CUDA/TensorRT providers, performs the high-frequency work synchronously: it decodes frames, extracts 14 clinically relevant joints with an OpenPose-18 ONNX model, runs the lightweight VSViG temporal branch to produce an instantaneous risk, and applies the series gate and latch. In parallel, a Python worker process runs the heavy ViViT transformer and its Cross-Joint classification head; the C++ engine pushes a five-second rolling buffer of keypoints to this worker over a Windows named pipe and receives back a Cross-Joint probability, which the gate fuses with the live VSViG risk. If the worker stalls (a Cross-Joint result older than 5.5 seconds), the engine seamlessly falls back to pure VSViG without blocking the monitoring loop. The gateway starts the service over HTTP and a background relay raises a single critical alert on the first frame that transitions into the seizure state.

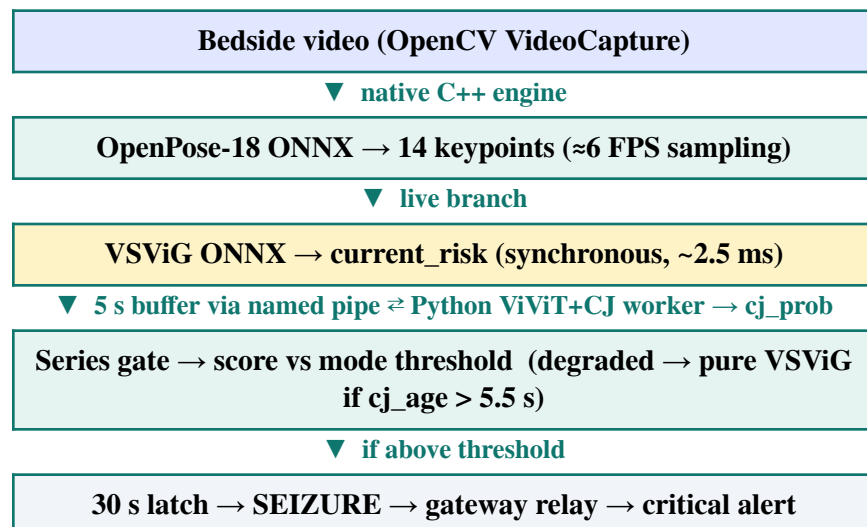


Figure 6: Real-time hybrid C++/ONNX + Python seizure-detection pipeline

#### Algorithm 4.9, Series-gate fusion (native engine)

```

v = current VSViG risk    c = latest Cross-Joint probability

if c is missing or cj_age > 5.5 s: gate = v    # CJ_PENDING / degraded
elif c >= 0.50:           gate = c            # trust CJ (CJ)
elif c >= 0.20:           gate = 0.30*c + 0.70*v  # BLEND
else:                     gate = c            # CJ_LOW (suppressor)

status = SEIZURE if (gate >= mode_threshold or latch_active) else NORMAL
if status == SEIZURE: hold latch until now + 30 s

```

Table 20: Series-gate constants and clinical sensitivity modes

| Parameter         | Value | Meaning                                                  |
|-------------------|-------|----------------------------------------------------------|
| THRESHOLD_MONITOR | 0.49  | Default mode, balanced continuous monitoring.            |
| THRESHOLD_SAFETY  | 0.88  | Safety mode, highest precision; lowest false-alert risk. |
| GATE_UPPER_BOUND  | 0.50  | Above this Cross-Joint confidence, trust it directly.    |
| GATE_LOWER_BOUND  | 0.20  | Below this, trust the non-seizure verdict directly.      |
| GATE_ALPHA        | 0.30  | Cross-Joint weight in the blend zone (rest on VSViG).    |
| ALERT_HOLD_SEC    | 30.0  | Latch hold after an alarm fires.                         |
| CJ_MAX_AGE_SEC    | 5.5   | Staleness limit; beyond it, fall back to pure VSViG.     |

The native runtime defines two clinical-sensitivity thresholds, monitor (0.49, default) and safety (0.88), which trade recall for precision; full UI-level mode selection is not yet wired into the native runtime (see §6.2). An additional held-out operating point (0.308) exists in the code for offline evaluation but is not selected at runtime.

For reproducibility: the deployed VSViG/ProtoGCN model has 6.211 million parameters and ships as a 2.24 MB ONNX graph plus a 22.56 MB weights file, with native inference around 2.5 ms. The deployed Cross-Joint head ships as a 490 KB ONNX graph plus a 13.56 MB weights file, with native inference around 2.16 ms; the frozen ViViT backbone itself (run in the Python worker, not exported to ONNX) takes approximately 52.8 ms per forward pass, well under its 217 ms token-building step and so not the bottleneck.

#### 4.5.1 Results and Discussion, Seizure Pipeline

The native runtime delivers genuine real-time performance: on the recommended hardware (an RTX 4060/5060-class GPU) it sustains a mean steady-state throughput of about 37.2 frames per second across the ten-video clinical demonstration set, comfortably above a 30-FPS camera. The asynchronous design is

what makes this possible: the Cross-Joint ONNX head itself takes only ~2.2 ms, but the full ViViT token-building step (~217 ms) runs off the critical path in the Python worker, so it never throttles the native loop. Segment-level and clinical-event metrics are summarized below.

Table 21: Seizure detector, segment-level classification metrics (952 segments)

| AUROC  | AUPRC  | Accuracy | F1     | Precision | Recall |
|--------|--------|----------|--------|-----------|--------|
| 0.9669 | 0.9429 | 0.9128   | 0.9018 | 0.8319    | 0.9845 |

*The 952-segment test split (565 interictal, 387 ictal, drawn from 14 EMU patients out of 2,722 total segments) is inferred from published per-class counts; exact patient assignment is not author-confirmed.*

Two further ablations were run during development and are noted here for completeness. A skeleton-only Channel-wise Topology Refinement GCN (CTR-GCN) classifier — using joint, bone, and motion streams identically to the fall-detection classifier but trained on seizure labels — reached only AUROC 0.852 (3-stream ensemble) versus the Cross-Joint head's 0.958, confirming that the frozen ViViT features carry discriminative information beyond skeleton topology alone. Separately, an attempt to improve early-ictal recall by soft-labeling the 30-second pre-EEG window and weighting near-onset clips as hard negatives collapsed precision to 0.580 (from a Cross-Joint reference precision in the 0.72–0.81 range), because much of that pre-EEG window is genuinely interictal; this approach was rejected and the standard hard-label training was retained. A third variant, using overlapping rather than back-to-back 5-second Cross-Joint segments, was also tried in the hope of smoother score updates; it instead lowered the achievable update rate and degraded both Latency to EEG Onset and Latency to Clinical Onset relative to the non-overlapping production configuration, and was rejected for that reason. A related runtime experiment compared a 1.0 s Cross-Joint slide interval (the deployed configuration) against a 2.0 s interval intended to reduce computational load. The longer interval nearly doubled steady-state FPS (from ~17 to ~27) but increased mean Cross-Joint packet age from 0.94 s to 1.36 s and caused one of eight test recordings to shift from a successful detection to a borderline-late one; the 1.0 s interval was retained because it gave the more reliable clinical outcome even though the throughput it requires is lower.

Table 22: Seizure detector, clinical runtime evaluation (10 videos)

| Metric                | Value                             |
|-----------------------|-----------------------------------|
| Successful detections | 9 of 10 (0 misses, 1 early false) |

| Metric                               | Value                           |
|--------------------------------------|---------------------------------|
| Mean Latency to EEG Onset (LEO)      | 5.86 s                          |
| Mean Latency to Clinical Onset (LCO) | −3.94 s (before clinical onset) |
| False Alert Rate (FAR)               | 5.3 / hour                      |
| Mean steady-state throughput         | 37.21 FPS                       |

*Per-video steady-state FPS ranges from 31.6 to 40.7 across the ten demonstration videos, with the lowest-throughput recording still exceeding the 30-FPS clinical requirement.*

The series gate remains the key engineering contribution: a cheap, always-on native branch provides instantaneous responsiveness while a heavier, asynchronous transformer periodically corrects it, and a graceful degradation path keeps the system live even if the worker stalls. The high recall (0.9845) is appropriate for a safety system, at the cost of a still-elevated false-alert rate that the team identifies as the main target for improvement. Each frame written to the alert CSV records the full decision context: the frame index and timestamp, the status (INITIALISING / NORMAL / SEIZURE), the fused signal and its driving source (CJ, CJ\_LOW, BLEND, or INITIALISING), the instantaneous VSViG risk, the latest Cross-Joint probability and its age, and whether the 30-second latch is active, giving a complete, auditable trace of every decision. The acknowledged limitations are that only visible motor manifestations are detected, that the false-positive rate is sensitive to domain shift, and that the underlying study used a single-institution, 14-subject EMU cohort, so cross-site generalization remains to be validated.

## 4.6 Frontend Single-Page Application

The client is a React 18 single-page application built with Vite, styled with Tailwind CSS, and backed by lightweight Zustand stores. It communicates with the gateway over REST (via Axios) and over persistent WebSockets for live telemetry and alerts. A single environment flag switches the whole app between a mock mode (simulated data and timers for offline demonstration) and a live mode (real backend calls), which makes development and demonstration independent of a running backend.

Routing and access control are declarative. A custom route guard reads the decoded user from the auth store and redirects users away from any route that does not match their role, sending each role to its own dashboard. Two stores hold global state: an auth store persists the JWT to local storage and decodes the user's id, name, and role; an alerts store keeps the most recent live alerts (capped at 200) and an unread counter.

### Algorithm 4.10, Client-side role guard

```
function RoleGuard(allowedRole):
  user = authStore.user
  if user is null:      redirect('/login')
  elif user.role != allowedRole: redirect(homeFor(user.role))
  else:                render(child routes)
```

Real-time behavior is encapsulated in two hooks. An alerts hook maintains the connection to the alert WebSocket with exponential-back-off reconnection and dispatches incoming alert frames into the alerts store. A chat-stream hook manages the chat WebSocket, interpreting the `think_start` / `think` / `think_done` frames to populate an expandable reasoning panel separately from the final answer, and opening an SSE connection for image uploads. The interface also includes a Recharts-based ECG viewer that renders selectable 12-lead waveforms alongside the arrhythmia service’s two-stage classification and probability bars, and a live-monitoring grid that renders fall and seizure camera tiles with NORMAL / INITIALISING / ALERT states and a depleting countdown bar that mirrors the backend’s 30-second seizure latch.

The alert feed is also audible: when a seizure or fall alert arrives, the client plays a looping alarm that automatically stops after 30 seconds, and the acknowledge, cancel, and expire events mutate the local alert list in place without a page refresh. Room cards, driven by the `room_update` broadcasts, show each room’s patient, enabled services, assigned nurses, live status, and risk score, giving staff a single ward-level view.

A small library of reusable clinical UI components (buttons with loader states, cards, modals, inputs, and color-coded severity and status badges) gives the application a consistent, professional clinical look. The components and pages are summarized below.

*Table 23: Principal frontend pages by role*

| Role   | Pages                                                                                                                 |
|--------|-----------------------------------------------------------------------------------------------------------------------|
| Auth   | Login (with demo-credential loader).                                                                                  |
| Doctor | Dashboard, Rooms board, Patient List, Patient Detail (Overview · Chat · ECG · logs), Live Monitoring, Sandbox Test.   |
| Nurse  | Triage Dashboard and room cards for assigned rooms, focused on unacknowledged alerts.                                 |
| Admin  | Service-health Dashboard, User-Management page, Room-Management page (create rooms, assign patients/nurses/services). |



## 4.7 Data Import Pipeline

Two paths populate the database. A development seeder creates one account per role (doctor, nurse, admin), ten synthetic patients with diagnoses, medications, visits, labs, and occasional allergies, and random patient–clinician assignments. A production importer ingests Synthea CSV exports into the schema. Because some Synthea files are very large (the observations file exceeds 400 MB), the importer reads in fixed-size chunks to bound memory, maps each source UUID to an integer key, and applies per-table transformations.

### Algorithm 4.11, Chunked Synthea import

```
uuid_map = {}
for chunk in read_csv('patients_localized.csv', chunksize=10000):
    for row in chunk:
        p = insert Patient(name=FIRST+''+LAST, dob=BIRTHDATE,
                           gender=GENDER, blood_type=random())
        uuid_map[row.Id] = p.id
for file, table, mapping in [conditions->diagnoses, medications,
                             observations->lab_results, allergies]:
    for chunk in read_csv(file, chunksize=10000):
        rows = [ transform(r, uuid_map) for r in chunk
                  if valid(r) ]      # e.g. numeric VALUE only
        bulk_insert(table, rows)
```

Table 24: Synthea-to-schema mapping (imported tables)

| Synthea file           | Target table | Notes                                                        |
|------------------------|--------------|--------------------------------------------------------------|
| patients_localized.csv | patients     | UUID→int map; blood type randomly assigned; localized names. |
| conditions.csv         | diagnoses    | CODE→icd10_code; NULL STOP ⇒ active.                         |
| medications.csv        | medications  | NULL STOP ⇒ active; default dose “As directed”.              |
| observations.csv       | lab_results  | Numeric VALUE rows only (~60% text rows dropped).            |
| allergies.csv          | allergies    | REACTION1 / SEVERITY1 if present.                            |

Table 25: Synthea export files and import status

| File                   | Approx. size             | Imported?         |
|------------------------|--------------------------|-------------------|
| patients_localized.csv | 660 KB (~1,000 patients) | Yes → patients    |
| conditions.csv         | 14.5 MB (~15,000)        | Yes → diagnoses   |
| medications.csv        | 43.7 MB (~50,000)        | Yes → medications |

| File                                                 | Approx. size           | Imported?                        |
|------------------------------------------------------|------------------------|----------------------------------|
| observations.csv                                     | 409 MB (~1M)           | Yes → lab_results (numeric only) |
| allergies.csv                                        | 379 KB (~3,000)        | Yes → allergies                  |
| encounters.csv / procedures.csv                      | 55.8 MB / 97.8 MB      | No                               |
| imaging_studies / claims / careplans / devices / ... | various (up to 1.3 GB) | No                               |

Table 26: Data-quality handling during import

| Issue                                     | Handling                                                     |
|-------------------------------------------|--------------------------------------------------------------|
| Synthea generates no blood types          | Randomly assigned with a realistic distribution.             |
| ~60% of observations are non-numeric text | Filtered out; only numeric VALUE rows imported.              |
| Medications lack detailed dosing          | Default dose “As directed”.                                  |
| Conditions use SNOMED CT, not ICD-10      | Stored in the icd10_code field as-is.                        |
| Localized patient names                   | patients_localized.csv substitutes region-appropriate names. |

A full import of roughly 1,000 patients, 15,000 conditions, 50,000 medications, ~200,000 numeric observations, and 3,000 allergies completes in about eleven minutes on a typical development machine while keeping memory under ~500 MB. A small post-import utility back-fills blood types for imported patients.

## 4.8 Integration and Inter-Service Communication

The gateway never embeds the heavy models directly; instead it talks to each inference service (arrhythmia on port 8001, fall on 8002, and seizure on 8003) through a dedicated asynchronous client module, with the service URLs configurable by environment variable. Each client exposes a health check, the operations that service supports, and graceful error handling so that a missing or slow service degrades cleanly rather than crashing the gateway. The dashboard and monitoring-status endpoints call all three health checks in parallel and surface the results so that staff can see, at a glance, which services are online. A single launcher script (start\_all\_services.ps1 on Windows, start\_all\_services.sh on Unix) starts the three inference services and the gateway together, each as its own Uvicorn process.

### 4.8.1 Arrhythmia Client

The arrhythmia client performs a simple request/response call: it posts the ECG array and optional QRS features to the service and returns the cascade result, or an error object if the service is unreachable. The arrhythmia router then maps an abnormal result to a severity (AF → high, IAVB → medium, otherwise by confidence) and publishes an alert.

### 4.8.2 Fall Camera Bridge

Fall detection is stream-oriented. A camera client opens a WebSocket to the gateway, which in turn opens a WebSocket to the fall service and runs two concurrent relays: one forwards incoming JPEG frames to the service, the other forwards the service's per-frame events back to the camera client and, when a fall is detected for a numeric room identifier, publishes a critical alert. This bridge pattern keeps the fall service unaware of authentication and database concerns, which remain the gateway's responsibility.

#### Algorithm 4.12, Fall camera bridge (two concurrent relays)

```
on camera_ws connect(room_id):
    fall_ws = connect(fall_service_url(room_id))
    relay_A: for frame in camera_ws.bytes(): fall_ws.send(frame)
    relay_B: for ev in fall_ws.messages():
        camera_ws.send(ev)
        if ev.fall_detected and room_id is numeric:
            alert_manager.publish(patient_id=room_id,
                                type='fall', severity='critical', ev)
    run relay_A and relay_B concurrently
```

### 4.8.3 Seizure Event Relay and Internal IPC

Starting a seizure session causes the seizure service to launch the native C++ runtime executable as a subprocess and an in-process Python token-producer thread for the same session, then tail the runtime's CSV output. Internally, the C++ engine and the Python producer communicate through a Win32 named pipe (one per session) carrying binary packets that begin with a four-byte magic header, followed by three timing doubles and the ViViT token and position tensors. Externally, the gateway spawns a background task that subscribes to the service's event stream and publishes an alert only on the transition into the seizure state, so a sustained episode produces a single alert rather than a storm. Sessions are keyed by patient identifier, and both starting and stopping a session are access-controlled. An earlier ONNX export of the Cross-Joint head silently produced incorrect probabilities (delaying a test alert by several seconds); the deployment pipeline now requires fixture-based parity verification between the PyTorch and exported ONNX models before any model is shipped.

A related lesson from development is that VSViG probability values are not comparable across pipeline versions: switching frame normalization, keypoint ordering, and the inference runtime (PyTorch → ONNX) each shifted per-clip VSViG scores by tens of percentage points even though the model itself was unchanged, so only the end-to-end clinical metrics (Tables 21–22), not raw intermediate scores, should be used to judge runtime correctness.

*Table 27: Inference-service interfaces and failure handling*

| Service (port)    | Call pattern                                                         | On failure                                                          |
|-------------------|----------------------------------------------------------------------|---------------------------------------------------------------------|
| Arrhythmia (8001) | Async POST request/response                                          | Returns error object; no alert raised.                              |
| Fall (8002)       | Bidirectional WebSocket relay                                        | Bridge closes; camera client notified.                              |
| Seizure (8003)    | HTTP start + event subscription;<br>C++/Python named-pipe internally | Start returns 503; relay task exits quietly; CJ stall → VSViG-only. |

This loosely coupled integration is what makes the platform a true microservices system: each model can be developed, deployed, scaled, or replaced independently, and the gateway remains a thin, responsive coordinator. With every component implemented, the next chapter turns to installing, running, and testing the system end-to-end.

## Chapter Five

# System Testing

---

- 5.1 Test Environment & Prerequisites
- 5.2 Installation & Running the System
- 5.3 User-Interface Walkthrough
- 5.4 Automated Test Suites
- 5.5 API Reference Summary
- 5.6 Consolidated Benchmark Results

## Chapter 5: System Testing

This chapter explains how to install, run, and test Shifaa. It begins with the prerequisites and the step-by-step launch procedure, then walks through the user interface screen-by-screen, before presenting the automated test suites, an API reference, and a consolidated view of the benchmark results that validate the system.

### 5.1 Test Environment and Prerequisites

| Requirement                  | Version                         | Notes                                                                  |
|------------------------------|---------------------------------|------------------------------------------------------------------------|
| Python                       | 3.10+                           | Backend and inference services.                                        |
| Database                     | SQLite (bundled)                | Default; no setup required. PostgreSQL 14+ optional for production.    |
| Node.js                      | 18+                             | Frontend build and dev server.                                         |
| Ollama (optional)            | latest                          | Local MedGemma 1.5 backend for full privacy.                           |
| NVIDIA GPU + CUDA            | 11.8 / 12.x                     | Required for real-time vision and seizure inference.                   |
| ONNX Runtime + C++ toolchain | ORT 1.23+, VS 2022, CMake 3.16+ | Windows-only; builds the native seizure runtime (uses named-pipe IPC). |

Test credentials are created by the database seeder, one account per role:

| Role   | Email               | Password  |
|--------|---------------------|-----------|
| Doctor | doctor@hospital.com | doctor123 |
| Nurse  | nurse@hospital.com  | nurse123  |
| Admin  | admin@hospital.com  | admin123  |

### 5.2 Installation and Running the System

#### 5.2.1 Backend, Database, and Services

Create the environment, configure secrets, seed the database, and launch the whole stack. By default the database is a self-contained SQLite file (medical\_db.db) created automatically on first run, so no separate database server is required; to use PostgreSQL instead, simply point DATABASE\_URL at it.

```

# 1. Environment
conda create -n medical_chatbot python=3.10 -y
conda activate medical_chatbot
pip install -r requirements.txt

# 2. Configure .env (SECRET_KEY, LLM_BACKEND, OLLAMA_MODEL, service ports)
cp .env.example .env
# DATABASE_URL is optional; defaults to sqlite:///./medical_db.db

# 3. Seed test data (one user per role + 10 patients + rooms)
python -m backend.seed

# 4. (Optional) local LLM
ollama pull medgemma1.5:latest && ollama serve

# 5a. Start EVERYTHING (gateway :8000 + arrhythmia/fall/seizure)
.\start_all_services.ps1      # Windows  (start_all_services.sh on Unix)

# 5b. ...or start just the gateway
uvicorn backend.main:app --reload --host 0.0.0.0 --port 8000

```

The launcher starts the gateway on port 8000 and the arrhythmia, fall, and seizure services on 8001–8003, skipping any port already in use and writing each service’s output to the logs directory. The interactive API documentation is then available at <http://localhost:8000/docs>. The native seizure runtime must be built once with CMake/Visual Studio before the seizure service can run (see its module README); the other services run directly from Python.

## 5.2.2 Frontend

```

cd frontend-react
cp .env.example .env      # set VITE_API_URL, VITE_WS_URL, VITE_DATA_MODE
npm install
npm run dev               # Vite dev server on http://localhost:5173

```

Setting `VITE_DATA_MODE` to `mock` runs the entire UI against simulated data for a self-contained demonstration; setting it to `live` connects to the running backend. The three inference services (arrhythmia, fall, seizure) are each launched from their own folders following the instructions in their README files.

## 5.3 User-Interface Walkthrough

This section walks through the application screen-by-screen, with a screenshot of each page. The interface is branded Shifaa and is organized into three role-specific portals (Doctor, Nurse, and Admin), each

reachable only by a user holding that role. The walkthrough follows a doctor through the clinical screens first, then covers the demonstration sandbox, the nurse portal, and the admin portal.

### 5.3.1 Login and Role-Based Entry

The application opens on a dark, branded login screen headed “Clinical decision support for every role,” which also previews the three monitoring modules (arrhythmia, fall, and seizure). The clinician signs in with an email and password; quick demo-credential chips can pre-fill the doctor, nurse, or admin test accounts. On success the gateway returns a JWT, the auth store decodes the user’s role, and the route guard redirects the user to their role-specific dashboard.

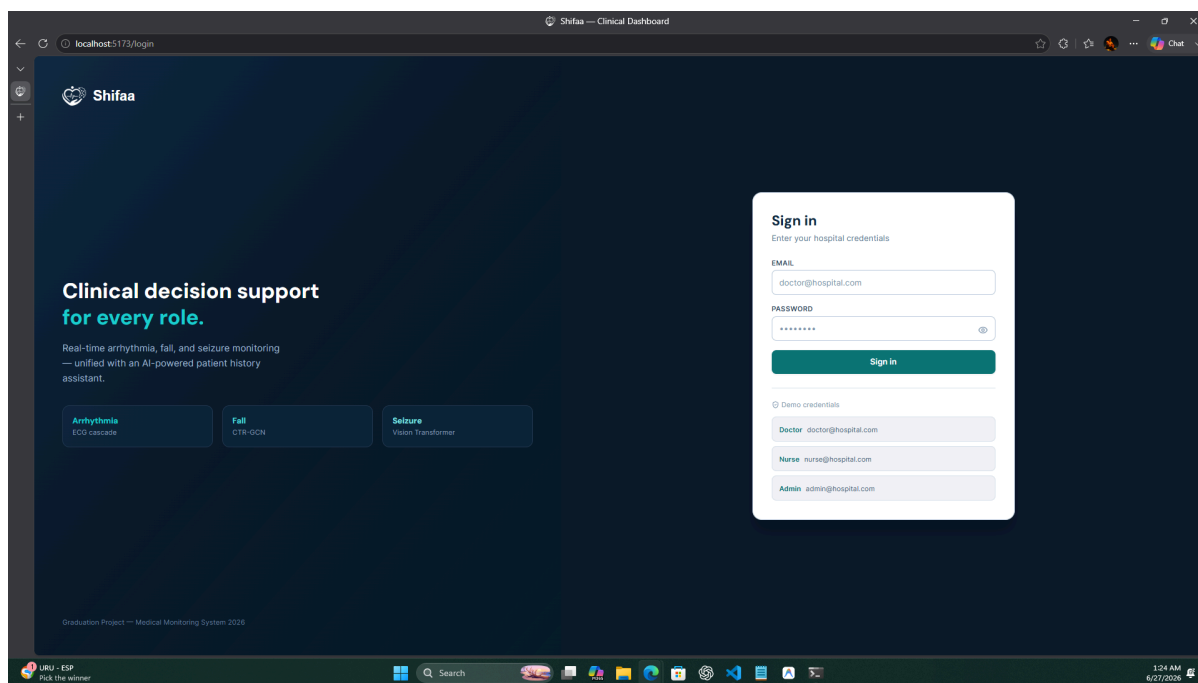


Figure 7: Login screen with role-based demo credentials

### 5.3.2 Doctor Dashboard

The doctor’s Clinical Dashboard summarizes the last 24 hours at a glance. Counter tiles across the top show how many seizure, ECG/arrhythmia, fall, and chat events occurred. The central Active Alerts feed lists unacknowledged alerts, each with its type, severity, patient, and inline acknowledge and open controls. On the right, a Service Health panel reports the live status of the main backend and the three inference services, and a Quick Actions panel links to the patient list and live monitoring.



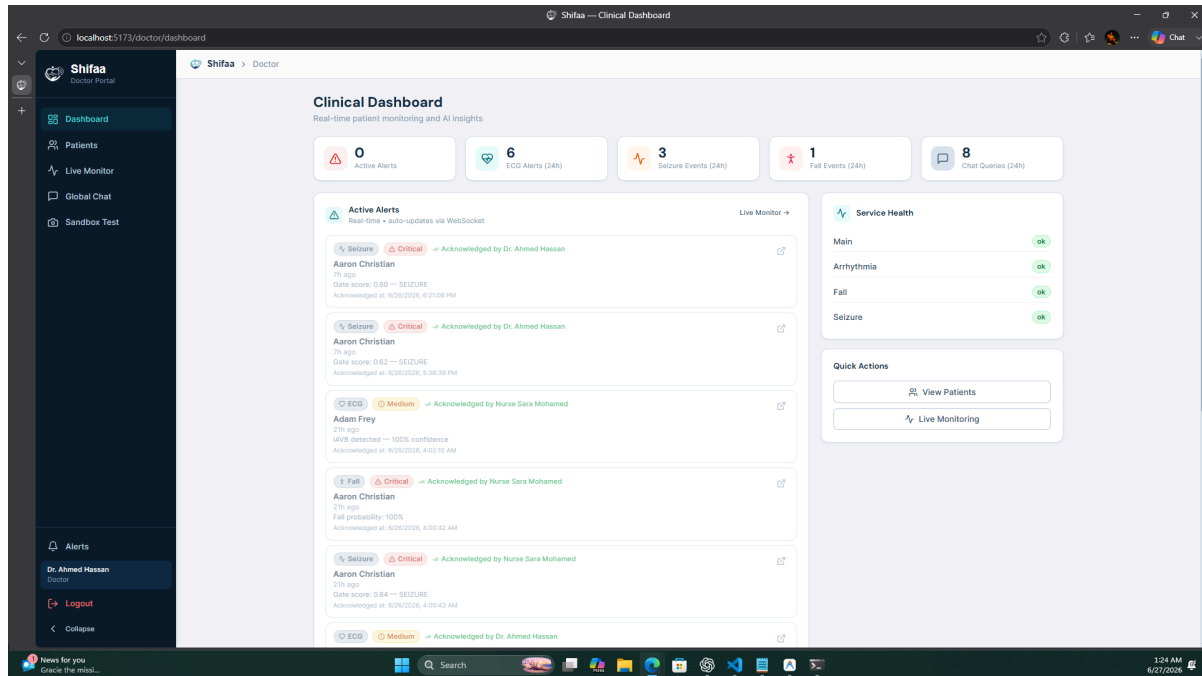


Figure 8: Doctor Clinical Dashboard: alert counters, active-alert feed, and service health

### 5.3.3 Patient Hub

The Patients page is the doctor’s patient hub. It lists every patient with a live count, a name search box, and an “Add Patient” action. Each row shows the patient’s name and ID, age and gender, blood type, active diagnoses, and a current alert status (for example, “Clear”). Selecting a row opens that patient’s detail view.

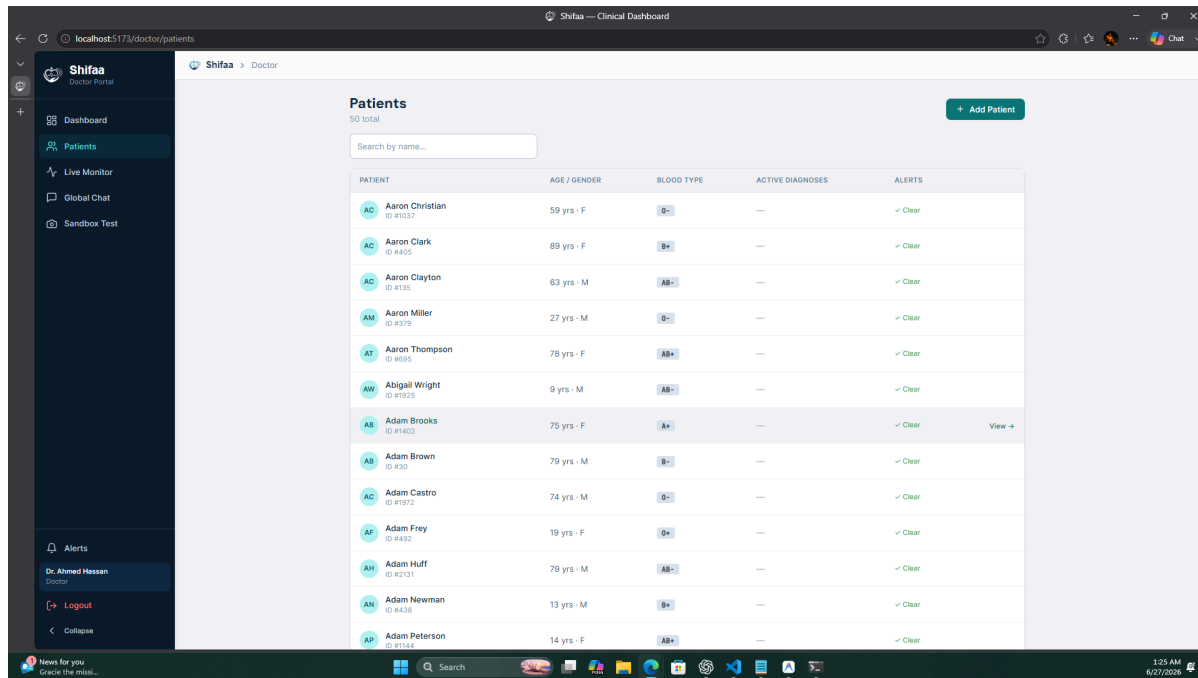
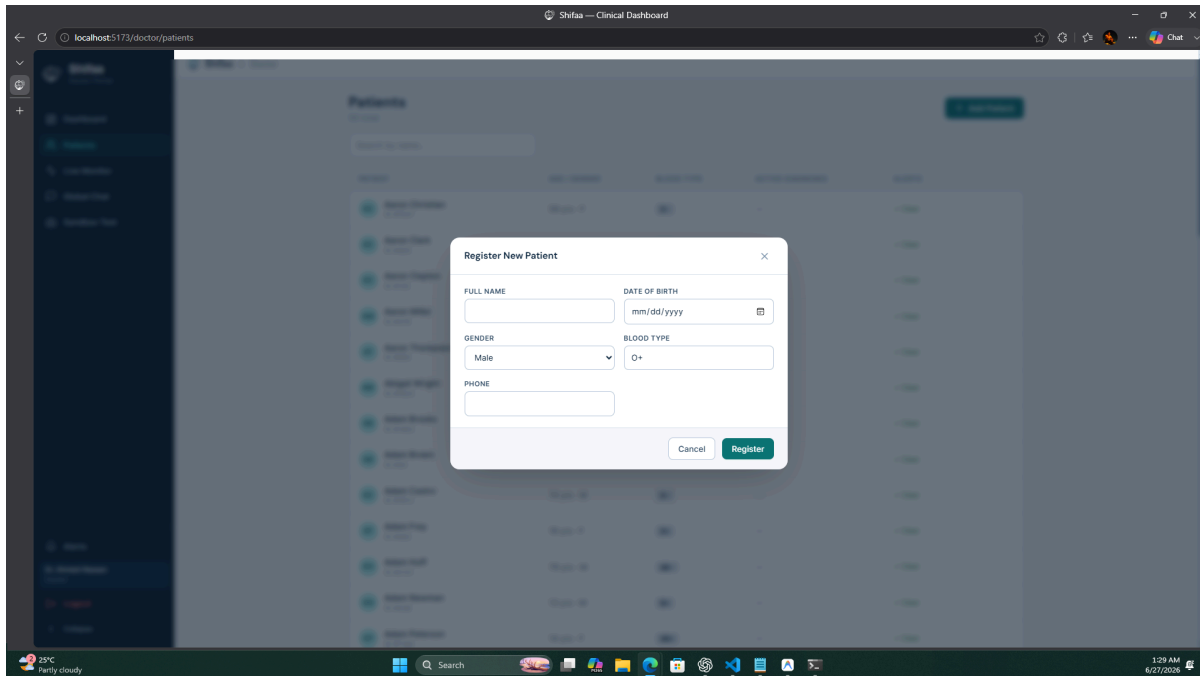


Figure 9: Patient hub: searchable patient list with demographics and alert status

### 5.3.4 Registering a New Patient

Clicking “Add Patient” opens the Register New Patient modal. The form captures the patient’s full name, date of birth, gender, blood type, and phone number. On submission the gateway creates the patient through POST /api/patients and the new record appears immediately in the patient hub.



*Figure 10: Register New Patient modal*

### 5.3.5 Patient Detail: Overview

Opening a patient presents a tabbed detail view with three tabs: Overview, Chat, and Monitoring. The Overview tab shows the patient's complete clinical record at a glance, with the list of active diagnoses on the left and current medications on the right, each entry dated and labelled. This is the read-only clinical summary the doctor reviews before asking the assistant a question.

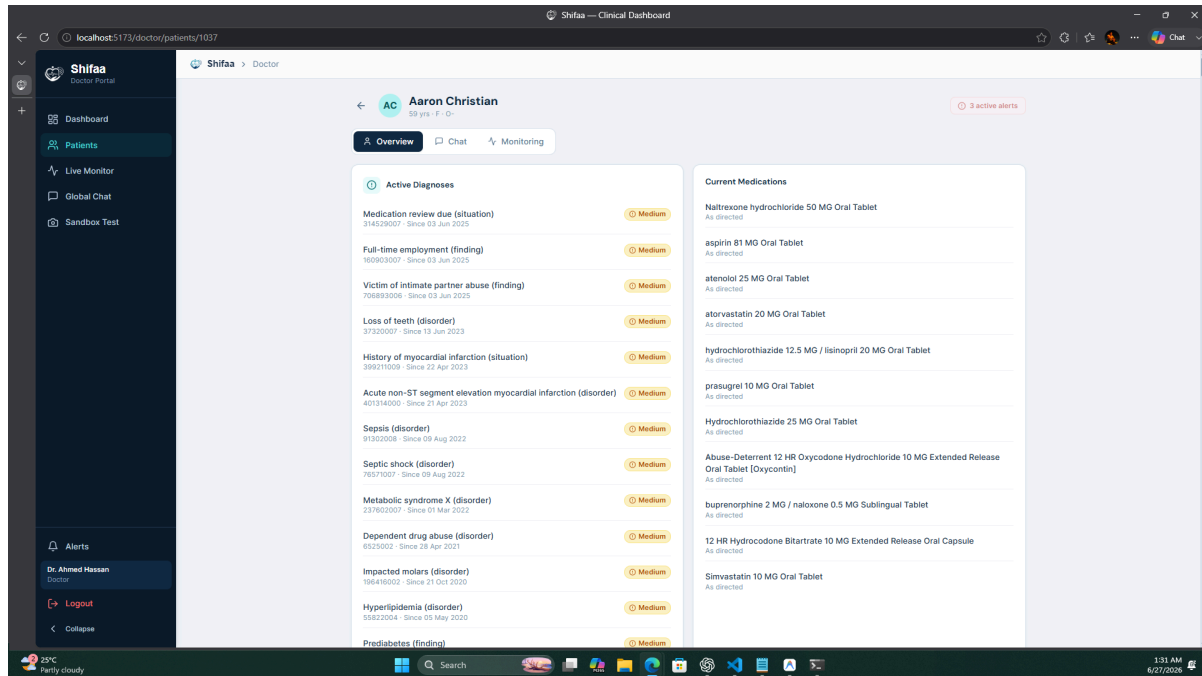


Figure 11: Patient detail: Overview tab (active diagnoses and current medications)

### 5.3.6 Patient Detail: Specialized Chat

The Chat tab is a patient-grounded conversation: every answer is restricted to that patient's record. The clinician asks a question in natural language (for example, "What is the patient's medical history?") and the answer streams back token-by-token. While MedGemma reasons, an expandable "Thought for N s" block exposes its step-by-step clinical reasoning; once reasoning ends the block collapses and the final, source-cited answer is rendered, here a structured medical history and current-medications summary drawn directly from the database.

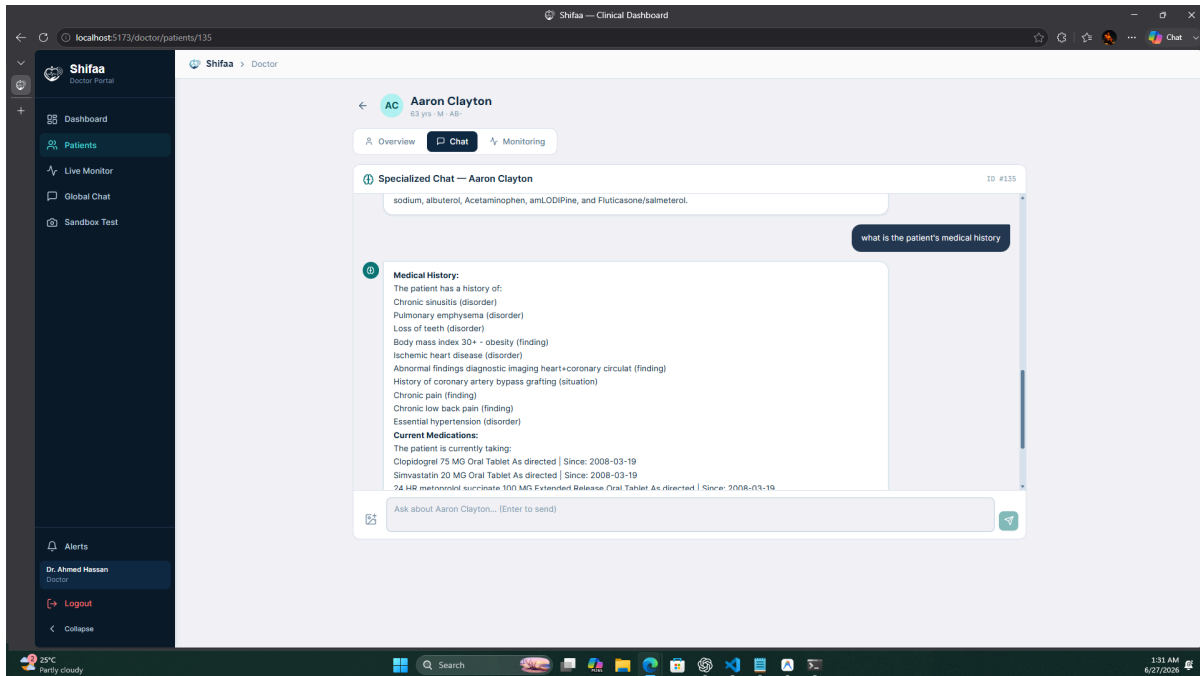


Figure 12: Patient-grounded specialized chat with a cited medical-history answer

The reasoning view is shown live as the model thinks; the screenshot below captures the collapsible “Thought for 0s” block at the moment a new question is submitted, before the grounded answer is streamed.

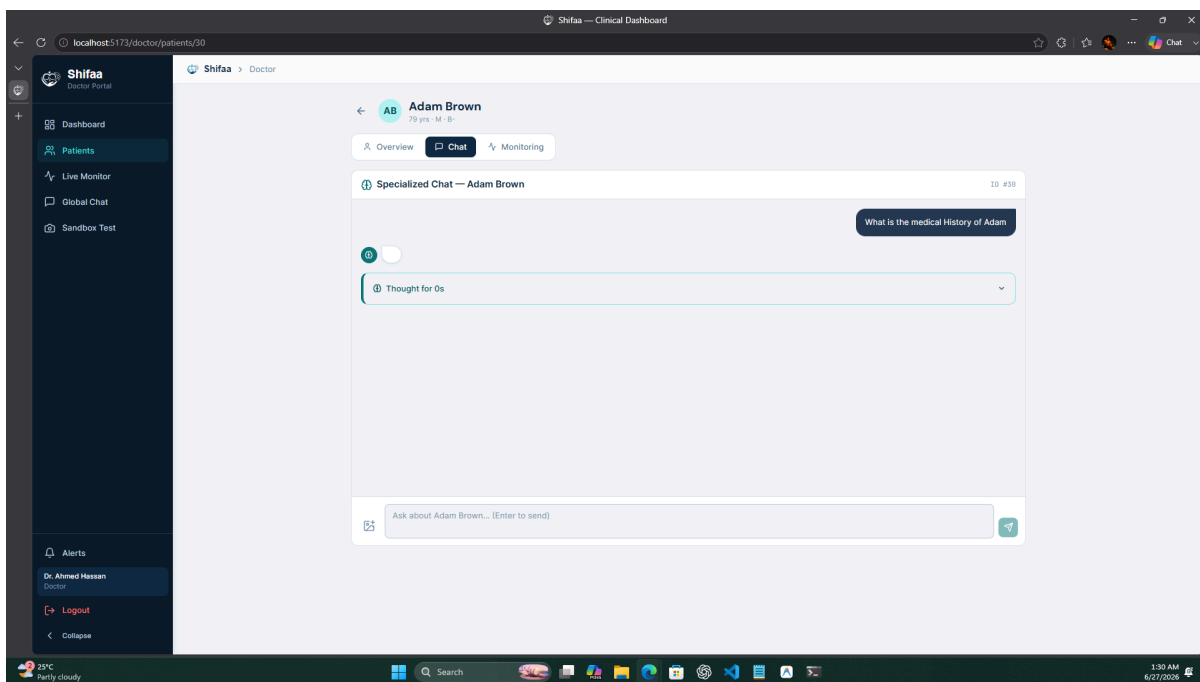


Figure 13: Specialized chat: live reasoning (thinking) block

### 5.3.7 Patient Detail: Monitoring History

The Monitoring tab is the per-patient alert history. It lists every AI-generated alert for the patient in reverse-chronological order, each card showing the alert type (seizure, fall, or arrhythmia), its severity, the time, and the driving evidence, a seizure gate score, a fall probability, or a detected arrhythmia class with confidence. Acknowledged alerts are annotated with the acknowledging clinician and timestamp, providing a complete audit trail at the bedside.

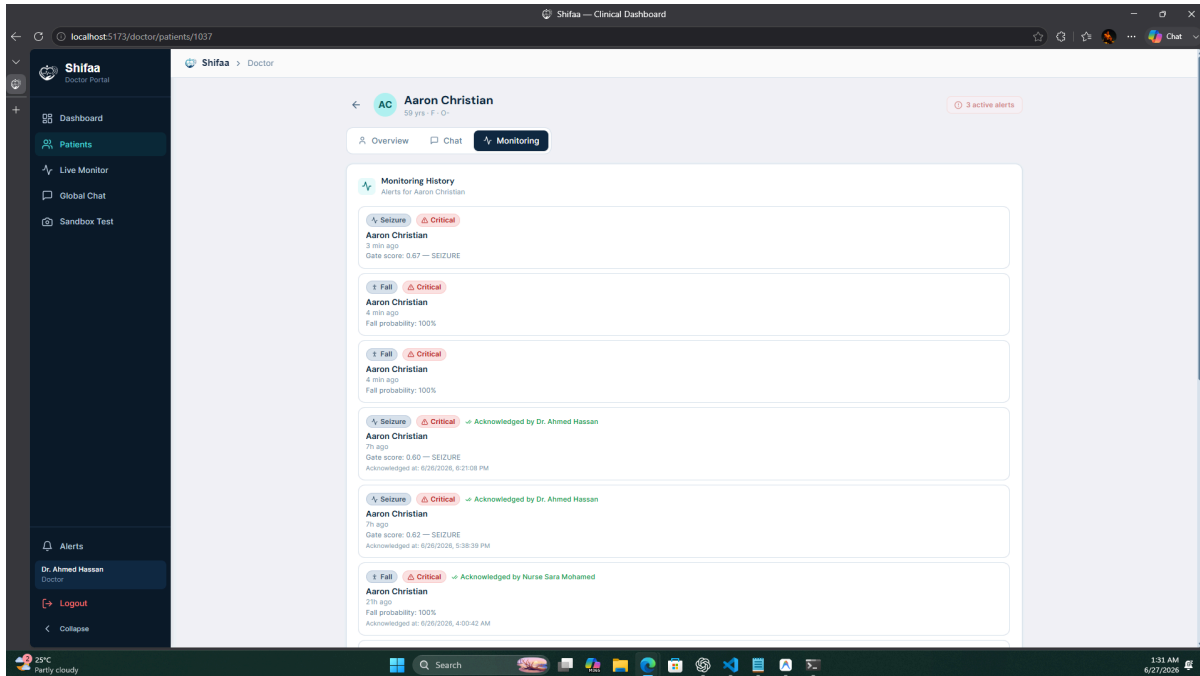


Figure 14: Patient detail: Monitoring history (per-patient alert audit trail)

### 5.3.8 Global Chat

Global Chat is a general medical question-and-answer mode that is independent of any single patient. It is intended for general clinical knowledge and for multi-modal image analysis: the clinician can attach a medical image (for example, an MRI) and ask the model to describe and analyze it. The assistant streams a structured radiological-style read, and, as in the patient chat, exposes its reasoning in a collapsible block.

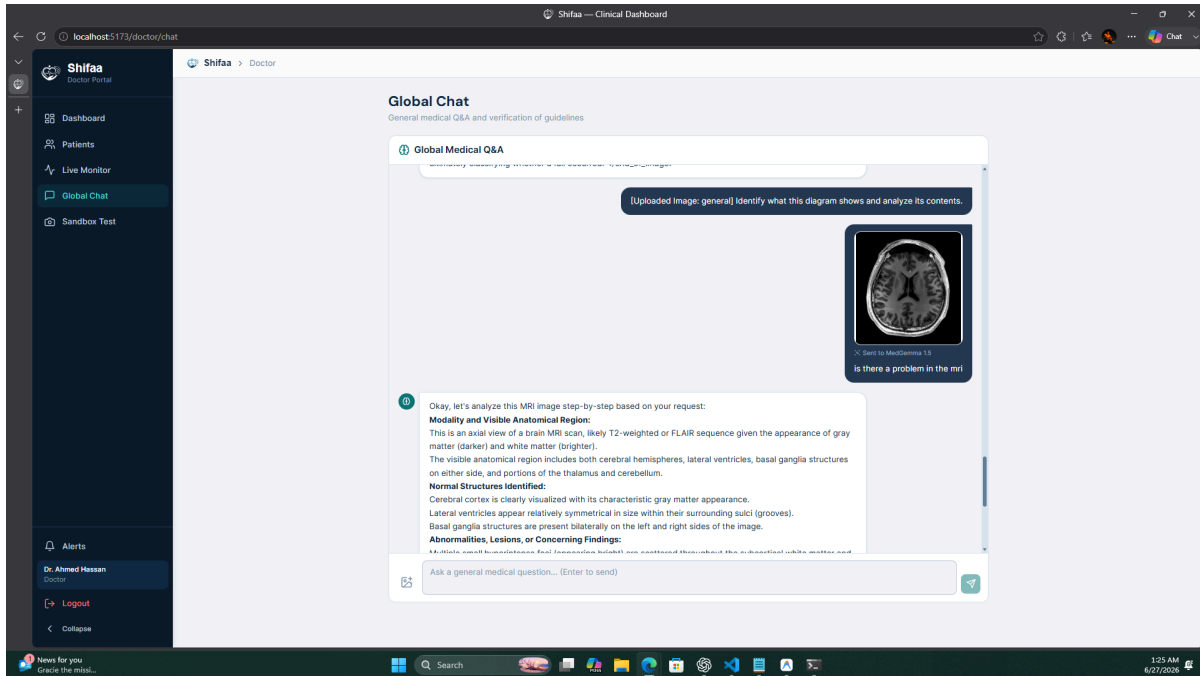


Figure 15: Global Chat: multi-modal MRI upload and AI analysis

Because Global Chat is not bound to a patient record, when it is asked a patient-specific question it correctly declines rather than inventing an answer, replying that the records do not contain the information, demonstrating the same strict grounding discipline used throughout the system.

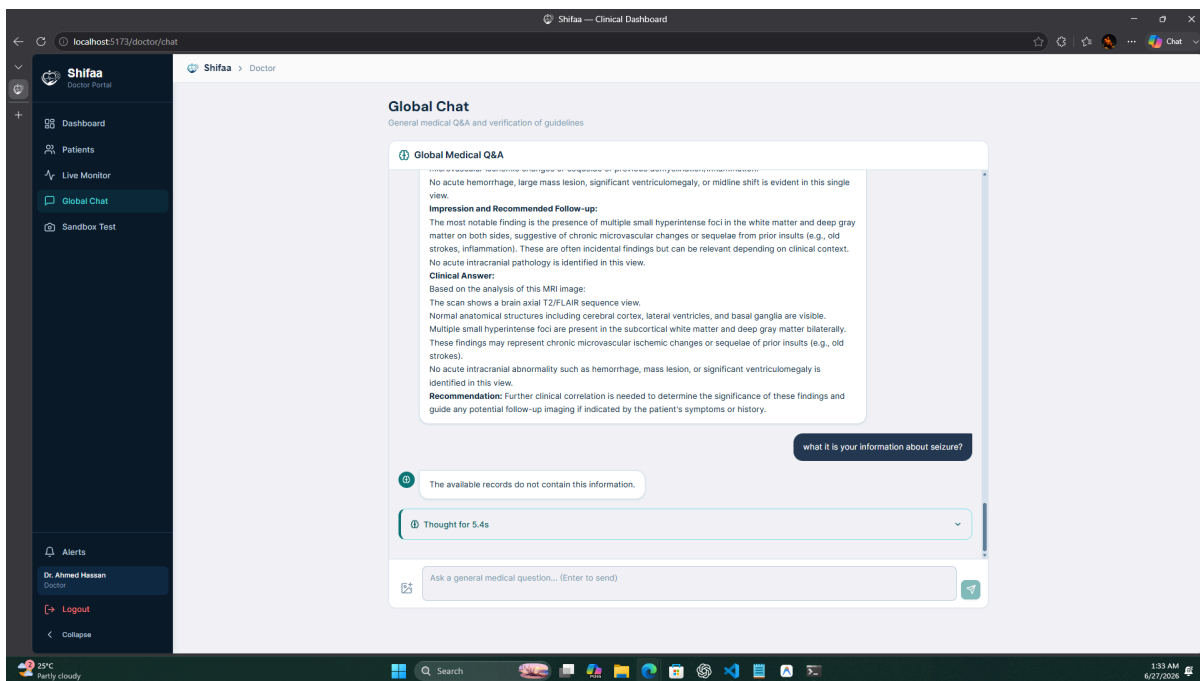


Figure 16: Global Chat: grounded refusal and reasoning block



### 5.3.9 Live Monitoring: Ward Central Station

The doctor's Live Monitor is the Ward Central Monitoring Station, an aggregated view of every monitored bed. Filter controls (by service, nurse, status, and room number) sit above a grid of room cards; each card shows the room number and floor, the assigned patient and ID, a live status (Monitoring or Idle), a “Signals Stable” indicator, the responsible nurse, and per-service risk icons. A doctor sees all rooms across the ward.

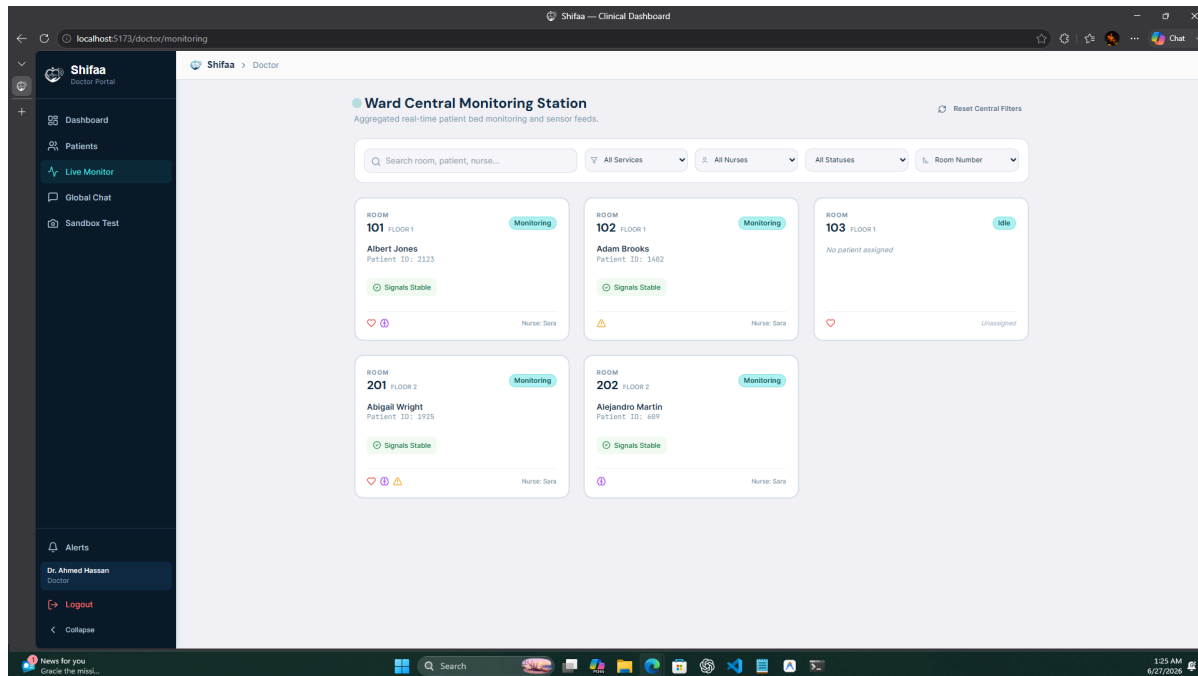


Figure 17: Ward Central Monitoring Station (doctor view of all rooms)

### 5.3.10 Room Monitoring View

Selecting a room opens its dedicated monitoring page. It combines, in one screen, the live camera stream relay (used by the fall and seizure detectors), a real-time ECG trace, the patient identity card, the room's assigned services and nurse, and a scrolling system-diagnostic log that records session start-up, camera-relay connection, and service configuration. This is the single-room equivalent of the ward station.

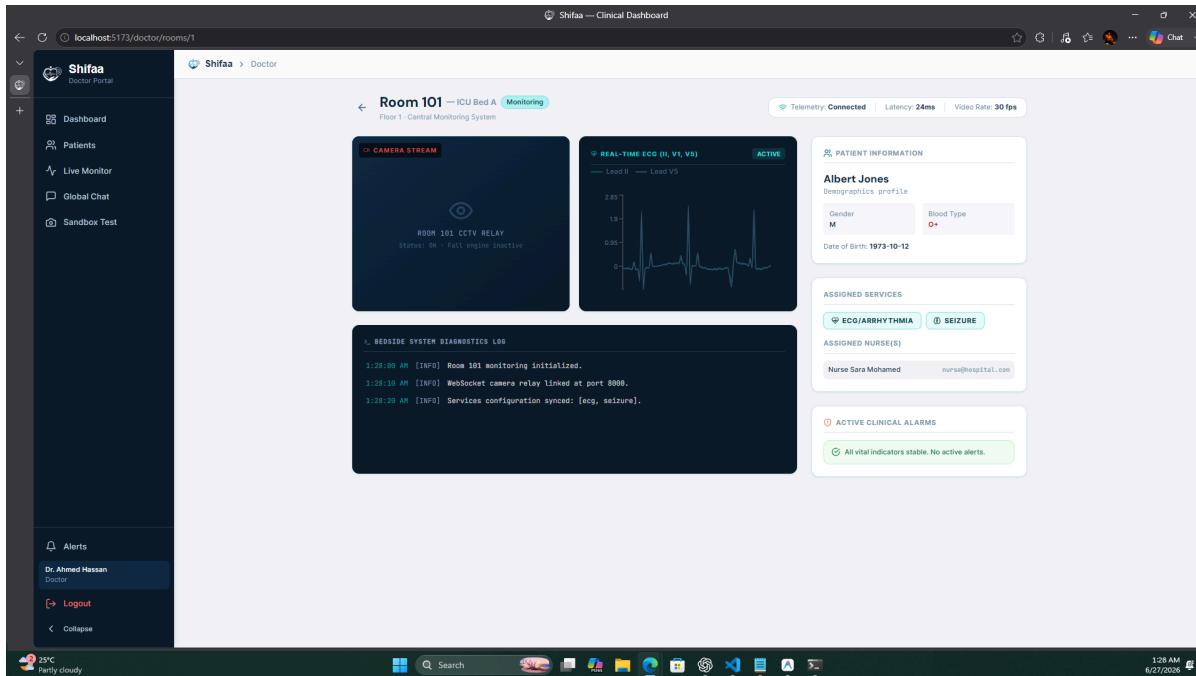


Figure 18: Single-room monitoring view (camera relay, live ECG, and diagnostics log)

### 5.3.11 Model Testing Sandbox (Demonstration)

The Model Testing Sandbox is provided for demonstration and verification only; it is not part of the routine clinical workflow. It lets a doctor stream a recorded signal or video, or a live webcam feed, through the actual deployed models and watch them respond in real time. The sandbox is split into two tabs, Vision Models (Fall & Seizure) and ECG Model (Arrhythmia), and a Live Log Feed records every model event. The table summarizes what each case exercises.

Table 28: Model Testing Sandbox: cases and models

| Case             | Model exercised                                   | Input modes                     | Output shown                                                                       |
|------------------|---------------------------------------------------|---------------------------------|------------------------------------------------------------------------------------|
| ECG / Arrhythmia | Two-stage cascade (binary screen + sub-type)      | Pre-recorded 12-lead ECG record | Live ECG trace, Stage 1 Normal/Abnormal, Stage 2 class, per-class probability bars |
| Fall             | CTR-GCN fall classifier (with YOLO + role + pose) | System webcam or uploaded video | Annotated video, fall probability, 30-second alert latch                           |
| Seizure          | Series-gate (VSViG + Cross-Joint) detector        | System webcam or uploaded video | Annotated video, gate score, INITIALISING/SEIZURE state, 30-second latch           |

ECG case. On the ECG tab the clinician selects a target patient and a sample 12-lead record, then starts the stream. The waveform animates in real time while the Arrhythmia Screening panel reports the cascade result, here a Stage 1 “Abnormal” screen followed by a Stage 2 class with a full bar chart of per-class probabilities (Normal, AF, IAVB, SB, STach).

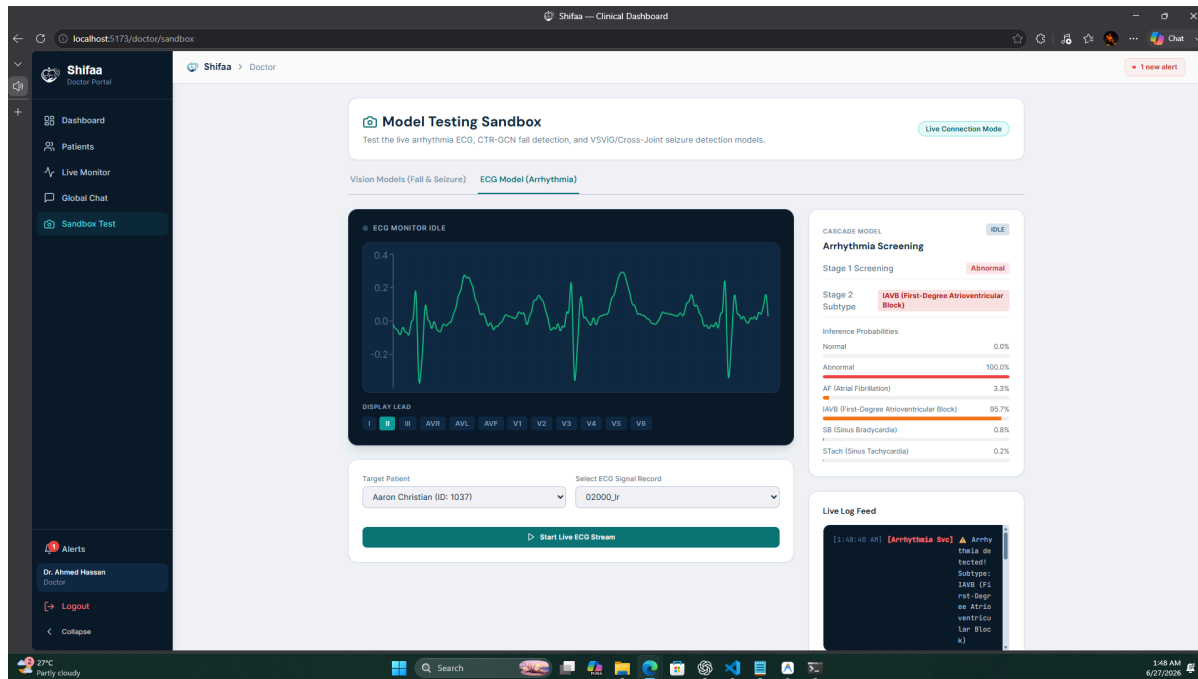


Figure 19: Sandbox, ECG arrhythmia cascade: live trace with Stage 1/Stage 2 results and probabilities

Fall case. On the Vision tab the clinician chooses a webcam or an uploaded video and starts fall detection. The CTR-GCN classifier processes the tracked patient’s skeleton; when a fall is confirmed the card turns red, a 30-second alert latch engages, and a “Resolve Latch / Stop Alarm” control appears.

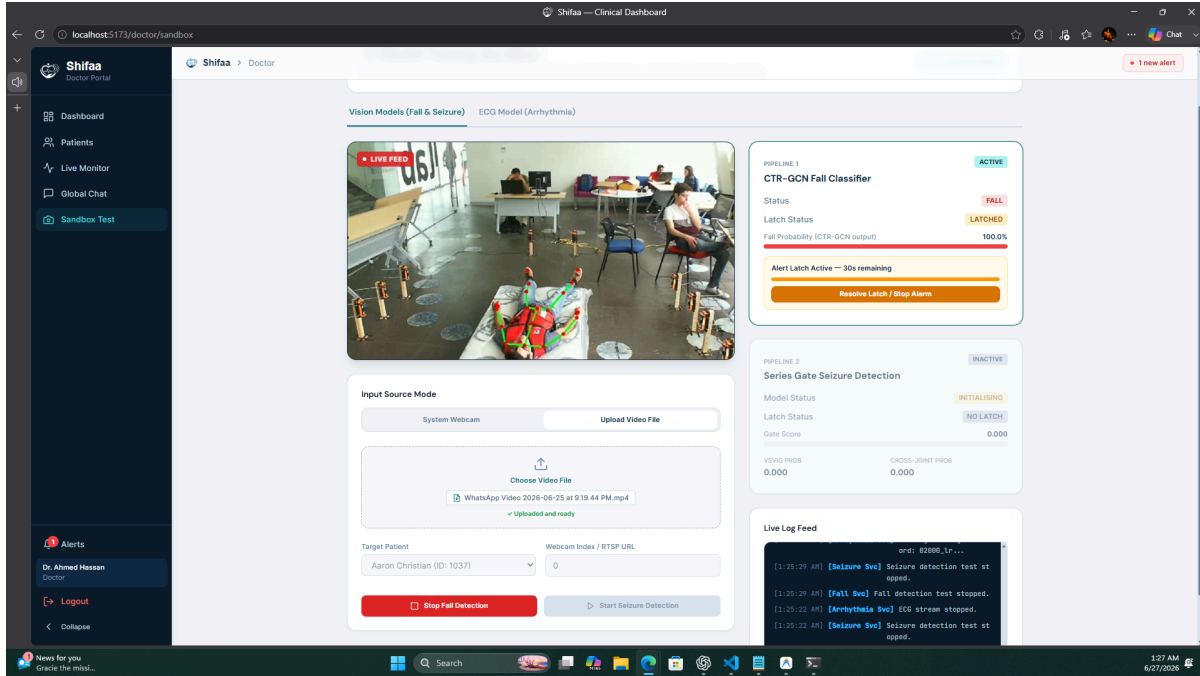


Figure 20: Sandbox, fall detection: CTR-GCN fall confirmed with active alert latch

Seizure case. Selecting seizure detection runs the hybrid series-gate detector on the chosen video. The Series-Gate Seizure Detection panel shows the live gate score and model state, transitioning from INITIALISING to SEIZURE when the fused score crosses the threshold, again holding the alert for 30 seconds via the latch.

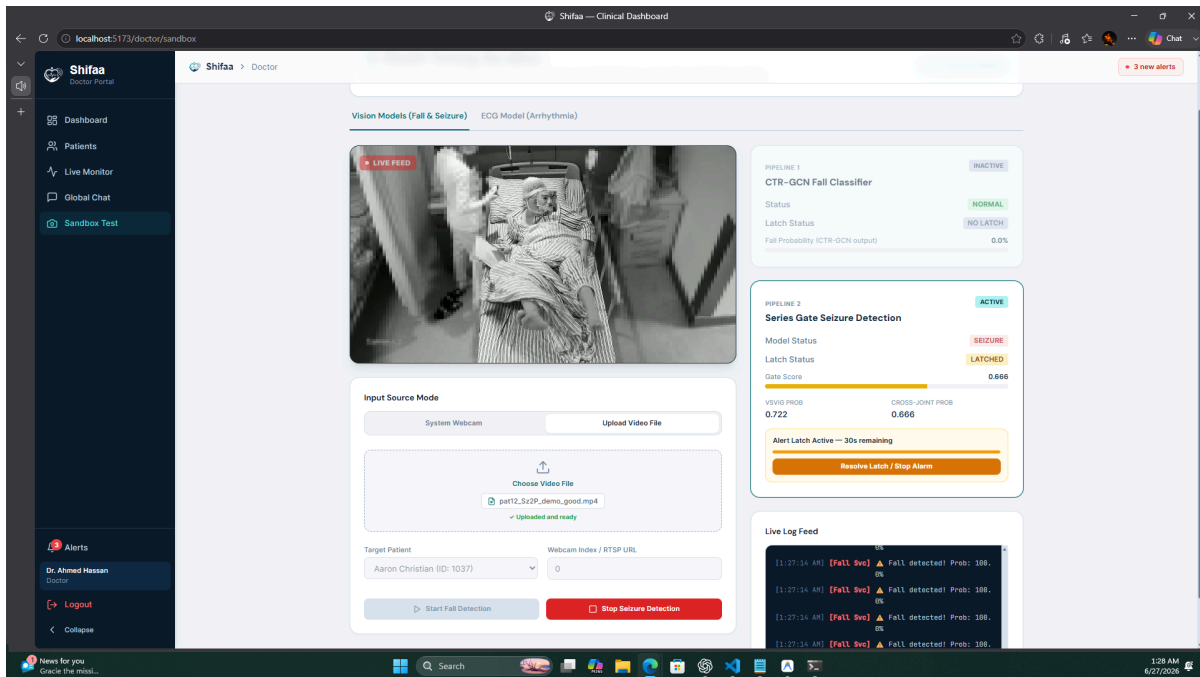


Figure 21: Sandbox, seizure detection: series-gate score crossing threshold with latch

### 5.3.12 Nurse Portal

The nurse signs into a streamlined Nurse Portal with three areas: Dashboard, Patients, and Live Monitor. The Nurse Dashboard is triage-focused: an “Alerts Requiring Attention” feed surfaces the unacknowledged critical alerts for the nurse’s assigned patients (clicking opens the patient; a single action acknowledges), beside a compact list of the nurse’s patients.

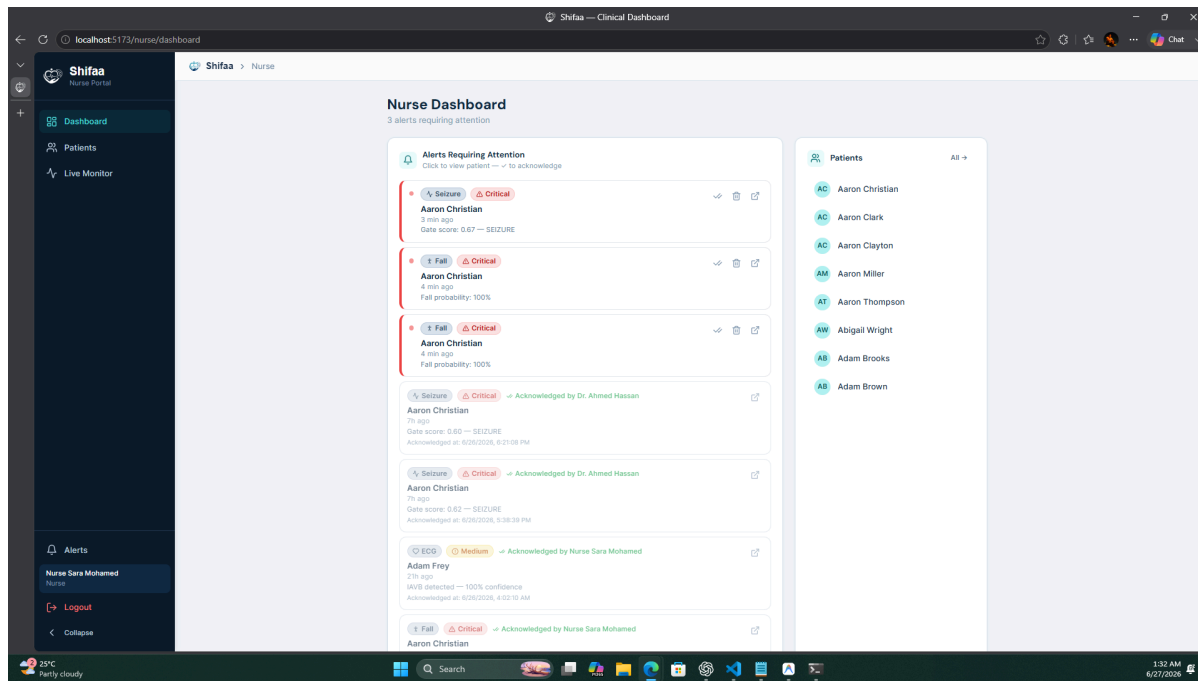


Figure 22: Nurse Dashboard: alerts requiring attention and assigned patients

The nurse’s Live Monitor is the same Ward Central Monitoring Station as the doctor’s, but scoped: a “My Rooms Scoped” filter restricts the grid to only the rooms the nurse is assigned to, enforcing the room-based data isolation described in Chapter 3.

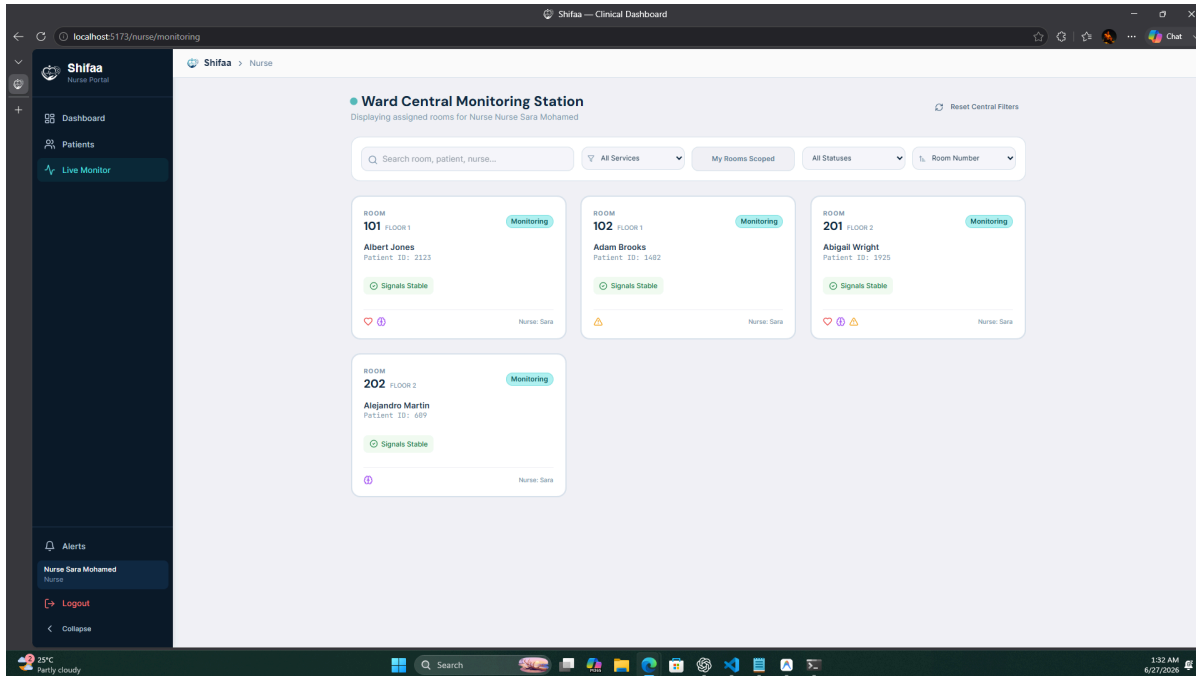


Figure 23: Nurse Live Monitor: ward station scoped to assigned rooms

### 5.3.13 Admin Portal: System, Users, and Rooms

The administrator's portal exposes four areas: System, Users, Live Monitor, and Rooms Config. The System Dashboard reports the operational health of all backend services, the main backend (8000) and the arrhythmia (8001), fall (8002), and seizure (8003) services, including model-loaded and CUDA status and active-session counts, alongside headline counts of doctors, nurses, and chat queries in the last 24 hours.

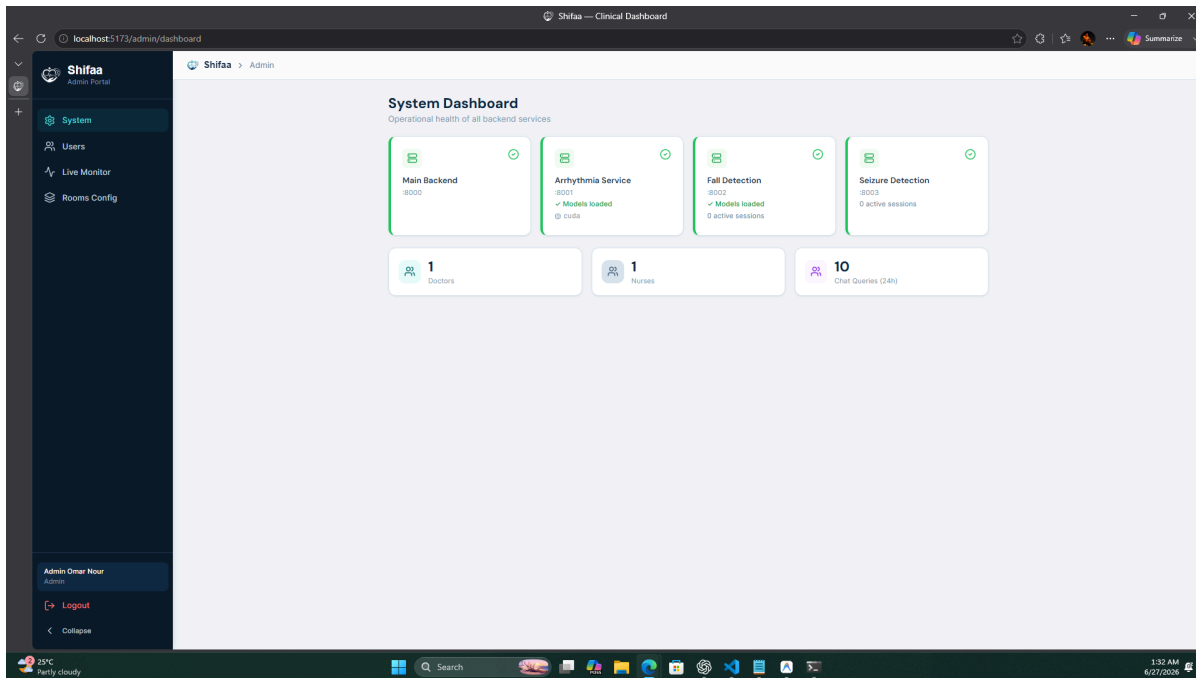


Figure 24: Admin System Dashboard: backend and inference-service health

The User Management screen lists every hospital account with its role, specialty, status, and creation date, and provides controls to add a user, toggle activation, and randomize patient assignments. Tabs filter the list by role (doctors, nurses, admins).

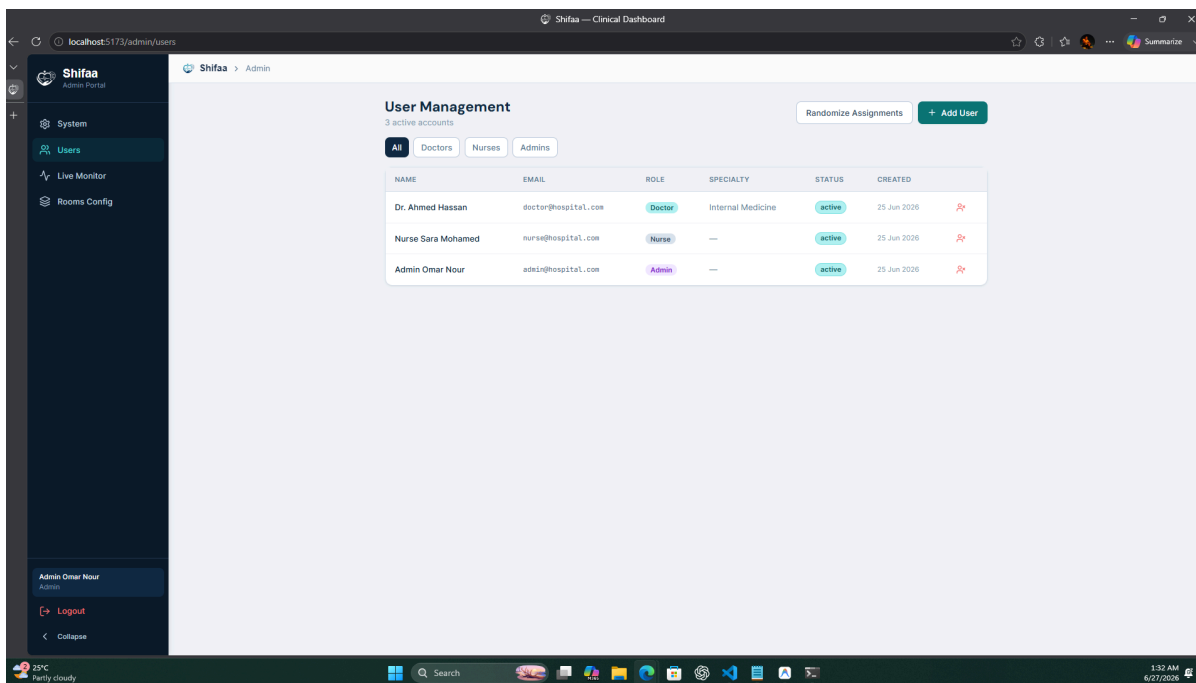
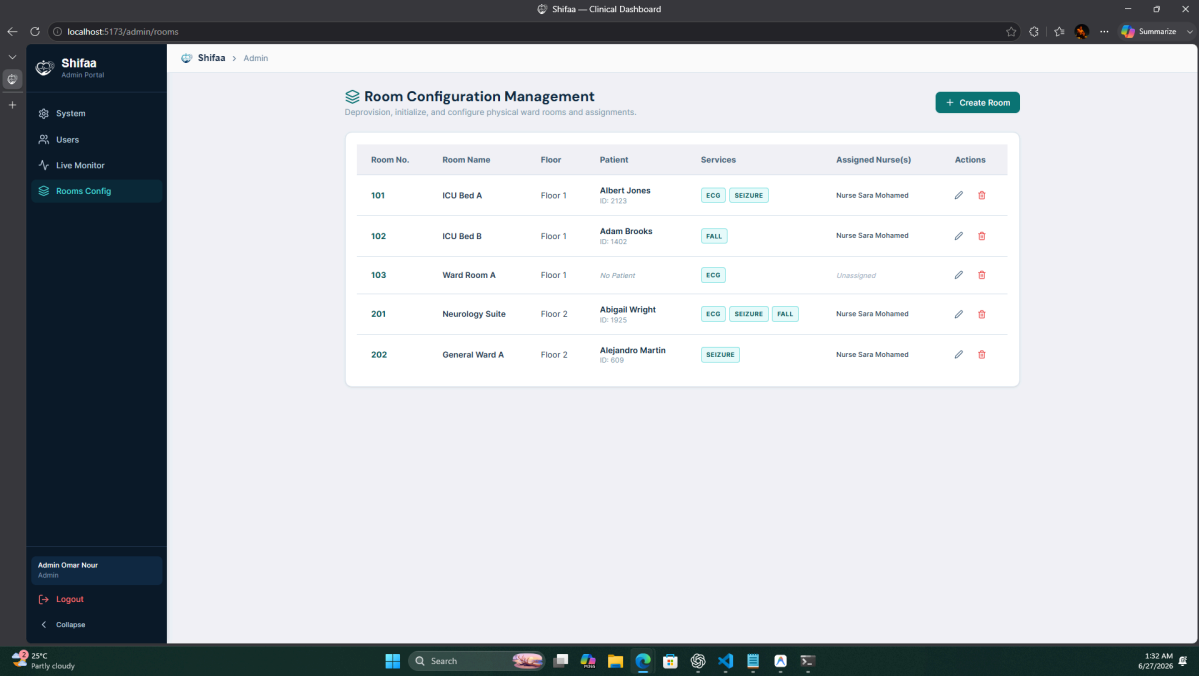


Figure 25: Admin User Management

Room Configuration Management is where the ward is defined. The table lists each room with its number, name, floor, assigned patient, enabled services (ECG, seizure, fall), and assigned nurses, with edit and delete actions and a “Create Room” button.



**Room Configuration Management**  
Deprovision, initialize, and configure physical ward rooms and assignments.

[+ Create Room](#)

| Room No. | Room Name       | Floor   | Patient                    | Services         | Assigned Nurse(s)  | Actions                                     |
|----------|-----------------|---------|----------------------------|------------------|--------------------|---------------------------------------------|
| 101      | ICU Bed A       | Floor 1 | Albert Jones<br>ID: 2153   | ECG SEIZURE      | Nurse Sara Mohamed | <a href="#">Edit</a> <a href="#">Delete</a> |
| 102      | ICU Bed B       | Floor 1 | Adam Brooks<br>ID: 1402    | FALL             | Nurse Sara Mohamed | <a href="#">Edit</a> <a href="#">Delete</a> |
| 103      | Ward Room A     | Floor 1 | No Patient                 | ECG              | Unassigned         | <a href="#">Edit</a> <a href="#">Delete</a> |
| 201      | Neurology Suite | Floor 2 | Abigail Wright<br>ID: 1925 | ECG SEIZURE FALL | Nurse Sara Mohamed | <a href="#">Edit</a> <a href="#">Delete</a> |
| 202      | General Ward A  | Floor 2 | Aljandro Martin<br>ID: 608 | SEIZURE          | Nurse Sara Mohamed | <a href="#">Edit</a> <a href="#">Delete</a> |

*Figure 26: Admin Room Configuration Management*

Creating or editing a room opens a configuration dialog in which the administrator sets the room number, name, and floor, assigns the occupying patient, toggles the enabled monitoring services (ECG/arrhythmia, seizure, fall), and selects the assigned nurses. Saving the room broadcasts a room update so every dashboard reflects the change immediately.



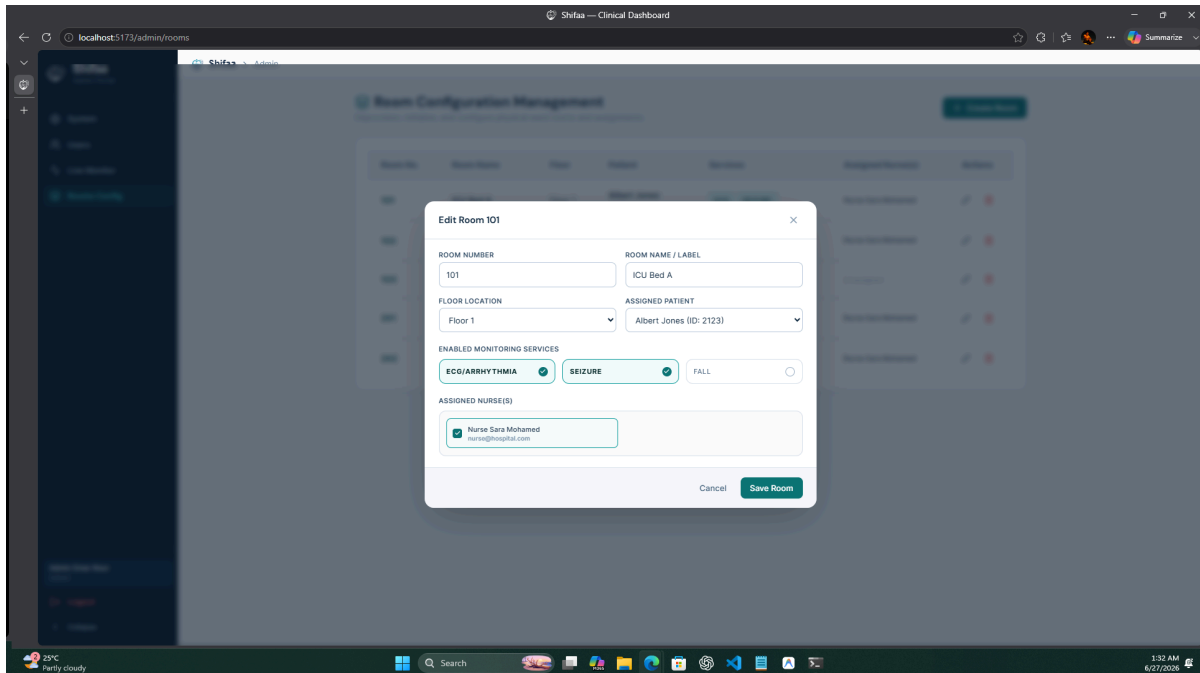


Figure 27: Admin: Edit Room dialog (services and nurse assignment)

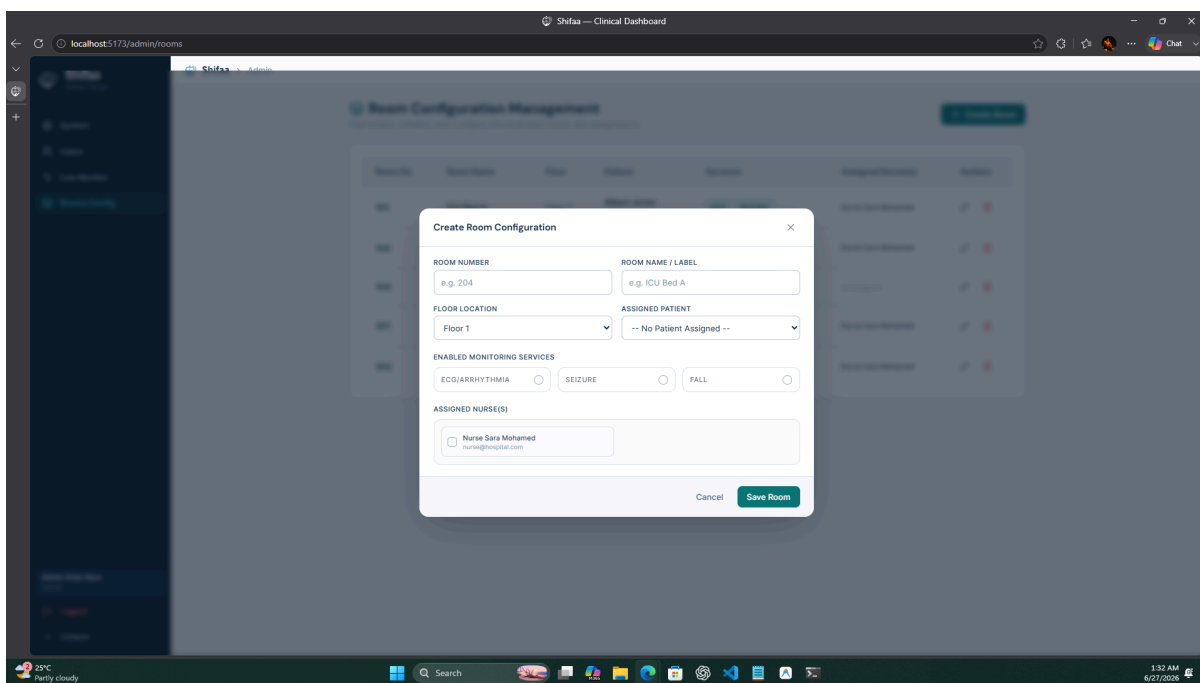


Figure 28: Admin: Create Room dialog

## 5.4 Automated Test Suites

The repository includes a suite of automated tests under the documentation tests folder, covering the chatbot pipeline stages, the API surface, and the monitoring integration. They are executed with pytest.

| Test file                      | What it verifies                                                  |
|--------------------------------|-------------------------------------------------------------------|
| test_intent.py                 | Intent classification across all 8 intents (80 queries).          |
| test_ner.py                    | Medical entity extraction (50 annotated queries).                 |
| test_retriever.py              | Intent-aware retrieval against a seeded database (no LLM needed). |
| test_api.py                    | REST endpoints: auth, patient CRUD, chat contract.                |
| test_integration_monitoring.py | End-to-end monitoring: alert publication and broadcast.           |
| benchmark_pipeline.py          | End-to-end latency benchmark of the chat pipeline.                |

```
# Run the full suite
python -m pytest docs/tests/ -v --tb=short

# Retriever tests only (deterministic, no LLM calls)
python -m pytest docs/tests/test_retriever.py -v

# Latency benchmark
python docs/tests/benchmark_pipeline.py
```

Table 29: Representative retriever test cases

| Test                          | Description                     | Pass condition               |
|-------------------------------|---------------------------------|------------------------------|
| medication_check, all active  | Retrieve all active medications | Exact match on active meds   |
| medication_check, drug filter | Filter by a specific drug       | Only matching drugs returned |
| lab_results, test filter      | Filter by a specific test       | Only that test returned      |
| lab_results, date filter      | Last 3 months only              | All results within window    |
| visit_summary, limit          | Last 3 visits only              | Exactly 3 visits returned    |
| diagnosis_check, condition    | Filter for diabetes             | Only diabetes diagnoses      |
| history_lookup, combined      | All record types                | ≥1 record per table type     |

A representative successful run prints, after the pytest summary, the headline pipeline metrics:

```
===== 42 passed in 18.35s =====
Intent Classification Accuracy: 94.1%
NER F1 Score (macro avg): 94.3%
Retriever Exact Match Rate: 97.0%
```

## 5.5 API Reference Summary

All endpoints except login require an Authorization: Bearer <token> header. The principal endpoints are listed below; the full, interactive contract is served at /docs.

Table 30: Principal API endpoints

| Method & Path                                                            | Purpose                                              |
|--------------------------------------------------------------------------|------------------------------------------------------|
| POST /api/auth/login                                                     | Authenticate; return JWT.                            |
| GET/POST /api/patients                                                   | List / register patients.                            |
| GET /api/patients/{id},<br>.../labs/medications/diagnoses/alerts         | Full record and filtered sub-records.                |
| GET/POST/PUT/DELETE /api/rooms[/]{id}]                                   | Room CRUD (admin write; clinicians read).            |
| POST<br>/api/rooms/{id}/assign-patient/assign-nurses/<br>assign-services | Bind patient, nurses, or services to a room (admin). |
| POST /api/chat, WS /api/ws/chat                                          | Synchronous / streaming chatbot answer.              |
| POST /api/chat/analyze-image                                             | Multimodal image analysis (SSE stream).              |
| POST /api/arrhythmia/analyze, GET<br>.../history/{id}                    | Run the ECG cascade; recent arrhythmia alerts.       |
| GET<br>/api/arrhythmia/sandbox/signals[/]{name}]                         | List / fetch sample ECG signals for testing.         |
| POST/DELETE<br>/api/monitoring/fall/seizure/{start/stop/reset-<br>atch}  | Manage fall/seizure monitoring sessions.             |
| WS /api/ws/alerts                                                        | Live alert + room-update stream for dashboards.      |
| WS /api/ws/camera/{room_id}                                              | Camera frame ingestion for fall detection.           |
| GET /api/dashboard/summary                                               | Aggregated dashboard data.                           |
| PATCH<br>/api/dashboard/alerts/{id}/acknowledge                          | Acknowledge → move alert to history.                 |
| DELETE /api/dashboard/alerts/{id}/cancel                                 | Cancel (delete) an active alert.                     |
| GET POST PATCH /api/admin/users                                          | User management (admin).                             |

For example, a streaming chat exchange sends a query and receives token chunks followed by a final message carrying the intent and sources:

```
→ { "query": "Show the last HbA1c result", "patient_id": 1,  
    "token": "<JWT>" }
```

```

← { "chunk": "The last", "done": false }
← { "chunk": " HbA1c was 8.1%", "done": false }
← { "chunk": "", "done": true, "intent": "lab_results",
    "sources": ["Lab #12: HbA1c = 8.1 % on 2025-11-20"] }

```

## 5.6 Consolidated Benchmark Results

The table below consolidates the quantitative results reported per-component in Chapter 4, giving a single view of the system’s validated performance.

*Table 31: Consolidated system benchmark results*

| Subsystem               | Metric                       | Result                   |
|-------------------------|------------------------------|--------------------------|
| Chatbot, Intent         | Accuracy / Macro-F1          | 94.1% / 0.94             |
| Chatbot, NER            | Macro-F1                     | 94.0%                    |
| Chatbot, Retrieval      | Exact match                  | 97.0%                    |
| Chatbot, Grounding      | Hallucination rate           | ≤ 1%                     |
| ECG, Stage 1 (binary)   | Accuracy / Macro-F1 / AUROC  | 96.10% / 93.42% / 98.26% |
| ECG, Stage 2 (sub-type) | Accuracy / Macro-F1 / AUROC  | 94.92% / 94.71% / 99.43% |
| Fall, CTR-GCN           | Accuracy / Fall-F1 / ROC-AUC | 91.59% / 94.87% / 95.98% |
| Fall, Role classifier   | Validation accuracy          | 80.93%                   |
| Seizure, segments       | AUROC / F1 / Recall          | 96.69% / 90.18% / 98.45% |
| Seizure, runtime        | Steady-state throughput      | ~37.2 FPS                |

*Table 32: Chat end-to-end latency by backend*

| Scenario          | First-token (median) | Total response (median) |
|-------------------|----------------------|-------------------------|
| CPU, local Ollama | 2.1 s                | 15.2 s                  |
| GPU, local Ollama | 0.8 s                | 5.5 s                   |
| Cloud model       | 0.5 s                | 4.2 s                   |

Taken together, these results show every subsystem meeting or exceeding its target: above-90% accuracy on all classification tasks, sub-second first-token latency on a GPU, a hallucination rate within the 1% safety budget, and a real-time-capable seizure detector.

## 5.7 Troubleshooting

Common issues encountered during installation and testing, and their resolutions, are listed below.

| Problem                                       | Likely cause                                         | Resolution                                                    |
|-----------------------------------------------|------------------------------------------------------|---------------------------------------------------------------|
| Database errors on first run                  | DATABASE_URL points at an unreachable server         | Unset it to use the default SQLite file, or start PostgreSQL. |
| 500 error on login                            | bcrypt backend version issue                         | Pin bcrypt==4.0.1.                                            |
| 401 on all requests                           | JWT expired or missing                               | Re-login to obtain a fresh token.                             |
| WebSocket refuses to connect                  | Backend not running / wrong URL                      | Verify Uvicorn and VITE_WS_URL.                               |
| LLM timeout                                   | Model not loaded                                     | Run “ollama pull medgemma1.5:latest”.                         |
| Blank patient/room list                       | Empty database                                       | Run “python -m backend.seed”.                                 |
| Seizure status stuck on SEIZURE               | 30-second alert latch active                         | Wait for the latch to clear.                                  |
| Seizure: ‘No providers found’ / pipe Errno 22 | C++ runtime built without CUDA EP, or engine crashed | Rebuild in Release with CUDA provider DLLs alongside the exe. |
| A monitoring service is offline               | Inference service not started on its port            | Run start_all_services or launch the service on 8001–8003.    |

## 5.8 Production Deployment

For production, the backend is served by Gunicorn with Uvicorn workers behind an Nginx reverse proxy that terminates TLS, serves the compiled frontend assets, and proxies API and WebSocket traffic to the gateway. CORS must be restricted to the real frontend origin, and the JWT secret replaced with a long random value.

```
# Backend (production)
gunicorn backend.main:app -w 4 -k uvicorn.workers.UvicornWorker \
    --bind 0.0.0.0:8000

# Nginx (excerpt)
location / {      root /path/to/frontend/dist; try_files $uri /index.html; }
location /api/ {  proxy_pass http://127.0.0.1:8000;
                  proxy_http_version 1.1;
                  proxy_set_header Upgrade $http_upgrade;
                  proxy_set_header Connection 'upgrade'; }
```

With these steps the platform runs as a set of independent, individually scalable services. The next chapter reflects on the project's outcomes and outlines future work.

## Chapter Six

# Conclusion and Future Work

---

6.1 Conclusion

6.2 Future Work

# Chapter 6: Conclusion and Future Work

## 6.1 Conclusion

Shifaa set out to address two enduring clinical problems (the slow, fragmented retrieval of patient information and the difficulty of continuously monitoring patients for life-threatening events) within a single, privacy-respecting platform. The project achieved this by combining a grounded, multi-stage Large Language Model assistant with three real-time AI monitoring subsystems, all unified behind a role-aware microservices backend and a modern React interface.

The conversational assistant demonstrates that a general-purpose medical LLM, MedGemma 1.5, can be turned into a trustworthy clinical tool by constraining it with strict prompts, structured intent and entity extraction, and database-grounded retrieval. It answered queries with 94.1% intent accuracy, 94.0% entity-extraction F1, and a hallucination rate within a 1% safety budget, while streaming its reasoning transparently to the clinician. The three monitoring subsystems each reached strong, validated performance: 96.10% and 94.92% accuracy on the two ECG cascade stages, a 94.87% fall F1-score with 98.83% recall, and a real-time native seizure detector sustaining ~37 frames per second with a 96.69% segment AUROC. Crucially, the entire stack can run locally, keeping sensitive patient data on-premises.

Beyond the numbers, the project delivered a coherent software system: a normalized clinical database seeded with realistic synthetic data, a secure JWT/RBAC gateway organized around hospital rooms, an alert hub that guarantees every alert is persisted, broadcast, and carried through an explicit acknowledge-and-archive lifecycle, and a polished interface spanning login, room board, patient management, streaming chat, ECG visualization, and live monitoring. The advantages and limitations of the resulting system are summarized below.

Table 33: Advantages and limitations of Shifaa

| Advantages                                                            | Limitations                                                                                                                        |
|-----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Unifies four clinical capabilities in one room-centered platform.     | The chatbot is stateless; it has no multi-turn conversational memory.                                                              |
| Grounded answers with citations; $\leq 1\%$ hallucination.            | Year-only date ranges (e.g. “2024”) are not yet parsed to a cutoff.                                                                |
| Real-time native seizure runtime (~37 FPS) with graceful degradation. | Seizure detection covers motor seizures only and still has an elevated false-alert rate; validated on a single-institution cohort. |



| Advantages                                                          | Limitations                                                                                                                                                                                                             |
|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Microservices on dedicated ports isolate heavy AI from the gateway. | Vision/seizure services require a CUDA GPU; the seizure runtime is Windows-only (named-pipe IPC).                                                                                                                       |
| Room-based RBAC with nurse scoping and a full alert lifecycle.      | Doctor access is broad (all rooms) rather than ward-scoped.                                                                                                                                                             |
| SQLite by default for zero-config deployment.                       | SQLite limits horizontal scale; production needs PostgreSQL. CORS is open in development.                                                                                                                               |
| Supports fully offline                                              | air-gapped deployment via local model files.   The Python sidecar currently sets HuggingFace offline mode unconditionally, so the ViViT checkpoint must be cached once with network access before first air-gapped run. |

## 6.2 Future Work

Several open directions would extend the platform in the coming year:

- **Conversational memory.** Add multi-turn context so the assistant can resolve follow-up questions that refer to earlier answers.
- **Richer temporal parsing.** Extend date-range handling to absolute years and arbitrary ranges, and add trend visualizations for lab values.
- **Domain fine-tuning.** Fine-tune MedGemma on the logged, anonymized query–answer pairs to further raise intent and grounding accuracy.
- **Vitals ingestion and rule engine.** Complete the real-time vitals pipeline (heart rate, SpO<sub>2</sub>, temperature, glucose) with threshold-based alerting alongside the ML detectors.
- **Cross-site validation.** Validate the seizure and fall detectors on multi-institution data to establish generalization.
- **Reduce seizure false alerts.** Train on a larger, more diverse dataset and apply lightweight (e.g. LoRA) fine-tuning to handle cross-site domain shift, and propagate the requested clinical mode all the way into the native C++ runtime.
- **Database migration.** Move from the default SQLite file to a server database (PostgreSQL) for concurrent, hospital-scale deployment.

- **Cross-platform seizure runtime.** Replace the Windows named-pipe IPC with a portable transport so the native runtime can run on Linux edge devices.
- **Reduce detection latency.** Optimize the ViViT backbone to lower inference complexity, enabling more frequent Cross-Joint updates and further reducing Latency to Clinical Onset without added hardware cost.
- **Non-motor seizure detection.** Extend beyond visible motor manifestations to absence and other non-motor seizure types, likely requiring new feature types such as facial action units or EEG-guided video analysis.
- **Ward-scoped doctor access.** Constrain doctor access to explicit wards or rooms rather than the whole hospital.
- **Edge deployment.** Quantize and deploy the vision services to bedside edge devices to reduce camera-stream bandwidth and latency.
- **Production hardening.** Restrict CORS, add rate limiting and refresh tokens, containerize all services, and add observability and automated CI.
- **Clinical study.** Conduct a usability and time-savings study with clinicians to quantify the platform's impact on real workflows.

These extensions would move Shifaa from a strong academic prototype toward a deployable clinical product, while preserving the core principles that guided its design: accuracy, transparency, and the privacy of patient data.

# Tools

The tools used to develop, train, and run Shifaa are grouped below into hardware and software.

## Hardware Tools

| Tool                                                | Purpose                                                                         |
|-----------------------------------------------------|---------------------------------------------------------------------------------|
| NVIDIA CUDA GPU (e.g. RTX 30/50-series laptop GPU)  | Training the ECG/vision models and real-time inference for the vision services. |
| Multi-core CPU workstation (8-core, 16 GB RAM, SSD) | Backend, database, data import, and CPU-only LLM fallback.                      |
| IP / webcam camera                                  | Video source for the fall- and seizure-detection pipelines.                     |
| 12-lead ECG acquisition (or recorded signals)       | Input to the arrhythmia cascade ( $5000 \times 12$ at 500 Hz).                  |
| IMU / accelerometer (training only)                 | Impact localization for fall-window alignment.                                  |

## Software Tools

| Category               | Tools                                                                                         |
|------------------------|-----------------------------------------------------------------------------------------------|
| Languages              | Python 3.10, JavaScript (ES2022), SQL.                                                        |
| Backend                | FastAPI, Uvicorn, SQLAlchemy, Pydantic, python-jose (JWT), passlib/bcrypt, Alembic.           |
| Database               | SQLite (default, bundled), PostgreSQL 14+ (optional), psycopg2.                               |
| LLM                    | MedGemma 1.5 via Ollama; OpenAI-compatible client SDK.                                        |
| Deep learning          | PyTorch, Ultralytics YOLOv8, MediaPipe, timm, transformers, OpenPose, ONNX Runtime.           |
| Native seizure runtime | C++17, ONNX Runtime (CUDA/TensorRT), CMake, Visual Studio 2022, OpenCV, Win32 named-pipe IPC. |
| Data                   | pandas, Synthea, wfdb (ECG), Faker (seeding).                                                 |
| Frontend               | React 18, Vite, Tailwind CSS, Zustand, React Router, Axios, Recharts, Lucide.                 |
| Testing                | pytest, httpx.                                                                                |
| Tooling                | Git & GitHub, Conda, Node.js/npm, VS Code.                                                    |

## Chapter Appendices

# Appendices

---

Appendix A, Environment Configuration Reference

Appendix B, Detailed API Examples

Appendix C, Glossary of Clinical Terms

## Appendix A: Environment Configuration Reference

The backend is configured through environment variables, typically supplied in a `.env` file. The complete set is listed below.

| Variable                      | Required  | Default                   | Description                                               |
|-------------------------------|-----------|---------------------------|-----------------------------------------------------------|
| DATABASE_URL                  | No        | sqlite:///./medical_db.db | DB connection string; point at PostgreSQL for production. |
| SECRET_KEY                    | Yes       | -                         | JWT signing key (long random string).                     |
| ALGORITHM                     | No        | HS256                     | JWT signature algorithm.                                  |
| ACCESS_TOKEN_EXPIRE_MINUTES   | No        | 60                        | Token lifetime in minutes.                                |
| LLM_BACKEND                   | No        | ollama                    | 'ollama' (local) or 'openai' (cloud).                     |
| OLLAMA_MODEL                  | No        | medgemma1.5:latest        | Model name served by Ollama.                              |
| OPENAI_API_KEY                | If openai | -                         | API key for the cloud backend.                            |
| ARRHYTHMIA_SERVICE_URL        | No        | http://localhost:8001     | Arrhythmia inference service base URL.                    |
| FALL_DETECTION_SERVICE_URL    | No        | http://localhost:8002     | Fall detection service base URL.                          |
| SEIZURE_DETECTION_SERVICE_URL | No        | http://localhost:8003     | Seizure detection service base URL.                       |

The React frontend uses three variables, supplied in `frontend-react/.env`:

| Variable     | Description                                                       |
|--------------|-------------------------------------------------------------------|
| VITE_API_URL | Backend REST base URL (e.g. <code>http://localhost:8000</code> ). |

| Variable       | Description                                              |
|----------------|----------------------------------------------------------|
| VITE_WS_URL    | Backend WebSocket base URL (e.g. ws://localhost:8000).   |
| VITE_DATA_MODE | 'mock' for offline demo, 'live' to use the real backend. |

## Appendix B: Detailed API Examples

This appendix gives representative request and response payloads for the principal endpoints. All examples assume a valid bearer token unless stated otherwise.

### B.1 Login

```
POST /api/auth/login
→ { "email": "doctor@hospital.com", "password": "doctor123" }
← 200 { "access_token": "eyJhbGciOiJIUzI1NiIsInR...",
      "token_type": "bearer" }
← 401 { "detail": "Incorrect email or password" }
```

### B.2 Patient Record

```
GET /api/patients/1
← 200 {
  "id": 1, "name": "Ahmad Hassan", "dob": "1978-04-12",
  "gender": "Male", "blood_type": "O+",
  "diagnoses": [ { "description": "Type 2 Diabetes Mellitus",
                  "icd10_code": "E11", "is_active": true } ],
  "medications": [ ... ], "lab_results": [ ... ],
  "allergies": [ ... ], "visits": [ ... ] }
```

### B.3 Arrhythmia Analysis

```
POST /api/arrhythmia/analyze
→ { "patient_id": 1, "signal": [...],[...], // 5000 x 12
    "qrs7": [ ... ] } // optional
← 200 {
  "stage1": "Abnormal", "stage1_confidence": 0.97,
  "stage2_class": "AF", "stage2_confidence": 0.91,
  "all_probabilities": { "AF": 0.91, "IAVB": 0.04, ... },
  "alert_created": true, "alert_id": 42 }
```

### B.4 Create and Configure a Room

```
POST /api/rooms (admin)
→ { "room_number": "4B", "room_name": "Neuro 4B",
    "floor": "4", "patient_id": 1,
    "services": ["ecg", "seizure", "fall"],
    "nurse_ids": [3] }
← 201 { "id": 7, "room_number": "4B", "monitoring_status": "Monitoring",
      "services": ["ecg", "seizure", "fall"], "nurses": [...],
      "active_alerts": [], "risk_score": 0.0 }
```

## B.5 Acknowledge an Alert

```
PATCH /api/dashboard/alerts/42/acknowledge
→ { "acknowledged_by": 1 }
← 200 { "id": 42, "moved_to_history": true,
      "acknowledged_at": "2026-06-27T10:15:00Z" }
# broadcasts an 'acknowledge' event; the active alert is removed
# and a permanent row is written to alert_history.
```

## B.6 Dashboard Summary

```
GET /api/dashboard/summary
← 200 {
  "alert_counts": [ { "alert_type": "fall", "count": 2 } ],
  "recent_alerts": [ ... ],
  "service_health": { "arrhythmia": { ... }, "fall": { ... },
    "seizure": { ... } },
  "active_sessions": { "fall": [...], "seizure": [...] },
  "chat_count_24h": 17 }
```



## Appendix C: Glossary of Clinical Terms

A brief glossary of the clinical terms used throughout this documentation.

| Term                           | Definition                                                                               |
|--------------------------------|------------------------------------------------------------------------------------------|
| Arrhythmia                     | An abnormal heart rhythm, too fast, too slow, or irregular.                              |
| Atrial Fibrillation (AF)       | A common irregular, often rapid rhythm originating in the atria.                         |
| First-Degree AV Block (IAVB)   | Delayed electrical conduction between atria and ventricles.                              |
| Sinus Bradycardia (SB)         | A regular but abnormally slow heart rate.                                                |
| Sinus Tachycardia (STach)      | A regular but abnormally fast heart rate.                                                |
| HbA1c                          | Glycated haemoglobin; reflects average blood glucose over ~3 months.                     |
| ICD-10                         | Standard international codes for diagnoses and diseases.                                 |
| SNOMED CT                      | A comprehensive, standardized clinical terminology.                                      |
| Epilepsy Monitoring Unit (EMU) | A hospital unit for video-EEG observation of seizures.                                   |
| Tonic-clonic seizure           | A generalized seizure with stiffening then rhythmic jerking.                             |
| Focal seizure                  | A seizure arising from one localized brain region.                                       |
| Vital signs                    | Core physiological measures: heart rate, SpO <sub>2</sub> , temperature, blood pressure. |
| Triage                         | Prioritizing patients by the urgency of their condition.                                 |

# References

*References are listed in IEEE style in order of their first citation in the text. Online sources are listed with their access date.*

- [1] World Health Organization, “Falls,” WHO Fact Sheets, Apr. 26, 2021. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/falls> (accessed Jun. 27, 2026).
- [2] Agency for Healthcare Research and Quality, “Preventing Falls in Hospitals: A Toolkit for Improving Quality of Care,” AHRQ, Rockville, MD, USA. [Online]. Available: <https://www.ahrq.gov/patient-safety/settings/hospital/fall-prevention/toolkit/index.html> (accessed Jun. 27, 2026).
- [3] World Health Organization, “Epilepsy,” WHO Fact Sheets, Feb. 7, 2024. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/epilepsy> (accessed Jun. 27, 2026).
- [4] World Health Organization, “Cardiovascular diseases (CVDs),” WHO Fact Sheets, Jun. 11, 2021. [Online]. Available: [https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-\(cvds\)](https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds)) (accessed Jun. 27, 2026).
- [5] A. Vaswani et al., “Attention is all you need,” in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), Long Beach, CA, USA, 2017, pp. 5998–6008.
- [6] A. Sellergren et al., “MedGemma 1.5 technical report,” arXiv:2604.05081, 2026.
- [7] Google DeepMind, “Gemma 3 technical report,” arXiv:2503.19786, 2025.
- [8] P. Wagner et al., “PTB-XL, a large publicly available electrocardiography dataset,” Sci. Data, vol. 7, art. 154, 2020.
- [9] E. A. Perez Alday et al., “Classification of 12-lead ECGs: the PhysioNet/Computing in Cardiology Challenge 2020,” Physiol. Meas., vol. 41, no. 12, art. 124003, 2020.
- [10] A. H. Ribeiro et al., “Automatic diagnosis of the 12-lead ECG using a deep neural network,” Nat. Commun., vol. 11, art. 1760, 2020.
- [11] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” software, 2023. [Online]. Available: <https://docs.ultralytics.com> (accessed Jun. 27, 2026).
- [12] Y. Zhang et al., “ByteTrack: Multi-object tracking by associating every detection box,” in Proc. Eur. Conf. Comput. Vis. (ECCV), Tel Aviv, Israel, 2022, pp. 1–21.
- [13] A. Howard et al., “Searching for MobileNetV3,” in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), Seoul, South Korea, 2019, pp. 1314–1324.
- [14] C. Lugaresi et al., “MediaPipe: A framework for building perception pipelines,” arXiv:1906.08172, 2019.
- [15] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, “OpenPose: Realtime multi-person 2D pose estimation using part affinity fields,” IEEE Trans. Pattern Anal. Mach. Intell., vol. 43, no. 1, pp. 172–186, Jan. 2021.

- [16] Y. Chen, Z. Zhang, C. Yuan, B. Li, Y. Deng, and W. Hu, “Channel-wise topology refinement graph convolution for skeleton-based action recognition,” in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), Montreal, QC, Canada, 2021, pp. 13359–13368.
- [17] Y. Xu et al., “VSViG: Real-time video-based seizure detection via skeleton-based spatiotemporal ViG,” in Proc. Eur. Conf. Comput. Vis. (ECCV), Milan, Italy, 2024.
- [18] A. Arnab, M. Dehghani, G. Heigold, C. Sun, M. Lučić, and C. Schmid, “ViViT: A video vision transformer,” in Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV), Montreal, QC, Canada, 2021, pp. 6836–6846.
- [19] O. Zamzam et al., “Learning cross-joint attention for generalizable video-based seizure detection,” arXiv:2603.23757, 2026.
- [20] J. Walonoski et al., “Synthea: An approach, method, and software mechanism for generating synthetic patients and the synthetic electronic health care record,” J. Amer. Med. Inform. Assoc., vol. 25, no. 3, pp. 230–238, Mar. 2018.
- [21] Microsoft, “ONNX Runtime: High-performance inference engine for machine learning models,” software, 2024. [Online]. Available: <https://onnxruntime.ai> (accessed Jun. 27, 2026).
- [22] S. Ramírez, “FastAPI,” software framework documentation. [Online]. Available: <https://fastapi.tiangolo.com> (accessed Jun. 27, 2026).
- [23] Meta Open Source, “React,” software library documentation. [Online]. Available: <https://react.dev> (accessed Jun. 27, 2026).
- [24] PostgreSQL Global Development Group, “PostgreSQL documentation.” [Online]. Available: <https://www.postgresql.org/docs> (accessed Jun. 27, 2026).
- [25] Ollama, “Ollama: Run large language models locally,” software. [Online]. Available: <https://ollama.com> (accessed Jun. 27, 2026).
- [26] Google, “MedGemma,” Health AI Developer Foundations documentation. [Online]. Available: <https://developers.google.com/health-ai-developer-foundations/medgemma> (accessed Jun. 27, 2026).
- [27] D. Osokin, “Lightweight human pose estimation (PyTorch),” software repository. [Online]. Available: <https://github.com/Daniil-Osokin/lightweight-human-pose-estimation.pytorch> (accessed Jun. 27, 2026).
- [28] Shifaa Team, “Medical-History-Chatbot (Shifaa) source repository.” [Online]. Available: <https://github.com/omarelnokrashy/Medical-History-Chatbot> (accessed Jun. 27, 2026).